



ENGENHARIA DE SOFTWARE

MICHEL CESAR DOS SANTOS



ACESSE AQUI ESTE
MATERIAL DIGITAL!

EXPEDIENTE

Coordenador(a) de Conteúdo

Flavia Lumi Matuzawa

Projeto Gráfico e Capa

Arthur Cantareli Silva

Editoração

Nivaldo Vilela de Oliveira Junior

Design Educacional

Barbara Tamires Neves

Revisão Textual

Cindy Luca

Ilustração

Eduardo Aparecido e Geison Oliveira

Fotos

Shutterstock , Envato e Unsplash

FICHA CATALOGRÁFICA

Ng64 Núcleo de Educação a Distância. **SANTOS**, Michel Cesar dos
Engenharia de Software / Michel Cesar dos Santos. - Florianópolis,
SC: Arquê, 2025.

180 p.

ISBN papel 978-65-279-0699-5

ISBN digital 978-65-279-0696-4

1. Engenharia 2. Software 3. EaD. I. Título.

CDD - 005.10685

Bibliotecária: Leila Regina do Nascimento - CRB- 9/1722.

Ficha catalográfica elaborada de acordo com os dados fornecidos pelo(a) autor(a).

Impresso por:

AVALIE ESTE LIVRO!



CRIAR MOMENTOS DE APRENDIZAGENS INESQUECÍVEIS É O NOSSO OBJETIVO E POR ISSO, **GOSTARIAMOS DE SABER COMO FOI SUA EXPERIÊNCIA.**

Conta para nós! leva menos de 2 minutos. Vamos lá?!

DIGITE O CÓDIGO

 02511909

 Aa

RESPONDA A PESQUISA









RECURSOS DE IMERSÃO



PENSANDO JUNTOS

Este item corresponde a uma proposta de reflexão que pode ser apresentada por meio de uma frase, um trecho breve ou uma pergunta.



APROFUNDANDO

Utilizado para temas, assuntos ou conceitos avançados, levando ao aprofundamento do que está sendo trabalhado naquele momento do texto.



EU INDICO

Utilizado para agregar um conteúdo externo.



ZOOM NO CONHECIMENTO

Utilizado para desmistificar pontos que possam gerar confusão sobre o tema. Após o texto trazer a explicação, essa interlocução pode trazer pontos adicionais que contribuam para que o estudante não fique com dúvidas sobre o tema.

PRODUTOS AUDIOVISUAIS

Os elementos abaixo possuem recursos audiovisuais. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.



PLAY NO CONHECIMENTO

Professores especialistas e convidados, ampliando as discussões sobre os temas por meio de fantásticos podcasts.



EM FOCO

Utilizado para aprofundar o conhecimento em conteúdos relevantes utilizando uma linguagem audiovisual.



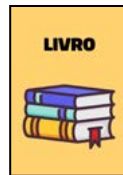
INDICAÇÃO DE FILME

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de filmes que se conectam com o tema do conteúdo.



INDICAÇÃO DE LIVRO

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de livros que agregarão muito na sua vida profissional.



CAMINHOS DE APRENDIZAGEM

7

UNIDADE 1

INTRODUÇÃO À ENGENHARIA DE SOFTWARE	8
---	---

25

UNIDADE 2

REQUISITOS DE SOFTWARE	26
MODELAGEM DE SISTEMAS	44

61

UNIDADE 3

ORIENTAÇÃO A OBJETOS	62
DIAGRAMAS DE SISTEMA	80

101

UNIDADE 4

IMPLEMENTAÇÃO DE SOFTWARE	102
GERENCIAMENTO DE SOFTWARE	122

143

UNIDADE 5

METODOLOGIAS ÁGEIS.	144
ESTUDO DE CASO DE ENGENHARIA DE SOFTWARE	164

unidade





TEMA DE APRENDIZAGEM 1

INTRODUÇÃO À ENGENHARIA DE SOFTWARE

MINHAS METAS

- Entender o que é a Engenharia de Software.
- Conhecer os elementos da Engenharia de Software.
- Analisar os tipos de software.
- Compreender os processos de software.
- Posicionar-se na linha do tempo.
- Identificar a necessidade da Engenharia de Software.
- Vislumbrar oportunidades para a Engenharia de Software no mercado de trabalho.

INICIE SUA JORNADA

Empresas e organizações enfrentam **desafios constantes**, sempre em busca de soluções para fazer a organização se diferenciar. Assim, frequentemente, surge o seguinte questionamento: como deixar a **empresa mais competitiva**?

Grande parte das vezes, essa busca passa pela necessidade de desenvolvimento de novos softwares, momento em que a Engenharia de Software se mostra fundamental, porque, munida de técnicas e métodos, ajudará a estruturar as soluções de software que surgirem.

Essa área envolve processos meticulosos de planejamento, design, implementação, testes e manutenção de sistemas de software, sendo muito mais do que a simples digitação de linhas de código.

Devemos considerar que a Engenharia de Software é um campo amplo e repleto de possibilidades, área que nos convida a trilhar um caminho de aprendizado muito interessante. Pense no seguinte cenário: um funcionário do departamento de recursos humanos precisa de um **software para controlar o fluxo de entrevistas** e contratações. Como você, estudante, levantaria as necessidades e as informações acerca desse software?

Refleta a respeito dos benefícios que esse software traria para o setor que solicitou o desenvolvimento.



PLAY NO CONHECIMENTO

As metodologias de desenvolvimento são cruciais para um bom projeto de desenvolvimento de software, pois passarão por uma fase de planejamento e forte atuação da Engenharia de Software levando a um produto de maior qualidade. Neste podcast, abordaremos o tema relativo às metodologias de desenvolvimento de software, incluindo *Scrum* e *Extreme Programming*. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

No vídeo a seguir, o canal Profissão Futuro nos explica como funciona a profissão de engenheiro de software. Eu o convido a assisti-lo e a recordar alguns pontos essenciais da profissão de engenheiro de software.

<https://www.youtube.com/watch?v=wVHalfEFRhQ>

DESENVOLVA SEU POTENCIAL

A área de Engenharia de Software é uma área fascinante e desafiadora. Nela, a atenção aos detalhes e aos processos é primordial. Por isso, faz-se necessário o entendimento do ponto do tempo em que estamos. Para tanto, é importante traçarmos uma linha do tempo dessa área. É o que veremos adiante.



INDICAÇÃO DE LIVRO

Engenharia de Software

Com mais de três décadas de liderança de mercado, a obra *Engenharia de Software* chega à 8ª edição como o mais abrangente guia dessa importante área.



A Engenharia de Software é muito mais do que um conjunto de processos e técnicas: é um modo de pensar e agir, ajudando a desvendar problemas complexos.

CONCEITOS BÁSICOS E APLICAÇÕES DE SOFTWARE

O software e a respectiva aplicação são importantes, pois possibilitam a automação de processos e tarefas. Isso aumenta a eficiência, reduz o esforço humano e acelera os resultados.

Segundo Tom DeMarco (1995 *apud* Pressman, 2016, p. 4):

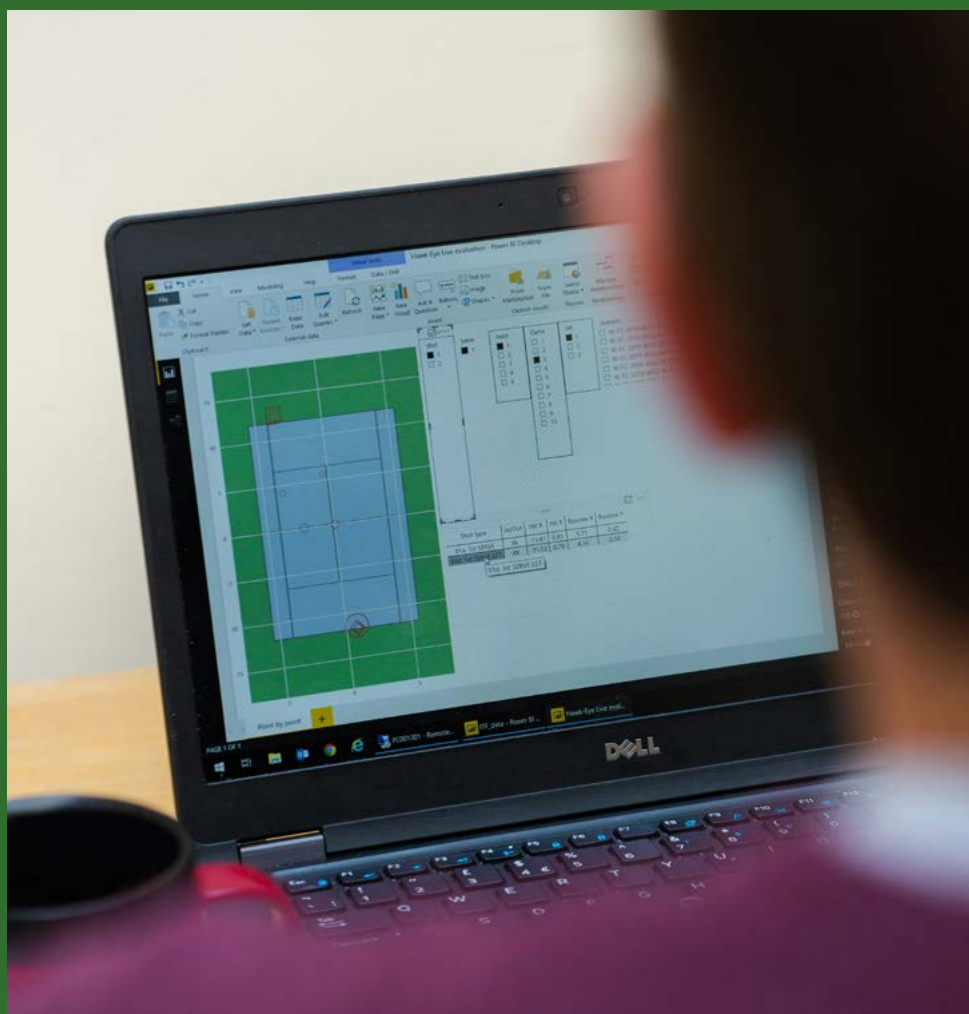


[...] software consiste em: (1) instruções (programas de computador) que, quando executadas, fornecem características, funções e desempenho desejados; (2) estruturas de dados que possibilitam aos programas manipular informações adequadamente; e (3) informação descritiva, tanto na forma impressa quanto na virtual, descrevendo a operação e o uso dos programas.

Os softwares podem ser categorizados em diversas áreas, desde aplicativos de comunicação, como e-mails, chats, entre outros, passando por banco de dados, que facilitam a tomada de decisões, chegando até a área de entretenimento, com jogos e músicas. Todas essas áreas geram uma categorização por tipo de software. Poderíamos agrupá-los da seguinte forma:

SOFTWARE DE SISTEMA:
Sistema operacional, como o Windows, ou <i>drivers</i> , como driver de comunicação com a placa de vídeo.
SOFTWARE DE APLICAÇÃO:
Programa que ajuda o usuário a realizar tarefas específicas, como aplicativos de escritório, navegadores WEB e software de comunicação.
SOFTWARE DE DESENVOLVIMENTO:
Ferramenta utilizada para criar outros softwares, como o Visual Studio ou outras IDEs.

No nosso dia a dia, é cada vez mais crescente a utilização de softwares e podemos encontrá-los nos mais diversos segmentos. Citamos, a seguir, as divisões que são abrangentes e nos dão uma amostra de como aplicativos e sistemas estão ramificados, tornando-nos dependentes dessa tecnologia.



Aplicativos Móveis: utilizamos muito o celular. Aplicativos, como WhatsApp e Google Maps, são cada vez mais essenciais.

Softwares para computadores pessoais: softwares, como editores de texto, planilhas eletrônicas e navegação na internet, são essenciais.

Sistemas empresariais: sistemas como *Enterprise Resource Planning* (ERP).

Planning ou CRM: *Customer Relationship Managements* são ótimos exemplos de sistemas empresariais.

Com essa visão, evidenciamos que os softwares estão cada vez mais propagados em nosso mundo. Isso faz com que os usuários se tornem cada vez mais dependentes dessa utilização, sendo beneficiados pela produtividade que o software proporciona.

Usuários dos sistemas

Não podemos deixar de mencionar as pessoas que utilizam o sistema. Seja qual for a categoria, um sistema será desenvolvido para que as pessoas que terão forte influência no processo o utilizem.

Mesmo existindo situações em que outros sistemas acessarão o sistema desenvolvido, sem que haja a interação de pessoas, o usuário humano é muito importante e tem um papel fundamental na utilização do sistema e do cenário, pois qualquer sistema tem, por tarefa final, a interação com pessoas.

HISTÓRIA DA ENGENHARIA DE SOFTWARE

Para abordarmos a história da Engenharia de Software precisamos nos situar na linha do tempo no que diz respeito à criação dos computadores:

- Os primeiros computadores eletrônicos surgiram na década de 1940, como Colossus e Eniac.
- Por volta dos anos de 1950 e 1960, surgiram as primeiras linguagens de alto nível, que eram Fortran e COBOL.
- Por volta de 1970 e 1980, temos o desenvolvimento de sistemas operacionais mais avançados, como UNIX e MS-DOS.
- Nos anos de 1990, tivemos a popularização da internet e do software de código aberto.

Os acontecimentos citados nos auxiliam a nos localizar no tempo. Eles nos ajudam a entender as evoluções dos computadores, para que, com isso, tenhamos claro os problemas que surgiram e, conseqüentemente, a necessidade de uma estruturação da área de software. Esses problemas foram evidenciados durante a crise do software nos anos de 1960, momento em que começaram a surgir

Esses problemas incluíam projetos e orçamentos atrasados, software com alta incidência de BUGs e manutenções difíceis e caras: todo esse cenário que se formou foi chamado de “Crise do Software”.

problemas relacionados ao desenvolvimento de software. Esses problemas incluíam projetos e orçamentos atrasados, software com alta incidência de BUGs e manutenções difíceis e caras: todo esse cenário que se formou foi chamado de “**crise do Software**”.

Alguns desses problemas se repetem hoje, porém em uma incidência muito menor do que ocorria.

Em 1968, a organização do Tratado Atlântico Norte fez uma conferência na Alemanha, momento em que o termo “Engenharia de Software” surgiu pela primeira vez. Esse termo foi criado para descrever a necessidade de uma abordagem mais disciplinada e estruturada para o desenvolvimento de software.



Quando o cliente pedia para desenvolver um produto que resolvesse o problema dele, os programadores se juntavam e construíam o software de maneira desorganizada, cada um desenvolvia do seu jeito.

E no final o produto a ser construído tinha que ser cancelado pois o software antes de ser concluído já começava a dar muito prejuízo para o cliente, como o estouro de orçamento, atrasos de entrega e péssima qualidade (Alcântara, 2023)

A crise do software terminou em 1970. Isso foi possível devido à criação da Engenharia de Software pois ela introduziu algumas práticas, como padrões, processos, documentações e metodologias. O uso de todas essas práticas permitiu o desenvolvimento de software de forma sistemática, com etapas de início, meio e fim, com o objetivo de facilitar o processo de desenvolvimento, atendendo à necessidade dos clientes e mantendo a eficiência. Consequentemente, isso resultou em produtos de maior qualidade.

O que é Engenharia de Software?

Grande parte dos problemas de software são grandes e difíceis de resolver. Por isso, precisamos ter uma abordagem sistemática para a solução. Devemos começar analisando o problema e dividindo-o em partes, para que possamos entender e analisar cada bloco, para, então, unirmos esses blocos para construirmos uma solução para o todo.



INDICAÇÃO DE FILME

A Rede Social.

O filme intitulado *A Rede Social* conta a história da criação do Facebook, mergulhando em aspectos, como desenvolvimento de software, prazos, qualidade e desafios técnicos e empresariais, tendo relação direta com a Engenharia de Software.



A Engenharia de Software abrange o uso de princípios, métodos e ferramentas para projetar, desenvolver, testar e gerenciar sistemas de software de alta qualidade. Em outras palavras, abarca técnicas aplicadas para cada etapa do ciclo de desenvolvimento de software. Entre elas, podemos citar:

Identificação de necessidades: etapa em que são verificadas as necessidades dos usuários. Pode ser conduzida por entrevistas ou questionários.

Levantamento de requisitos: etapa em que os requisitos são levantados e servirão de insumos para a criação do software.

Definição da arquitetura: nesta etapa, a arquitetura a ser implementada será definida entre a área técnica.

Implementação: a implementação, de fato, é realizada. O código é desenvolvido e a arquitetura é implementada.

Testes e validação: o controle de qualidade é executado mediante testes ou validações no software.



Como engenheiros de software, utilizamos nossos conhecimentos sobre computadores e computação para ajudar a resolver problemas.

Frequentemente, o problema com o qual estamos lidando está relacionado a um computador ou a um sistema computacional já existente, mas, algumas vezes, as dificuldades que são apresentadas não têm relação com computadores. Portanto, é essencial entender primeiro a natureza do problema.

Devemos ser muito cautelosos para não impor máquinas e técnicas computacionais a toda questão que aparecer em nosso caminho. Primeiro, devemos resolvê-la. Então, se necessário, utilizar a tecnologia como ferramenta para implementar a nossa solução (Pfleeger, 2004, p. 2).

CENÁRIOS DE APLICAÇÕES DE SOFTWARE

Quanto aos cenários de aplicação de software, podemos dividi-los em sete grandes categorias.

Segundo Pressman (2016, p. 6), essas categorias são:

- **Software de Sistema:** Inclui diversos tipos de software de sistema, como compiladores, editores e ferramentas de gerenciamento de arquivos.
- **Software de Aplicação:** São programas autônomos que atendem a uma necessidade específica de negócio.
- **Software de Engenharia/Científico:** Envolve uma ampla gama de programas destinados a cálculos complexos, utilizados em áreas como astronomia, vulcanologia e análise de estresse em automóveis.
- **Software Embarcado:** Integrado a um produto ou sistema, este tipo de software é usado para implementar e controlar funções tanto para o usuário quanto para o próprio sistema, como no caso do controle de painel de um forno micro-ondas.

- **Software para linha de produtos:** Software para linha de produtos pode se concentrar em um mercado hermético e limitado (por exemplo, produtos de controle de inventário).
- **Aplicações Web/Aplicativos Móveis:** Esta categoria de software, voltada para redes, inclui uma ampla gama de aplicações, abrangendo tanto aplicativos para navegadores quanto software instalado em dispositivos móveis.
- **Software de Inteligência Artificial:** Utiliza algoritmos não numéricos para resolver problemas complexos que não podem ser computados ou analisados diretamente.

Agora, para continuarmos a evolução dos nossos conhecimentos, compreenderemos os processos de software.

PROCESSOS DE SOFTWARE

O conjunto de atividades relacionadas que podem levar à construção de um sistema de software é chamado de processo de software. Esses processos devem incluir quatro atividades fundamentais, de acordo com Sommerville (2018). São eles:

O conjunto de atividades relacionadas que podem levar à construção de um sistema de software é chamado de processo de software.

- **Especificação:** devem ser claramente definidos os recursos do software e as restrições para o funcionamento.
- **Desenvolvimento:** o software precisa ser criado conforme a especificação previamente definida.
- **Validação:** o software deve passar por um processo de validação para garantir que cumpre as expectativas e exigências do cliente.
- **Evolução:** o software deve ser capaz de evoluir e se adaptar às mudanças nas necessidades dos clientes.

Segundo Pfleeger (2004), todo processo tem as seguintes características:

- O processo estabelece todas as principais atividades de maneira clara.
- Ele faz uso de recursos, é guiado por restrições específicas, como prazos, e resulta em produtos tanto intermediários quanto finais.

- O processo pode incluir subprocessos que se inter-relacionam, formando uma estrutura hierárquica em que cada subprocesso tem o próprio modelo.
- Cada atividade dentro do processo possui critérios de entrada e saída definidos, o que permite identificar com precisão quando o processo começa e termina.
- As atividades são organizadas em uma ordem sequencial, deixando evidente a relação entre as diferentes etapas.
- Além disso, todo processo é acompanhado de diretrizes que esclarecem os objetivos de cada atividade.

Um **modelo de processo de software** define como será o ciclo de desenvolvimento de software. Algumas vezes, os nomes são até usados como sinônimos. Esses modelos incluem abordagens, como:

Modelo em cascata: as fases são separadas, uma fase finaliza para dar início à outra.

Desenvolvimento incremental: o software é desenvolvido em estágios. Uma iteração desenvolve uma parte. Depois, outra iteração desenvolve mais um pouco e, assim por diante, até ter o software completo.

Como pudemos ver, a Engenharia de Software é uma área fundamental para o correto desenvolvimento de um projeto de desenvolvimento de software. Essa área ajuda a reduzir retrabalho, reduzir custos para o projeto, aumentar a qualidade do projeto e melhorar a satisfação do usuário em relação ao produto entregue. Muito do levantamento das necessidades e dos respectivos detalhes da aplicação passa pela disciplina de Engenharia de Software.

**EM FOCO**

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A conexão entre teoria e prática é fundamental para a preparação para o ambiente profissional, sobretudo, em um campo tão dinâmico como desenvolvimento de software, de forma que a Engenharia de Software oferece uma demanda crescente para profissionais que se especializam nessa área. O entendimento dos processos abordados em cada situação é um fator-chave para diferenciá-lo no mercado de trabalho.

Além das linguagens utilizadas, *frameworks* e ferramentas de desenvolvimento, você, estudante, precisará dominar os processos de Engenharia de Software as atividades e as aplicações. Ao expormos uma possível resposta para o cenário citado no início da nossa jornada, você poderia levantar as necessidades do software realizando entrevistas com usuários-chave e especialistas.

A questão inicial sobre como deixar a empresa mais competitiva chama a atenção para um ponto que é almejado por todas as empresas do mercado, pois trará vantagens competitivas para a organização, o que torna, consequentemente, a disciplina estratégica para qualquer empresa. O início de um projeto de sucesso e que trará competitividade para a empresa passa pelos pontos abordados ao longo deste tema, como entendimento da linha do tempo, classificação de software e abordagem da Engenharia de Software

VAMOS PRATICAR

1. É importante examinar as características do software que o tornam diferenciado de outras coisas que os seres humanos constroem. Software é mais um elemento do sistema lógico do que físico. Portanto, o software tem uma característica fundamental que o torna consideravelmente diferente do hardware: software não "se desgasta".

Os softwares podem ser categorizados em diversas áreas. Marque a alternativa que não corresponde a uma dessas áreas:

- a) Aplicativos de comunicação.
 - b) E-mail.
 - c) Chats.
 - d) Banco de dados.
 - e) Processo de fabricação de móveis
-
2. O papel do software passou por uma mudança significativa no decorrer da metade final do século passado. Aconteceram aperfeiçoamentos significativos no desempenho do hardware, mudanças profundas nas arquiteturas computacionais, um vasto aumento na capacidade de memória e armazenamento e uma ampla variedade exótica de opções de entrada e saída.

Todos os aperfeiçoamentos citados levaram ao uso cada vez maior do software em nosso cotidiano. Considerando exemplos de uso de software no dia a dia, analise as afirmativas a seguir:

- I - Comunicação: aplicativos, como Microsoft Teams e Slack, são amplamente utilizados para facilitar a comunicação e a colaboração em equipes, permitindo videoconferências, troca de mensagens e compartilhamento de arquivos.
- II - Organização Pessoal: softwares, como Google Calendar e Todoist, ajudam a organizar compromissos e tarefas diárias, enviando lembretes e permitindo a criação de listas de afazeres.
- III - E-commerce: plataformas, como Shopify e WooCommerce, permitem que pequenas empresas e empreendedores criem e gerenciem lojas on-line, facilitando a venda de produtos e serviços pela internet.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

VAMOS PRATICAR

3. A crise de software foi um período em que os softwares eram construídos de maneira desorganizada, causando muito prejuízo para os clientes mediante estouros de orçamento, atraso de entrega e péssima qualidade, acabando não resolvendo o problema do cliente.

Marque a alternativa que colocou fim à crise do software:

- a) Engenharia de Software.
- b) Desenvolvimento de Software.
- c) Testes de Software.
- d) Engenharia Civil.
- e) Arquitetura de Software.

REFERÊNCIAS

ALCÂNTARA, M. R. História da Engenharia de Software. **LinkedIn**, 12 nov. 2023. Disponível em: <https://pt.linkedin.com/pulse/hist%C3%B3ria-da-engenharia-de-software-marcos-alc%C3%A2ntara-rhlhf>. Acesso em: 16 out. 2024.

PFLEEGER, S. L. **Engenharia de Software**: teoria e prática. 2. ed. São Paulo: Pearson, 2004.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

PRESSMAN, R. S. **Engenharia de Software**: uma abordagem profissional. 8 ed. [S. l.]: McGraw Hill, 2016.

CONFIRA SUAS RESPOSTAS

1. Alternativa E.

"Aplicativos de comunicação" é uma categoria de software que inclui ferramentas usadas para facilitar a comunicação entre indivíduos ou grupos, como mensageiros instantâneos e plataformas de videoconferência.

"E-mail" é uma categoria de software que abrange programas e serviços utilizados para enviar, receber e gerenciar mensagens de correio eletrônico.

"Chats" se refere a uma categoria de software destinada a permitir a comunicação em tempo real mediante mensagens de texto entre usuários.

"Banco de dados" é uma categoria de software que envolve sistemas usados para criar, gerenciar e manipular bases de dados, essenciais para armazenar e recuperar informações estruturadas.

"Processo de fabricação de móveis" não é uma categoria de software. Embora haja software específico para o design e a manufatura de móveis, essa alternativa fala do processo de fabricação.

2. Alternativa E.

A afirmativa I está correta, porque esses aplicativos melhoram a produtividade e a eficiência, permitindo que membros de uma equipe se comuniquem rapidamente e colaborem em tempo real, independentemente da localização.

A afirmativa II está correta, porque esses softwares auxiliam na gestão do tempo e na priorização de tarefas, contribuindo para uma melhor organização pessoal e profissional, além de reduzirem o risco de esquecer compromissos importantes.

A afirmativa III está correta, porque essas plataformas oferecem ferramentas acessíveis e intuitivas para a criação de lojas virtuais, permitindo que os negócios alcancem um público maior e aumentem as vendas sem a necessidade de conhecimentos técnicos avançados.

3. Alternativa A.

A alternativa A é a correta porque foi assim que surgiu o termo "Engenharia de Software" e foi colocado fim à crise do software. A alternativa B corresponde à execução e a criação do software. A opção C corresponde aos testes, somente uma parte de toda a solução. As alternativas D e E não têm relação com a área de software.



unidade



TEMA DE APRENDIZAGEM 2

REQUISITOS DE SOFTWARE

MINHAS METAS

- Compreender os conceitos e os fundamentos dos requisitos de software.
- Entender a importância das habilidades de comunicação.
- Conhecer técnicas que podem ser utilizadas no levantamento de requisitos.
- Explorar a necessidade de uma documentação bem elaborada.
- Identificar as etapas que devem ser verificadas antes de iniciar o levantamento.
- Entender que, embora o levantamento de requisitos seja complexo, com técnicas adequadas, essa complexidade é contornada.
- Compreender que o levantamento de requisitos é crucial para a fundamentação da teoria.

INICIE SUA JORNADA

Um software sem um objetivo claro, sem um levantamento correto das funcionalidades ou sem considerar as necessidades do usuário se mostra um cenário perfeito para a instauração do caos. É nesse contexto que entra a Engenharia de Software mais especificamente, o **levantamento dos requisitos**.

Você já observou, estudante, que, algumas vezes, um software se inicia sem um objetivo claro, o que pode levar a retrabalhos, frustrações, estresse e sem atingir o objetivo principal? Em um projeto com essas características, como você reduziria esses problemas?

Os requisitos de software tendem a ajudar a preencher as lacunas que se formam, atuando como **as peças de um grande quebra-cabeça do desenvolvimento**. Eles definem desde as funcionalidades mais básicas até as funcionalidades mais complexas.

O desenvolvedor deve dominar a técnica de levantamento dos requisitos para entender exatamente o que deve ser executado, os desejos do cliente e os objetivos detalhados. Tudo isso permite a construção de um software de qualidade e que atenda às necessidades a que se propõe.

O entendimento da teoria é muito importante, porém esse entendimento precisa ser complementado com situações práticas, seja a implementação em projetos pessoais, seja a implementação em projetos para grandes corporações, o que inclui várias situações, como o levantamento de requisitos mediante entrevistas com o usuário do software ou entrevistas com especialistas sobre determinada área.

Precisamos refletir sobre essa área de atuação e entender que se trata de um segmento em que precisamos nos atualizar constantemente, procurando treinamentos ou conversas com profissionais experientes.



PLAY NO CONHECIMENTO

Neste podcast, abordamos o tema *Erros comuns em requisitos de software e como evitá-los*. Ouça para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal relembrar a importância do levantamento de requisitos? O vídeo a seguir aborda o levantamento de requisitos e os eventuais problemas que podem ocorrer devido à negligência dessa etapa.

<https://www.youtube.com/watch?v=rVbJ7ykuLig>

DESENVOLVA SEU POTENCIAL

Quando pensamos em um software concluído, cada funcionalidade que esse sistema tem é um meio para atender a um requisito solicitado e planejado até chegar ao software, o qual é executado no computador do usuário.

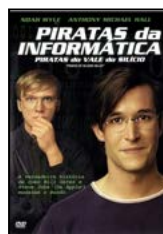
Para exemplificar, podemos imaginar um software criado para o departamento de Recursos Humanos. Esse software possui algumas funcionalidades, como cadastro da pessoa, cadastro do departamento e superior imediato e o cenário em que cada uma dessas funcionalidades pode representar um requisito que foi identificado, levantado e planejado para, posteriormente, ser implantado no computador do usuário.

INDICAÇÃO DE FILME

Piratas da Informática.

O filme relata a história do surgimento da Apple e da Microsoft, evidenciando a evolução dos computadores pessoais e softwares.

O filme mostra muito bem como é necessário considerar os desejos dos usuários e transformá-los em produtos. Além disso, elucida o modo como esse fator é crucial para o sucesso de um projeto.





Piratas da Informática

Direção: Martyn Burke

Fases, como planejamento, fazem parte de um projeto de desenvolvimento de software, momento em que a Engenharia de Software, com o levantamento de requisitos e a definição de cenários, torna-se primordial para o desenvolvimento e a qualidade do produto.

REQUISITOS DE SOFTWARE

Para iniciarmos o planejamento de um software, devemos entender a que ele se propõe e quais são as expectativas, especificando claramente o que esse software e cada parte dele deve fazer.

Em um ambiente ideal, os desenvolvedores e os clientes estão próximos e podem apenas conversar para que os desenvolvedores tenham o entendimento dos requisitos. Contudo, isso nem sempre ocorre, podendo fazer com que tenham de identificar requisitos de clientes em outros locais ou outros empecilhos.

Dessa forma, algumas etapas são necessárias para iniciar bem um projeto. Segundo Pressman e Maxim (2016), essas etapas são:

- **Identificação dos envolvidos:** cada *stakeholder* tem uma perspectiva única do sistema, obtendo benefícios distintos quando o desenvolvimento do sistema é bem-sucedido e enfrentando diferentes riscos caso o projeto falhe.
- **Reconhecimento de diversos pontos de vista:** as informações de todos os envolvidos (inclusive, os requisitos inconsistentes e conflitantes) devem ser classificadas, de maneira que permita aos tomadores de decisão escolher um conjunto internamente consistente de requisitos para o sistema.
- **Trabalho em busca da colaboração:** o papel do engenheiro de requisitos envolve a detecção tanto dos requisitos consensuais, ou seja, daqueles aceitos por todos os *stakeholders*, quanto a identificação das áreas de conflito ou discrepância, momento em que os desejos de um *stakeholder* entram em conflito com os de outros. É evidente que esses conflitos representam um dos maiores desafios para o engenheiro de requisitos.
- **Questões iniciais:** as questões iniciais compreendem questões abertas no início (como: Quem vai usar a solução?), passando por questões mais direcionadas (como: Quais problemas a solução vai resolver?) e perguntas necessárias para garantir a eficiência da atividade de comunicação (como: Você responde oficialmente por esse sistema?).

Os requisitos de um sistema de software são classificados frequentemente como funcionais e não funcionais.

- **Requisitos não funcionais:** são requisitos não tão concretos, como performance, segurança ou restrição geral do sistema.
- **Rastreabilidade:** “rastreabilidade é um termo da Engenharia de Software que se refere a mapeamentos documentados entre os artefatos de Engenharia de Software (por exemplo, requisitos e casos de teste). Uma matriz de rastreabilidade permite a um engenheiro de requisitos representar a relação entre os requisitos e outros artefatos da Engenharia de Software” (Pressman; Maxim, 2016, p. 139).

Os requisitos de um sistema de software são classificados frequentemente como funcionais e não funcionais. Os requisitos não funcionais apresentados foram descritos segundo a ótica de Pressman e Maxim (2016). Agora, vale citar o mesmo sob a ótica de Sommerville (2018).

- **Requisitos funcionais:** descrevem o que o sistema deve fazer e dependem do tipo de software que está sendo desenvolvido, dos usuários esperados para o software e da abordagem geral adotada pela organização ao escrever os requisitos.
- **Requisitos não funcionais:** “são aqueles que não possuem relação direta com os serviços específicos fornecidos pelo sistema a seus usuários. Esses requisitos não funcionais normalmente especificam ou restringem as características do sistema como um todo” (Sommerville, 2018, p. 91).

Levantamento de requisitos

Também chamado de elicitação de requisitos, é uma técnica que permite ao engenheiro de requisitos identificar quais são os desejos e as necessidades que, posteriormente, serão convertidos em funcionalidades do software. O levantamento pode ser realizado por diversas formas, incluindo:

Observação: ao observar as tarefas diárias dos usuários, o engenheiro de requisitos consegue identificar as necessidades. No entanto, trata-se de uma técnica que pode causar certa distorção, pois será a visão do observador.

Questionário: são elaboradas perguntas, as quais são aplicadas aos usuários. Deve-se ter cuidado, pois a pergunta pode ser mal interpretada.

Entrevistas: conversa usada para extrair todo o funcionamento desejado. Apesar de ser uma técnica consistente, requer habilidades de comunicação para extrair o necessário.

Análise de documentos: ótima técnica para a coleta dos processos, mas não leva em conta os desejos e os fatores humanos.

Análise da concorrência: importante, pois sempre existe algo a aprender com a concorrência.

O processo de levantamento de requisitos também é chamado de elicitação de requisitos. Esse processo deve obedecer a algumas fases para execução, segundo Sommerville (2018).

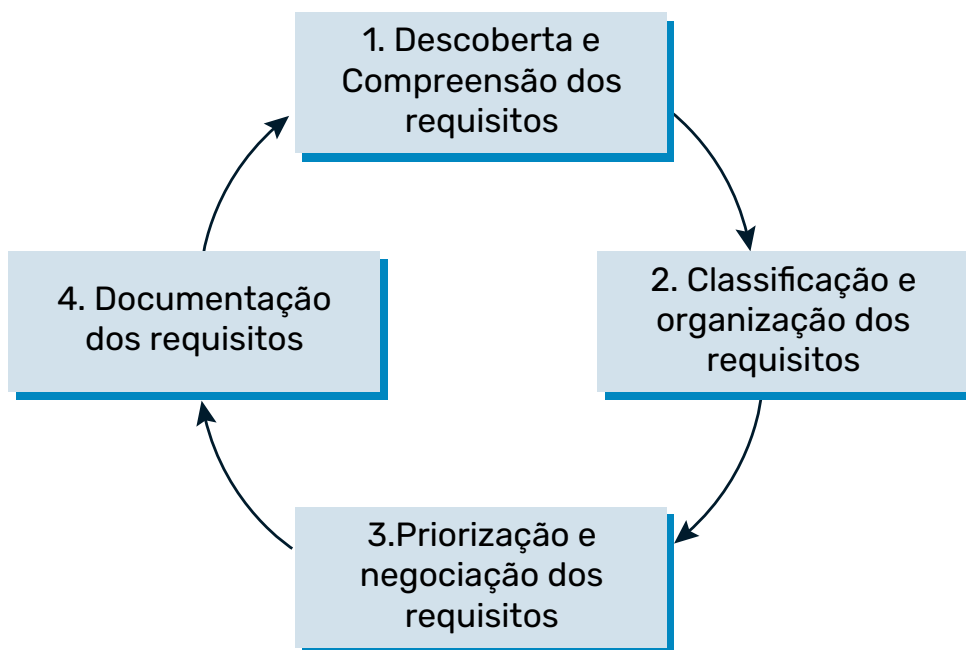


Figura 1 - O processo de elicitação e análise de requisitos / Fonte: Sommerville (2018, p. 97).

Descrição da Imagem: é exposta uma figura com as atividades do processo, representadas por quatro quadrados ligados por uma linha formando um círculo. Cada atividade está descrita dentro de um quadrado. São elas: 1. Descoberta e Compreensão dos requisitos; 2. Classificação e organização dos requisitos; 3. Priorização e negociação dos requisitos; e 4. Documentação dos requisitos. Fim da descrição.

A Figura 1 representa as etapas de um processo de elicitação de requisitos, sendo que cada um dos passos é descrito a seguir.

O processo de descoberta e compreensão de requisitos envolve a interação com os *stakeholders* do sistema para identificar os respectivos requisitos. Em seguida, ocorrem a classificação e a organização dos requisitos, momento em que o conjunto não estruturado de requisitos é agrupado e organizado em grupos coerentes.

A etapa de priorização e negociação dos requisitos está relacionada à definição de prioridades e à resolução de conflitos por meio da negociação. Por fim, os requisitos são documentados, servindo como base para a próxima fase do desenvolvimento.



EU INDICO

No texto disponível no QR Code, o site indicado nos apresenta formações e desafios do mercado de trabalho. <https://www.devdo futuro.com/carreira-em-engenharia-de-software/>

Durante o levantamento de requisitos, que abrange qualquer área de desenvolvimento de software, não se limitando a uma tecnologia específica, podemos nos perguntar qual é o nível de profundidade de nossa análise, se não estamos descendo a um nível de detalhe extremo e, muitas vezes, desnecessário, ou se não estamos sendo superficiais.

Segundo Pressman e Maxim (2016), o grau com o qual a modelagem de requisitos para Web Apps e aplicativos móveis é enfatizado depende dos seguintes fatores relacionados ao tamanho:

- (1) O tamanho e a complexidade do incremento da aplicação.
- (2) O número de envolvidos (a análise pode ajudar a identificar requisitos conflitantes provenientes de diferentes fontes).
- (3) O tamanho da equipe de desenvolvimento de aplicativos.

O processo de descoberta e compreensão de requisitos envolve a interação com os stakeholders do sistema para identificar os respectivos requisitos.

(4) Por quanto tempo os membros da equipe trabalharam juntos (a análise pode ajudar a desenvolver um entendimento comum do projeto).

(5) Até que ponto o sucesso da organização depende diretamente do sucesso da aplicação.

DOCUMENTOS E REQUISITOS

Os requisitos elicitados e analisados devem ser documentados. Na atividade de documentação, um Documento de Especificação de Requisitos de Software (DERS ou DRS) é normalmente gerado. Ele contém os requisitos de software de determinado projeto.



EU INDICO

Observe um modelo de documento de requisitos <https://analisederequisitos.com.br/documento-de-requisitos-de-software/>

Existem vários modelos de DERS e cada organização pode definir o próprio modelo de acordo com as próprias necessidades. Qualquer que seja o formato, o DERS é um dos artefatos mais importantes do processo de software, uma vez que é a base para praticamente todas as atividades de construção subsequentes.

Assim, é imprescindível avaliar a qualidade de um DERS. Para tanto, características de qualidade devem ser apontadas e métricas devem ser estabelecidas (Nardi; Falbo, 2014).

A seguir, são exibidas as etapas em que os requisitos são utilizados durante o ciclo de vida do projeto:

- Definição de critérios de aceitação e validação (pelos *stakeholders*).
- Definição do que o sistema deve fazer (especificação para a equipe).
- Teste e verificação do sistema sendo desenvolvido.

- Informações para o gerenciamento de mudanças (rastreabilidade e análise de impacto).
- Alocação de tarefas para a equipe.
- Estimativa de custo/esforço/cronograma.
- Acompanhamento e controle do andamento do projeto (Introdução [...], 2013).

O DRS deve ser elaborado durante a fase de análise de requisitos do projeto de software. Isso fica mais evidente em projetos que seguem a metodologia *waterfall*, pois eles são realizados por fase. Esse documento deve ser constantemente atualizado ao longo do projeto para refletir as mudanças solicitadas pelo cliente.



PENSANDO JUNTOS

Metodologias de gerenciamento de projetos nos ajudam a conduzir um projeto de forma estruturada, focando na qualidade do produto final. A metodologia **waterfall** é considerada preditiva, portanto, uma fase deve ser concluída e, em seguida, temos a outra fase, formando uma cascata.

O DRS é um documento muito importante ao levantamento de requisitos, dado que definirá os requisitos e a descrição, servindo de fundamento ao restante do projeto. Não só, mas também deve conter:

- **Visão geral do produto**

Apresenta, de forma clara e concisa, a visão. Embora a visão seja geral, deve apresentar uma clareza suficiente ao entendimento.

- **Descrição dos requisitos**

Neste tópico, as funcionalidades, os atributos e as descrições são definidos de forma detalhada e inequívoca, para que não gere dupla interpretação.

- Casos de uso

Descrição dos cenários específicos do modo como os usuários interagirão com o software. Neste tópico, a descrição deve ser feita de forma clara, para não gerar dupla interpretação.

Documentos e comunicação

Qualquer projeto de desenvolvimento de software tem, como ponto sensível, a comunicação, pois envolve uma equipe, muitas vezes, multidisciplinar. Em outras palavras, cada profissional é de uma área distinta e um tipo de comunicação é esperado.

A documentação de software pode ser entendida como um artefato cuja função é transmitir informações sobre o sistema de software ao qual está relacionada. No entanto, é importante diferenciar entre modelos, documentos, código-fonte e a própria documentação.

De acordo com Ambler (2001 apud Souza *et al.*, 2007), na perspectiva da modelagem ágil, um documento é qualquer artefato que esteja fora do código-fonte e que tenha, como objetivo, comunicar informações de forma duradoura. Um modelo, por sua vez, é uma abstração que representa um ou mais aspectos de um problema ou de uma possível solução.

Alguns modelos podem se transformar em documentos, ser integrados a eles ou até mesmo ser descartados após cumprirem o propósito. Já o código-fonte é uma sequência de instruções acompanhada de comentários que explicam essas instruções. É destinado a um sistema computacional. O termo documentação abrange tanto os documentos quanto os comentários presentes no código-fonte (Souza *et al.*, 2007).

Assim, uma documentação de requisitos bem estruturada facilita o entendimento de todos os envolvidos, pois todos falam o mesmo “idioma”.

Engenharia de Requisitos

A Engenharia de Requisitos (ER) é a subárea da Engenharia de Software que trata do processo de definição dos requisitos de software. Esse processo é sistemático e abrange diversas atividades, tais como elicitação, análise e negociação, documentação, validação e gerência de requisitos. Cada uma das atividades que

compõem a Engenharia de Requisitos forma a base para um projeto de desenvolvimento de software, pois definirão o escopo do trabalho a ser realizado. Essas atividades são explicadas a seguir.

- Elicitação: nesta atividade, os requisitos são coletados com os usuários mediante técnicas, como a realização de entrevistas.
- Análise e negociação: a análise dos dados coletados é feita, e negociações, como limitações de escopo, são realizadas.
- Documentação: a documentação é realizada.
- Validação: em seguida, a documentação é validada.
- Gerência de requisitos: os requisitos são gerenciados.

Requisitos de software são sentenças que expressam as necessidades dos clientes e condicionam a qualidade do software ou especificações de serviços que o sistema deve prover, restrições no sistema e conhecimentos necessários para desenvolvê-lo.

Requisitos de software são sentenças que expressam as necessidades dos clientes

**INDICAÇÃO DE LIVRO****Engenharia de Requisitos**

O PMI aponta que a gestão de requisitos é uma competência fundamental na gestão de projetos, mas também destaca que é a causa de 47% dos projetos que fracassam.

Trata-se de uma das disciplinas fundamentais da Engenharia de Software e, talvez, uma das mais negligenciadas. Os requisitos são a base para o trabalho de quase todas as atividades do projeto e uma falha pode provocar um impacto em cascata.



Uma vez capturados, os requisitos de software devem ser modelados, documentados, validados e acompanhados. Nesse processo, as propriedades de um requisito e os relacionamentos com outros elementos do processo de software são definidos e alterados (Nardi; Falbo, 2014).

NOVOS DESAFIOS

O mundo da Engenharia de Software acaba por ser um campo muito dinâmico. Ele integra teoria e prática e proporciona inúmeros desafios complexos a serem enfrentados no mercado de trabalho.

A disciplina de Engenharia de Software possui inúmeras abordagens. A área de Engenharia de Requisitos é uma das áreas que requer maior atenção e nível de detalhamento, pois é nos requisitos e no correto levantamento e detalhamento que reside grande parte do potencial de sucesso de um projeto de software. Qualquer lacuna ou ambiguidade pode levar a falhas do produto final, que é o software em execução.

No ambiente profissional, a habilidade de realizar um levantamento e uma descrição de maneira clara e concisa é uma característica altamente valorizada e que pode diferenciar você, estudante, dos demais profissionais. Empresas de todos os setores dependem de softwares para otimizar as operações e a demanda por engenheiros de software com forte competência em Engenharia de Requisitos é crescente.

À medida que a tecnologia evolui, o papel desse profissional cresce em relação à importância, pois tecnologias, como DEVOPS, exigem abordagens mais interativas e colaborativas, inclusive, levantamento de requisitos, o que fará com que estudantes que se dedicarem à essa área tenham vantagem competitiva.

É muito importante que você pratique constantemente abordagens diferenciadas de levantamento e descrição de requisitos. A descrição, muitas vezes, pode aparentar uma simplicidade que não se reproduz na prática, pois, na prática, você lidará com pessoas de várias culturas. Essas diferenças podem tornar o processo de levantamento um pouco mais complexo do que nos mostra a teoria.



VAMOS PRATICAR

1. Os objetivos dos processos de elicitação de requisitos são compreender os trabalhos que os *stakeholders* realizam e entender como usariam um novo sistema para apoiar o trabalho deles.

Assinale a alternativa que representa corretamente uma fase do processo de elicitação de requisitos:

- a) Definição de Recursos Humanos.
 - b) Análise de Custos e Benefícios.
 - c) Priorização e Negociação dos Requisitos.
 - d) Teste de Funcionalidades.
 - e) Avaliação de Desempenho.
2. A documentação deve ser feita ao longo do desenvolvimento do software. Problemas de custos, imprecisão dos serviços fornecidos e dificuldades de manipulação podem ser causados pela ausência de documentação ou quando ela é construída após o desenvolvimento do software. A documentação exige um planejamento, isto é, a identificação dos documentos, o estabelecimento da organização, a definição da linguagem a ser utilizada e um estudo das diferentes tecnologias para tal.

A respeito do Documento de Especificação de Requisitos, analise as afirmativas a seguir:

- I - Deve conter a visão geral do produto.
- II - Deve conter a descrição dos requisitos.
- III - Deve conter os casos de uso do software.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

VAMOS PRATICAR

3. Antigamente, dizia-se que requisitos eram sinônimos de funções, ou seja, tudo que o software deveria fazer funcionalmente. No entanto, atualmente, assumiu-se que os requisitos de software são muito mais do que apenas funções. Requisitos são, além de funções, objetivos, propriedades, restrições que o sistema deve ter para satisfazer contratos, padrões ou especificações de acordo com o(s) usuário(s). De forma mais geral, um requisito é uma condição necessária para satisfazer a um objetivo.

Os requisitos são classificados em dois tipos. São eles:

- a) Requisitos Funcionais e Abstratos.
- b) Requisitos Funcionais e Não funcionais.
- c) Testes e Requisitos Abstratos.
- d) Requisitos Abstratos e Não Funcionais.
- e) Requisitos de Interface e Abstratos

REFERÊNCIAS

INTRODUÇÃO a Requisitos de Software. **DevMedia**, 2013. Disponível em: <https://www.devmedia.com.br/introducao-a-requisitos-de-software/29580>. Acesso em: 26 out. 2024.

NARDI, J. C.; FALBO, R. de A. **Uma Ontologia de Requisitos de Software**. [S. l.: s. n.], 2014.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Belo Horizonte: McGraw Hill, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

SOUZA, S. C. B. de *et al.* Documentação essencial para manutenção de software II. *In*: WORKSHOP DE MANUTENÇÃO DE SOFTWARE MODERNA, 4., 2007, Porto de Galinhas. **Anais [...]**. Porto de Galinhas: [s. n.], 2007.

CONFIRA SUAS RESPOSTAS

1. Alternativa C.

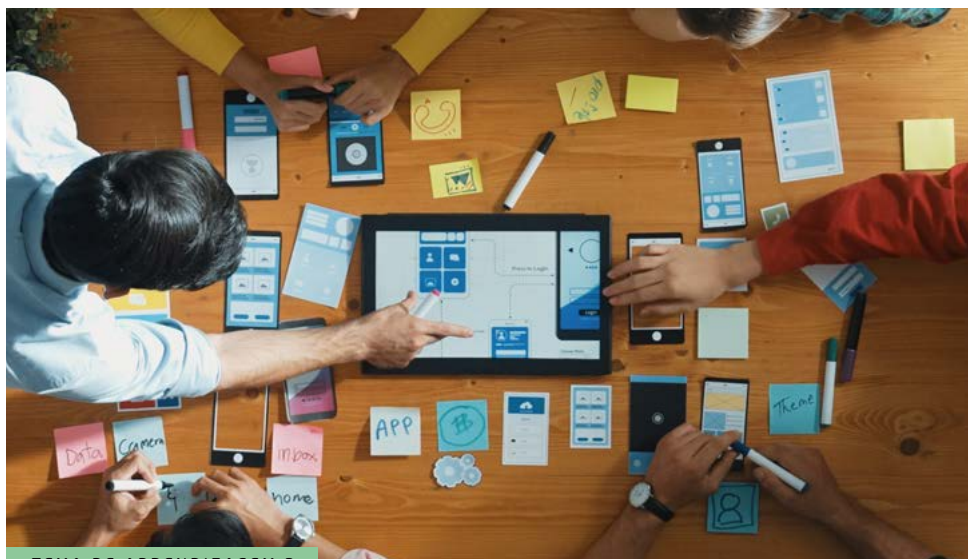
A alternativa A está incorreta, porque "Definição de Recursos Humanos" não é uma fase do processo de elicitação de requisitos. Esse processo está focado em entender e capturar as necessidades dos *stakeholders* para o desenvolvimento do sistema, e não em alocar ou definir recursos humanos. A alternativa B está incorreta, porque "Análise de Custos e Benefícios" também não faz parte do processo de elicitação de requisitos. Esse tipo de análise é geralmente feito durante a fase de viabilidade ou planejamento de projetos, e não diretamente na elicitação de requisitos. A alternativa C está correta, porque "Priorização e Negociação dos Requisitos" é uma fase importante do processo de elicitação. Nessa etapa, os requisitos são avaliados quanto à importância e viabilidade, e decisões são tomadas para determinar quais serão implementados primeiro. A alternativa D está incorreta, porque "Teste de Funcionalidades" não faz parte do processo de elicitação de requisitos. O teste de funcionalidades ocorre em fases posteriores do ciclo de vida do software, geralmente, durante a fase de verificação e validação. A alternativa E está incorreta, porque "Avaliação de Desempenho" não é uma fase da elicitação de requisitos. A avaliação de desempenho geralmente acontece durante a fase de testes ou manutenção, após o desenvolvimento do sistema.

2. Alternativa E.

A afirmativa I está correta, pois o Documento de Especificação de Requisitos deve conter a visão Geral do Produto, que apresenta de forma clara e concisa a visão. Embora essa visão seja geral, deve apresentar uma clareza suficiente ao entendimento. A afirmativa II está correta, pois o Documento de Especificação de Requisitos deve conter a descrição dos requisitos, em que as funcionalidades, os atributos e as descrições são definidos de forma detalhada e inequívoca para que não gere dupla interpretação. A afirmativa III está correta, pois o Documento de Especificação de Requisitos deve conter Casos de Uso, nos quais é exibida a descrição dos cenários específicos de como os usuários irão interagir com o software. Nesse tópico, a descrição deve ser feita de forma clara para não gerar dupla interpretação.

3. Alternativa B.

A alternativa A está incorreta, porque requisitos abstratos não fazem parte da classificação de requisitos. A alternativa B está correta, porque essa é a divisão dos requisitos: os funcionais representam o que deve fazer funcionalmente, ao passo que os não funcionais representam as propriedades e restrições do sistema. A alternativa C está incorreta, porque requisitos abstratos ou testes não fazem parte da classificação de requisitos. A alternativa D está incorreta, porque requisitos abstratos não fazem parte da classificação de requisitos. A alternativa E está incorreta, porque requisitos abstratos ou de interface não fazem parte da classificação de requisitos.



TEMA DE APRENDIZAGEM 3

MODELAGEM DE SISTEMAS

MINHAS METAS

- Conhecer a importância da Engenharia de Software.
- Explorar a relevância da modelagem.
- Definir a *Unified Modeling Language* (UML)
- Estudar o Diagrama de Caso de Uso.
- Compreender a relevância da modelagem no mercado de trabalho.
- Conhecer ferramentas de auxílio (como as ferramentas CASE).
- Entender a importância de se desenvolver observando os testes.

INICIE SUA JORNADA

Considere um cenário em que você é o responsável por desenvolver um aplicativo estratégico para uma grande empresa. Desde a fase inicial, você percebe que o projeto apresenta falhas na comunicação, problemas de priorização das atividades a serem realizadas e falta de clareza nos requisitos do cliente. Como resultado, você percebe que esse projeto terá constantes atrasos e não atenderá às expectativas do cliente.

É nesse contexto que entra a Engenharia de Software, que não se resume apenas à programação, sendo fundamental para o desenvolvimento de um sistema, porque nos auxilia a pensar nas soluções e a planejar em detalhes o que será codificado, reduzindo as chances de encontrarmos um problema posteriormente. Ela ampliará consideravelmente as suas habilidades, consequentemente, deixando-te mais preparado para desenvolver soluções mais complexas e projetos desafiadores.

Retornando ao exemplo inicial, como poderíamos resolvê-lo? O ideal seria mostrar ao usuário, de forma clara, a interação das funcionalidades e dos módulos que comporão o sistema, para que você tenha a certeza de que está no caminho correto.

VAMOS RECORDAR?

Para refletir um pouco mais sobre a importância da Engenharia de Software, assista ao vídeo a seguir, que aborda definições a respeito da Engenharia de Software e pode te proporcionar uma melhor compreensão sobre o assunto.

<https://www.youtube.com/watch?v=39yo4FkPMIs>



PLAY NO CONHECIMENTO

Ouçá o nosso podcast, em que abordamos o seguinte tema: *Desenvolvimento orientado a testes - TDD*. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.

DESENVOLVA SEU POTENCIAL

A modelagem de sistemas é uma parte crucial do sistema de software, que trabalha com técnicas e nomenclaturas comuns ao processo de desenvolvimento, permitindo a representação visual e descritiva dos componentes, das funcionalidades e dos fluxos do sistema. Isso faz com que problemas possam ser identificados mais cedo e seja aprimorada a comunicação entre os envolvidos no projeto, considerando os membros da equipe, como desenvolvedores e analistas, e os usuários externos, como usuários do sistema e outros *stakeholders*.

INTRODUÇÃO À UML

Durante o planejamento de software, vários detalhamentos são realizados, tais como diagramas criados para cada cenário que se apresenta no software. Entretanto, cada desenvolvedor tinha a própria maneira de escrever essas representações. Diante disso, surgiu a necessidade de se padronizar essa modelagem, o que promoveu uma discussão a respeito da padronização das linguagens e artefatos.

Na década de 1990, surgiu a Unified Modeling Language (**UML**), também conhecida como Linguagem de Modelagem Unificada. Ela surgiu como uma solução para a falta de padronização, tornando-se referência na área e consolidando diversas metodologias aplicadas em separado.

A **comunicação** é um dos maiores desafios em um projeto de desenvolvimento de software, uma vez que há diversas partes interessadas no projeto. Assim, a UML nos apresenta uma forma estruturada de representar os diferentes aspectos de um sistema.

A comunicação é um dos maiores desafios em um projeto

Os sistemas de software apresentam uma complexidade considerável e exigem uma comunicação clara entre a equipe do projeto. A UML vem totalmente de encontro a essa necessidade, pois oferece uma maneira estruturada de representar os diferentes aspectos do sistema.



“Assim como os arquitetos criam plantas e projetos para serem usados por uma empresa de construção, os arquitetos de software criam diagramas UML para ajudar os desenvolvedores de software a construir o software. Se você entender o vocabulário da UML (os elementos visuais do diagrama e seus significados), poderá facilmente entender e especificar um sistema e explicar o projeto desse sistema para outros interessados” (Pressman; Maxim, 2016, p. 869).



INDICAÇÃO DE FILME

Som na Faixa (The Playlist).

Neste relato ficcionalizado, um empreendedor sueco e seus parceiros decidem revolucionar a indústria musical com uma plataforma legalizada de *streaming*. A minissérie explora aspectos relacionados ao desenvolvimento de software, incluindo comunicação e colaboração entre equipe, concepção, desenho de soluções e escalabilidade e desempenho.



A UML é composta por uma série de diagramas divididos entre duas categorias, segundo Sommerville (2018, p. 121-146):

TIPOS DE DIAGRAMAS UML

Estruturais: diagramas que focam na estrutura do sistema, mostrando como os elementos estão organizados.

Diagrama de Classes
Diagrama de Objetos
Diagrama de Componentes
Diagrama de Pacotes
Diagrama de Estrutura Composta
Diagrama de Implantação
Diagrama de Perfil

COMPORTAMENTAIS: USADOS PARA MODELAR O COMPORTAMENTO, IDENTIFICANDO COMO OS ELEMENTOS DO SISTEMA INTERAGEM AO LONGO DO TEMPO.

Diagramas de Caso de Uso
Diagrama de Sequência
Diagrama de Atividades
Diagrama de Estados
Diagrama de Comunicação
Diagrama de Tempo
Diagrama de Interação

Cada um dos diagramas citados possui uma finalidade específica dentro da UML. Logo, cada um é aplicado a uma determinada situação.

No mercado, nem todos os diagramas são utilizados. Isso depende muito da necessidade do projeto ou da empresa. Alguns diagramas podem ser citados como comuns. Segundo Bento (2020, p. 52), são eles:

- Diagrama de Caso de Uso
- Diagrama de Classes
- Diagrama de Atividades
- Diagrama de Sequência
- Diagrama de Comunicação
- Diagrama de Componentes
- Diagrama de Implementação

Esses diagramas são considerados essenciais, pois cobrem as principais necessidades de modelagem. Além do mais, atuam desde a estrutura estática até o comportamento dinâmico do sistema e interação com os usuários.

FERRAMENTAS CASE

As ferramentas CASE são definidas para auxiliar o desenvolvedor ao longo do ciclo de vida de um projeto de software. Elas auxiliam em tarefas, como a modelagem, possibilitando a criação de diagramas UML, até a codificação e a realização de testes.

***Assim como
os arquitetos
criam plantas
e projetos
para serem
usados por uma
empresa de
construção, os
arquitetos de
software criam
diagramas UML
para ajudar os
desenvolvedores
de software
a construir o
software***



As ferramentas CASE podem auxiliar em áreas, como:

- **Gestão dos requisitos:** organizar e rastrear requisitos.
- **Modelagem e design:** criar diagramas, como UML
- **Gerador de código:** algumas podem gerar código de forma automática, baseadas na modelagem.
- **Controlador de versão:** gerenciamento de versões do software.
- **Testes:** auxilia em testes automatizados.
- **Documentação:** atuação na geração e manutenção de documentos.
- **Gestão de projetos:** gestão do projeto, considerando recursos, alocação e acompanhamento.

Segundo Vavassori, Souza e Fiamoncini (2001, p. 1), exemplos de ferramentas CASE incluem:

- FastTrack.
- TaskManager.
- Delegator.
- Costar.
- Calico.

Também podemos citar outras ferramentas comerciais, como:

- Azure Devops.
- IBM Rational Rose.
- Jira Software.
- Atlassian Confluence.
- IBM Rational Suite.

Para uma ferramenta ser considerada CASE, ela deve oferecer suporte integrado a mais de uma área do ciclo de desenvolvimento de software. As ferramentas CASE auxiliam todas as fases do ciclo de desenvolvimento de software, desde modelagem e design até testes e documentação.

MODELAGEM DE SISTEMAS

O desenvolvimento de um sistema deve contar com a criação das estruturas e representações abstratas e gráficas em fases anteriores à codificação. Nessa técnica considerada fundamental à Engenharia de Software, os eventuais problemas serão detectados mais cedo, o que contribui para a comunicação entre os envolvidos no projeto.

"Os modelos são criados para entender melhor o que será construído. Quando a entidade for algo físico (por exemplo, um edifício, um avião, uma máquina), podemos construir um modelo que seja idêntico na forma e no formato, porém em menor escala. Entretanto, quando a entidade a ser construída for software, nosso modelo deve assumir uma forma diferente. Ele deve ser capaz de representar as informações que o software transforma, a arquitetura e as funções que permitem a transformação, as características que os usuários desejam e o comportamento do sistema à medida que a transformação ocorra. Os modelos devem cumprir esses objetivos em diferentes níveis de abstração – primeiro, descrevendo o software do ponto de vista do cliente e, depois, em um nível mais técnico" (Pressman; Maxim, 2016, p. 114)

Scott Ambler e Ron Jeffries (2002 *apud* Pressman; Maxim, 2016) estabelecem um conjunto de princípios a serem seguidos. Eles nos orientam durante uma modelagem, para que o foco não se perca e tenhamos um produto de qualidade:

- **Princípio 1:** o objetivo principal da equipe de software é construir software, e não criar modelos.
- **Princípio 2:** seja objetivo: não crie mais modelos do que precisa.
- **Princípio 3:** esforce-se ao máximo para produzir o modelo mais simples possível.
- **Princípio 4:** construa modelos que facilitem alterações.
- **Princípio 5:** estabeleça um propósito claro para cada modelo
- **Princípio 6:** adapte os modelos que desenvolveu ao sistema à disposição.
- **Princípio 7:** crie modelos úteis: esqueça a construção de modelos perfeitos.
- **Princípio 8:** não se torne dogmático quanto à sintaxe do modelo. Se ela consegue transmitir o conteúdo, a representação é secundária.

- **Princípio 9:** se os instintos dizem que um modelo não está correto, mesmo parecendo correto no papel, provavelmente, há motivos para se preocupar.
- **Princípio 10:** obtenha feedback o quanto antes.

É possível desenvolver modelos tanto do sistema existente quanto do sistema a ser desenvolvido, segundo Sommerville (2018):

1. Modelos do sistema existente são utilizados durante a Engenharia de Requisitos. Eles ajudam a esclarecer o que o sistema atual faz e podem ser usados para focar a discussão dos *stakeholders* nos pontos fortes e fracos desse sistema.
2. Modelos do novo sistema são utilizados durante a análise de requisitos para ajudar a explicar para outros *stakeholders* os requisitos propostos. Os engenheiros usam esse modelo para discutir propostas de projeto e documentar o sistema para implementação. Ao usar um processo de engenharia dirigida por modelos, é possível gerar uma implementação completa ou parcial do sistema a partir desses modelos.

A modelagem de sistemas se mostra cada dia mais relevante para o contexto do desenvolvimento de software, pois, com o entendimento dos objetivos mencionados, conseguimos compreender toda a aplicação e, consequentemente, o papel importante que ela desempenha na antecipação de problemas, no desenho de soluções futuras e no auxílio em manutenções de softwares legados.

Diagramas de Caso de Uso

Os Diagramas de Caso de Uso são um dos diagramas fundamentais da UML, porque mostram a interação entre as diversas funcionalidades do sistema e os atores. Esse diagrama permite que os usuários consigam entender e analisar a documentação de forma que não exija conhecimentos técnicos: logo, é possível validar o diagrama.

O Diagrama de Caso de Uso ajuda a determinar as funcionalidades do sistema sob a perspectiva do usuário. Esse diagrama é composto por alguns elementos, a saber:

ELEMENTO	DESCRIÇÃO	REPRESENTAÇÃO GRÁFICA
Ator	Qualquer entidade externa que se comunique com o sistema. Pode ser usuário ou outro sistema.	 Descrição: boneco de palito: representa o ator.
Caso de Uso	Representa as funcionalidades que os sistemas possuem e oferecem aos atores. O caso de uso descreve as ações e os fluxos executados	 Descrição: elipse que representa o caso de uso.
Relações	Linhas que conectam os atores aos casos de uso ou entre casos de uso	Descrição: linha de conexão.
Relações entre casos de uso	Inclui o comportamento de outro caso de uso ou extensão que representa uma relação em que um caso de uso entende o comportamento de outro.	<<include>> OU <<extend>>

Quadro 1 – Elementos básicos de um Diagrama de Caso de Uso / Fonte: adaptado de Pressman e Maxim (2018).

O Diagrama de Caso de Uso é um dos diagramas comportamentais mais utilizados em um levantamento junto ao usuário. Esse diagrama representa os limites do sistema e suas funcionalidades de forma clara, conseguindo comunicar com objetividade a proposta do sistema e o que ele contém como funcionalidades.

Exemplo de um Diagrama de Caso de Uso

O exemplo a seguir demonstra um Diagrama de Caso de Uso de um sistema de música. Nele, o teste organiza as músicas em uma biblioteca e carrega a lista de músicas.

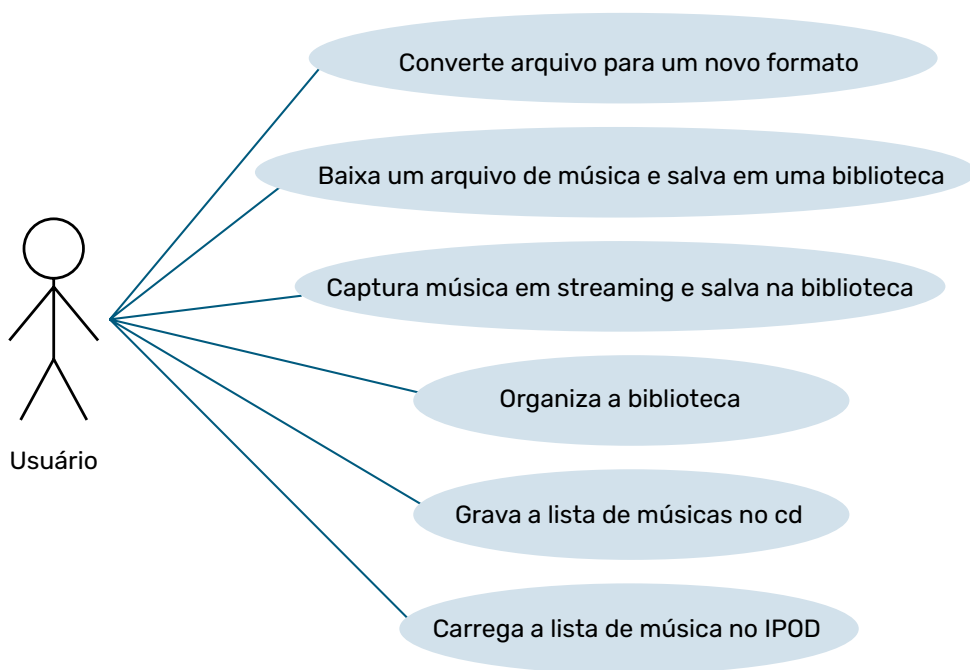


Figura 1 - Diagrama de caso de uso que mostra a compra de um produto / Fonte: adaptada de Pressman e Maxim (2016).

Descrição da Imagem: trata-se de um Diagrama de Caso de Uso que mostra o funcionamento do sistema de músicas. Ele é representado por seis elipses com os textos: "Converte arquivo para um novo formato", "Baixa um arquivo de música e salva em uma biblioteca", "Captura música em streaming e salva na biblioteca", "Organiza a biblioteca", "Grava a lista de músicas no CD" e "Carrega a lista de música no IPOD". Essas elipses representam os casos de uso e são ligadas mediante linhas contínuas a um boneco de palito, que representa o ator em UML. Fim da descrição.

O Diagrama de Caso de Uso é uma representação fundamental para evidenciar o funcionamento do sistema, servindo como meio de comunicação com os envolvidos.

Com as técnicas de Engenharia de Software abordadas, conseguimos vislumbrar a abrangência e o nível de detalhamento que uma modelagem bem executada traz ao nosso projeto de software e a respectiva relevância na execução diária em nossas atividades no mercado de trabalho.

 EM FOCO

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

No mercado de trabalho, a **modelagem** e a **UML** possuem um lugar de destaque, pois moldam habilidades e definem sistemas e comportamentos dos sistemas, fazendo com que as empresas economizem muito com a identificação de eventuais erros que só apareceriam posteriormente, já no momento do planejamento. O conhecimento em Engenharia de Software prepara os estudantes para desafios que ocorrem no mercado de trabalho, como aqueles envolvendo gestão ou outras áreas do ciclo de desenvolvimento de software. Profissionais com uma sólida compreensão desses conceitos são altamente valorizados no mercado de trabalho.

Problemas que frequentemente ocorrem em razão da falta de comunicação ou expectativas não atendidas podem ser evitados e contornados mediante a UML e a modelagem, de forma que se trata de uma área muito abrangente e um bom segmento para se investir tempo de estudo e se especializar. Com isso, estudante, você certamente terá as chances potencializadas no mercado de trabalho.

VAMOS PRATICAR

1. Esse diagrama documenta o que o sistema faz do ponto de vista do usuário. Em outras palavras, ele descreve as principais funcionalidades do sistema e a interação dessas funcionalidades com os usuários do mesmo sistema. Nesse diagrama não nos aprofundamos em detalhes técnicos que dizem como o sistema faz (DevMedia, 2012).

O texto fala a respeito do:

- a) Diagrama de Classes.
 - b) Diagrama de Componentes.
 - c) Diagrama de Pacotes.
 - d) Diagrama de Atividades.
 - e) Diagrama de Casos de Uso.
-
2. A clínica médica Saúde Perfeita precisa de um sistema de agendamento de consultas e exames. Um paciente entra em contato com a clínica para marcar consultas visando realizar um check-up anual com seu médico de preferência. A recepcionista procura data e hora disponíveis mais próxima na agenda do médico e marca as consultas. Posteriormente o paciente realiza a consulta, e nela o médico pode prescrever medicações e exames, caso necessário (DevMedia, 2012).

Diante do cenário apresentado, analise as afirmativas a seguir:

- I - O cenário possui os atores Paciente, Secretária, Médico.
- II - A relação entre casos de uso pode ser por include.
- III - A relação entre casos de uso pode ser por extend.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

VAMOS PRATICAR

3. A UML não é um método é uma linguagem de modelagem designada para especificar, visualizar, construir e documentar um sistema. A linguagem de modelagem é a notação que o método utiliza para expressar projetos, enquanto o processo indica quais passos seguir para desenvolver um projeto (UML..., 2024).

Considerando o tema apresentado, marque a alternativa correta:

- a) A UML é uma linguagem de modelagem, não um método, usada para especificar e documentar sistemas.
- b) A UML é um método que descreve todas as etapas para o desenvolvimento de software.
- c) A UML é uma linguagem de modelagem e um processo para criar e usar diagramas.
- d) A UML é uma ferramenta que gera código automaticamente a partir dos diagramas. E. própria.

REFERÊNCIAS

BENTO, L. H. **UML**: Um estudo sobre o uso em empresas de desenvolvimento de software em São Carlos - SP e região. 2020. Trabalho de Conclusão de Curso (Graduação em Engenharia de Software) – Universidade de São Paulo, São Carlos, 2020.

DEVMEDIA. O que é UML e Diagramas de Caso de Uso: Introdução Prática à UML. **DevMedia**, 2012. Disponível em: <https://www.devmedia.com.br/o-que-e-uml-e-diagramas-de-caso-de-u-so-introducao-pratica-a-uml/23408>. Acesso em: 28 out. 2024.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8. ed. McGraw Hill, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

UML - Conceitos Básicos II. **Macoratti.net**, c2024. Disponível em: https://www.macoratti.net/vb_uml2.htm. Acesso em: 28 out. 2024.

VAVASSORI, F. B.; SOUZA, E. W. de; FIAMONCINI, J. C. Ferramenta CASE para Gerenciamento de Projetos e Métricas de Software. In: SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE, 15., 2001, Rio de Janeiro. **Anais** [...]. Rio de Janeiro: Sociedade Brasileira de Computação, 2001. p. 362-367.

CONFIRA SUAS RESPOSTAS

1. Alternativa E.

A alternativa A está incorreta, pois esse diagrama mostra um relacionamento entre as classes. A alternativa B está incorreta, pois esse diagrama mostra os diversos componentes. A alternativa C está incorreta, porque mostra os pacotes de instalação. A alternativa D está incorreta, pois esse diagrama mostra o fluxo de atividades. A alternativa E está correta, pois mostra a interação entre os diversos atores e as funcionalidades

2. Alternativa E.

A afirmativa I está correta, porque, ao analisarmos o cenário, temos três envolvidos. A afirmativa II está correta, porque uma das formas de relacionamento é include, um caso de uso inclui o comportamento de outro. A afirmativa III está correta, porque uma das formas de relacionamento é extend, um caso de uso estende o outro.

3. Alternativa A.

A alternativa A está correta, porque a UML (Unified Modeling Language) é, de fato, uma linguagem de modelagem usada para especificar, visualizar, construir e documentar sistemas, mas não é um método em si. A alternativa B está incorreta porque a UML não é um método que descreve etapas de desenvolvimento de software; é uma notação utilizada para modelar sistemas. A alternativa C está incorreta, porque a UML é apenas uma linguagem de modelagem, não um processo ou método para criar e usar diagramas. A alternativa D está incorreta, porque a UML não é uma ferramenta de geração automática de código, mas uma linguagem para modelagem de sistemas. A alternativa E está incorreta, porque a UML inclui uma linguagem de modelagem própria, ao contrário do que a frase sugere.

unidade





MINHAS METAS

- 62

INICIE SUA JORNADA

No desenvolvimento de software, estamos refletindo cada elemento do mundo real para uma representação no sistema. A orientação a objetos faz uso de classes que representam esses elementos como unidade de programação, refletindo, desse modo, os objetos do mundo real.

A transposição para um software de computador pode ser um tanto desafiadora e é um problema enfrentado no desenvolvimento de um projeto de software. Imagine, estudante, que você esteja projetando o sistema de uma biblioteca e terá que transpor objetos do mundo real, como livros ou pessoas, para objetos em um software e precisará considerar conceitos, como herança, polimorfismo, classes, entre outros.

Para possibilitar a **criação de sistemas complexos e robustos**, você deve entender a importância dos conceitos relacionados, pois são as práticas mais modernas no desenvolvimento de software e possibilitam que sistemas bem projetados sejam executados com excelência. Sistemas semelhantes já elaborados podem auxiliar na elaboração do sistema em questão, assim como técnicas de modelagem de sistemas, como diagramas de caso de uso ou diagramas de classes da UML.

Diante do cenário exposto pela orientação a objetos, refletimos como essas práticas moldam o desenvolvimento de software moderno, auxiliando no devido desenvolvimento e fazendo com que ele seja mais flexível. Neste tema de aprendizagem, abordaremos a orientação a objetos e os respectivos conceitos fundamentais.



PLAY NO CONHECIMENTO

Nesse podcast, abordamos o tema *Comparando orientação a objetos com outras abordagens*. Ouça para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal relembrar a importância da orientação a objetos? O canal Código Fonte TV demonstra essa abordagem. <https://www.youtube.com/watch?v=QYoKdg83orY>

DESENVOLVA SEU POTENCIAL

A orientação a objetos é um paradigma amplamente utilizado no desenvolvimento de software, trazendo consigo várias vantagens, como a reutilização de código por meio da herança, pois classes derivadas podem reutilizar e estender classes-base. Também podemos citar o encapsulamento, que protege os dados internos, expondo somente o necessário, contribuindo para a manutenção e proporcionando uma maior segurança ou, ainda, o polimorfismo, que permite que diferentes objetos sejam tratados de forma semelhante.

Projetos de orientação a objetos facilitam a comunicação com os usuários, pois fazem uma representação direta de objetos do mundo real. Isso permite uma maior compreensão por parte do usuário final ou demais envolvidos.

INTRODUÇÃO À ORIENTAÇÃO A OBJETOS

A programação orientada a objetos envolve objetos reais que podemos visualizar, mas transformados em classes. Como exemplo, podemos citar uma pessoa que irá interagir com um sistema: ela será o objeto “Pessoa”; ao transferirmos para um software, utilizaremos uma classe chamada “Pessoa”. Essa classe terá os atributos dessa pessoa do mundo real, que são as características dela, incluindo nome, idade e gênero.

A representação de uma classe na UML seria da maneira exibida a seguir: na

parte de cima, é exibido o nome da classe. Na metade, logo abaixo, encontram-se os atributos definidos para essa classe.

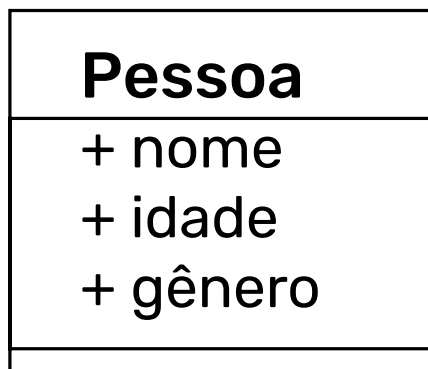


Figura 1 - Classe Pessoa / Fonte: o autor.

Descrição da Imagem: trata-se de um quadrado dividido com duas linhas horizontais. A primeira parte contém o nome da classe “Pessoa”, enquanto a do meio exibe os nomes dos atributos “nome”, “idade” e “gênero”. Fim da descrição.

Pessoa
+ nome
+ idade
+ gênero
+ andar ()
+ correr ()

Cada objeto do mundo real tem ações associadas a ele. Por exemplo, um carro acelera e freia, ao passo que uma pessoa também tem ações associadas, como andar e correr. Para essas ações, damos o nome “Métodos”. Portanto, a representação ficaria da seguinte forma:

Figura 2 - Classe pessoa com métodos / Fonte: o autor.

Descrição da Imagem: trata-se da representação de uma classe chamada “Pessoa”. Essa representação se dá mediante um quadrado com as propriedades “nome”, “idade” e “gênero” e “métodos”, os quais são representados por linhas. Os métodos andar() e correr() se encontram na terceira parte do quadrado. Fim da descrição.



INDICAÇÃO DE FILME

Matrix.

Matrix é um filme que se passa em um futuro próximo e mostra uma realidade alternativa criada por computadores. Cada elemento da Matrix é um objeto que precisou ser desenvolvido. Esse conceito espelha a forma como a orientação a objetos modela classes e objetos do mundo real. O filme levanta questões sobre realidade ou abstração, conceitos que permeiam a orientação a objetos.



Os objetos são representados mediante as classes definidas com as características dos objetos, ou seja, os atributos deles. Desse modo, conseguimos acessar e manipular as características mencionadas.



APROFUNDANDO

“É uma prática comum deixar os nomes dos membros e da classe nos diagramas em inglês, como faríamos em um código real. No entanto, comentários e notas também podem ser escritos em português” (Shvets, 2021, p. 9)

Suponha que você esteja criando um programa que usa a classe “Pessoa”. Cada vez que você faz referência a uma pessoa para essa classe, você está criando um objeto dessa classe.

Por exemplo, se você criar o objeto João e Maria baseados na classe Pessoa, você estará criando duas instâncias desse objeto.

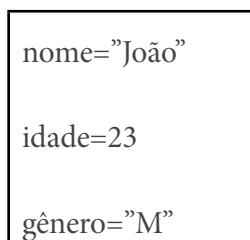


Figura 3 - Objeto pessoa Nome João / Fonte: o autor.

Descrição da Imagem: trata-se de uma representação do objeto instanciado com nome, idade e gênero de João, ou seja, um quadrado com o nome igual a João, idade igual a 23 e gênero igual a M, um em cada linha representando os atributos da classe. Fim da descrição.

Os objetos são representados mediante as classes definidas com as características dos objetos, ou seja, os atributos deles. Desse modo, conseguimos acessar e manipular as características mencionadas.

A seguir, temos a mesma representação para outro objeto da mesma classe.

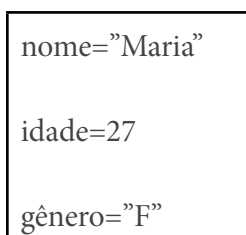


Figura 4 - Objeto pessoa Nome Maria / Fonte: o autor.

Descrição da Imagem: trata-se de uma representação do objeto instanciado com nome, idade e gênero de Maria, ou seja, um quadrado com o nome igual a Maria, idade igual a 27 e gênero igual a F, um em cada linha representando os atributos da classe. Fim da descrição.



A representação do objeto foi demonstrada com sua criação a partir da classe `pessoa`, a essa criação do objeto chamamos de instanciação.

A orientação a objetos é baseada em pilares que definem a arquitetura desse modelo de programação. Esses pilares são descritos no próximo tópico.

PILARES DA ORIENTAÇÃO A OBJETOS

A programação orientada a objetos se baseia em pilares. Esses pilares frequentemente apresentam características a respeito do próprio funcionamento ou associações. Além disso, são cinco os pilares que tornam o paradigma de orientação a objetos tão poderoso. São eles: especialização, herança, abstração, polimorfismo e encapsulamento.

Herança

Suponha que você esteja desenvolvendo um sistema para uma biblioteca. Logo, a classe pessoa será a que utilizará o sistema. Ela pode ser “Aluno” ou “Professor”. A classe mãe, isto é, a classe pessoa, chamamos de superclasse (ou classe-base) e as respectivas filhas são chamadas de subclasses (ou classes derivadas). Essa “pirâmide” de classes forma uma **hierarquia**.

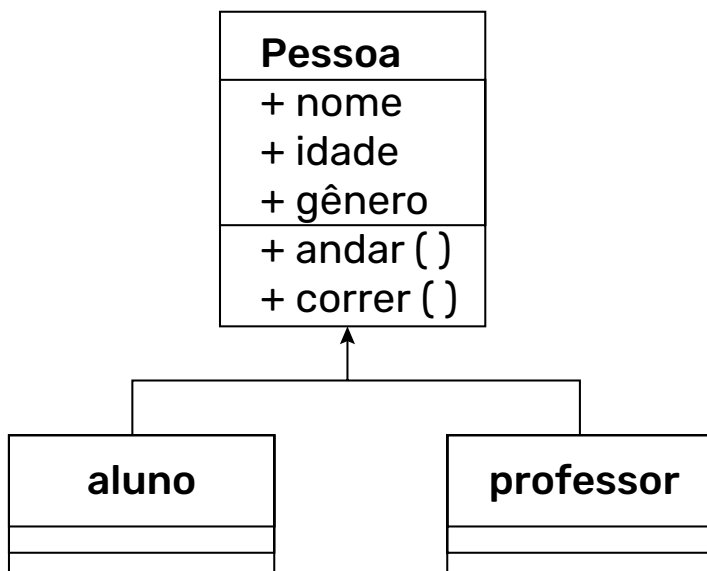


Figura 5 - Herança de classe / Fonte: o autor.

Descrição da Imagem: trata-se de uma representação da classe pessoa. Há o nome “Pessoa” e três linhas representando os atributos (nome, idade e gênero) com os métodos (duas linhas “andar” e “correr”). Logo abaixo, encontram-se dois retângulos representando as classes “aluno” e “professor” interligadas por uma linha (representando a herança da classe pessoa). Fim da descrição.

Considerando um objeto, isto é, cadeira, Pressman e Maxim (2016, p. 891) nos mostram a associação de classes:



Uma cadeira (*chair*). Chair é uma subclasse de uma classe muito maior que chamamos de PieceOfFurniture (PeçaDeMóvel). Cadeiras individuais são membros (usualmente chamados de instâncias) da classe Chair. Um conjunto de atributos genéricos pode ser associado a cada objeto da classe PieceOfFurniture. Por exemplo, todos os móveis têm um custo, dimensões, peso, localização e cor, entre muitos outros atributos possíveis. Isso se aplica independentemente de estarmos falando de uma mesa (*table*) ou uma cadeira (*chair*), um sofá (*sofa*) ou um armário (*armoire*). Como Chair é membro de PieceOfFurniture, Chair herda todos os atributos definidos para a classe.

Outra característica muito importante e que compõe um dos pilares da orientação a objetos é a especialização.

Especialização

Quando aplicamos a herança, mas adicionamos métodos ou propriedades da classe filha (no exemplo, alunos ou professores), estamos especializando a classe.

Por exemplo, **Pessoa** pode ser uma classe genérica, enquanto **Aluno** pode ter um atributo que apenas cabe a essa classe, como o **nome da turma**. Dessa forma, a classe aluno teria um atributo adicional chamado Nome Turma.

Outra característica muito importante e que compõe um dos pilares da orientação a objetos é a abstração, assunto que veremos a seguir.

Abstração

É o conceito em que uma classe representa uma visão simplificada de um conjunto de objetos. São escondidos detalhes complexos, o que permite trabalhar com uma interface geral.

Uma classe pessoa (abstrata) poderia existir em um sistema de controle de bibliotecas e em um sistema de compras on-line. No sistema de compras on-line, haveria métodos ou atributos que não seriam interessantes se fossem exibidos

no sistema de biblioteca, e vice-versa, de forma que o objeto seria limitado a um contexto específico.

A classe abstrata serve como um modelo (template) para outras classes. Essas subclasses teriam que implementar os métodos representados.

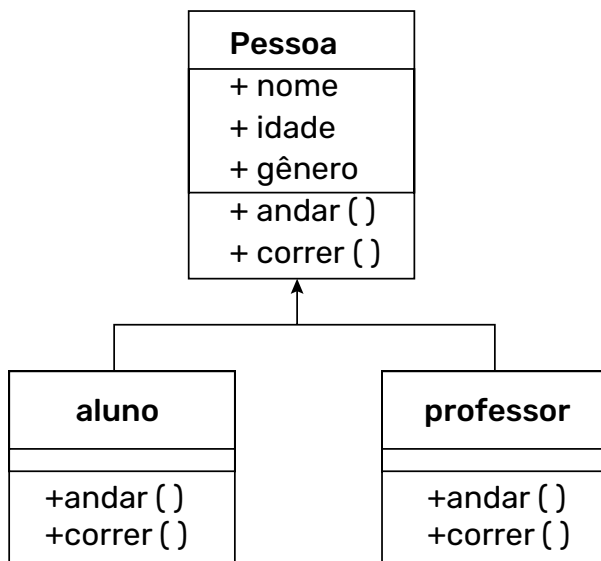


Figura 6 - Herança de uma classe abstrata / Fonte: o autor.

Descrição da Imagem: trata-se de uma representação da classe abstrata “pessoa”. Assim, são exibidos o nome Pessoa e três linhas, as quais representam os atributos (“nome”, “idade” e “gênero”), com os métodos (duas linhas: “andar” e “correr”). Logo abaixo, encontram-se dois retângulos representando as classes “aluno” e “professor”, interligadas por uma linha (representando a herança da classe pessoa). Cada uma das classes (“aluno” e “professor”) tem as palavras “andar” e “correr”. Fim da descrição.

Polimorfismo

Ocorre o polimorfismo quando chamamos um método de mesmo nome existente em várias especializações de classes. Um exemplo é se chamarmos um método correr, que existe em várias classes especializadas de Pessoa, como Aluno ou Professor, e o programa identifica a classe que irá chamar.

Segundo Shvets (2021, p. 21):



O Polimorfismo é a habilidade de um programa detectar a classe real de um objeto e chamar sua implementação mesmo quando seu tipo real é desconhecido no contexto atual. Você também pode pensar no polimorfismo como a habilidade de um objeto “fingir” que é outra coisa, geralmente uma classe que ele estende ou uma interface que ele implementa.

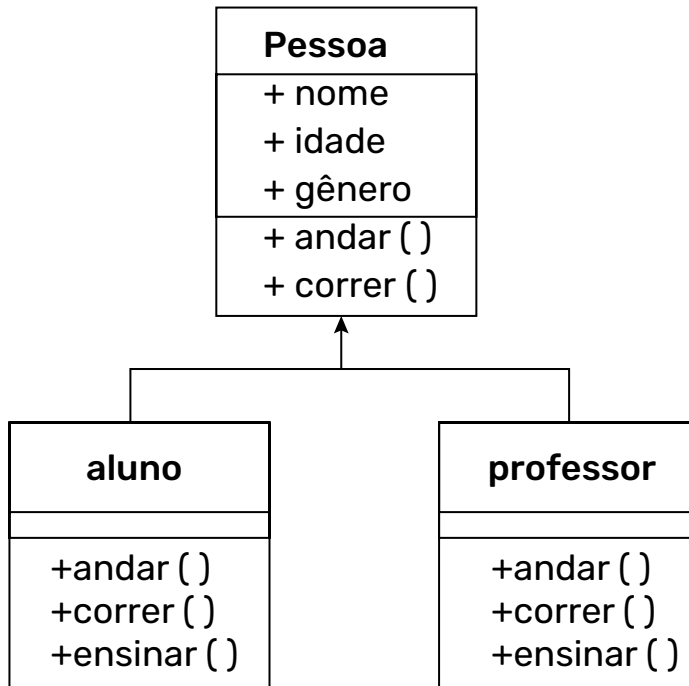


Figura 7 - Herança de uma classe abstrata no polimorfismo / Fonte: o autor.

Descrição da Imagem: trata-se de uma representação da classe abstrata “pessoa”. Há o nome Pessoa, e três linhas representando os atributos (nome, idade e gênero), com os métodos (duas linhas andar e correr). Logo abaixo, encontram-se dois retângulos representando as classes “aluno” e “professor”, interligadas por uma linha (representando a herança da classe pessoa). Cada uma das classes (aluno e professor) tem as palavras “andar” e “correr”. A classe “aluno” possui o texto “estudar”, enquanto a classe “professor” possui o texto “ensinar”. Fim da descrição.

A representação UML do polimorfismo é igual à representação da abstração. A diferença ocorre na implementação, momento em que é possível instanciar qualquer uma das subclasses, especificando que elas são do tipo pessoa.

APROFUNDANDO

Com relação ao polimorfismo, valem algumas observações. Como se trata de um assunto que está intimamente conectado à herança, entender os dois em conjunto é uma boa ideia. Outro ponto é o fato de que as linguagens de programação implementam o polimorfismo de maneiras diferentes. O C#, por exemplo, faz uso de métodos virtuais (com a palavra-chave *virtual*) que podem ser reimplementados (com a palavra-chave *override*) nas classes filhas. Já em Java, apenas o atributo “@Override” é necessário (DevMedia, 2014)

A abstração e o polimorfismo são fundamentais para a caracterização e a flexibilidade da orientação a objetos.

Outra característica muito importante e que compõe um dos pilares da orientação a objetos é o encapsulamento.

Encapsulamento

Você não precisa ter a visão daquilo que ocorre em um método, como ele chega a determinado resultado, todas as contas que outro programador fez dentro do método e que fazem com que determinado valor seja retornado. A capacidade de um objeto de “esconder” o que ocorre nesse método, expondo apenas uma interface para o resto da aplicação, corresponde ao conceito de encapsulamento.

Na figura a seguir, a classe *Pessoa* tem os atributos privados, assim como o método *andar* : eles apenas podem ser acessados de dentro da classe, encapsulando a chamada.

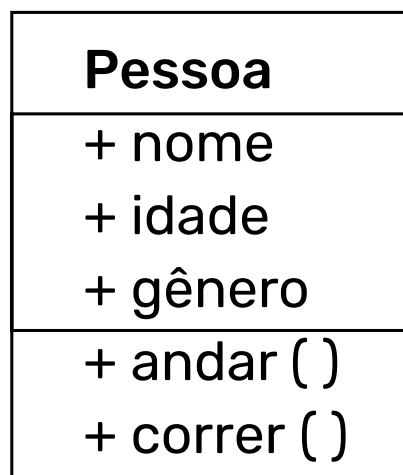


Figura 8 - Classe *Pessoa* com métodos privados
Fonte: o autor.

Descrição da Imagem: trata-se de uma representação de uma classe chamada “ *Pessoa* ” mediante um quadrado, com as propriedades “ *nome* ”, “ *idade* ” e “ *gênero* ”, tendo um sinal de menos na frente indicando que são privados. Os métodos são representados por linhas, e os métodos *andar()* e *correr()* se encontram na terceira parte do quadrado. O método *andar* também possui o sinal de menos. Fim da descrição.



O encapsulamento de software é um conceito quase tão antigo quanto o próprio software. No princípio da década de 40, alguns programadores notaram que o mesmo modelo de instrução apareceria diversas vezes dentro do mesmo programa. Alguns estudiosos (tais como Maurice Wilkes e seus colegas da Cambridge University) logo concluíram que esse tipo de modelo repetido poderia ser transportado para um canto do programa, e mesmo requisitado, sob a forma de um único nome a partir de vários pontos diferentes do programa principal (Page-Jones, 2001, p. 3).



INDICAÇÃO DE LIVRO

Orientação a Objetos.

À medida que aumentava a demanda por novos paradigmas que correspondessem às necessidades do dia a dia dos programadores, surgia a orientação a objetos com a missão de cobrir as insuficiências do modelo estrutural. O paradigma orientado a objeto tem como principal característica uma melhor e maior expressividade das nossas demandas e possibilita criar unidades de código mais próximas da forma como pensamos e agimos.



Segundo Sommerville (2018, p. 177), para desenvolver um projeto de sistema desde o conceito até o projeto detalhado e orientado a objetos, é preciso:

1. Compreender e definir o contexto e as interações externas com o sistema.
2. Projetar a arquitetura do sistema.
3. Identificar os principais objetivos do sistema.
4. Desenvolver modelos de projeto.
5. Especificar interfaces.

Neste tema de aprendizagem, abordamos conceitos a respeito da orientação a objetos e os respectivos pilares. Com isso, conseguimos identificar a devida aplicação e como ela pode ser benéfica aos nossos projetos, auxiliando nas atividades diárias em nossa trilha profissional.

 EM FOCO

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A orientação a objetos fornece as bases para a programação moderna. Isso se dá de forma organizada, facilitando a criação de sistemas modulares e reutilizáveis. Assim, conceitos, como os pilares da orientação a objetos, contribuem para que essa abordagem seja extremamente robusta.

Durante o próprio trabalho, o desenvolvedor é experimentado constantemente, tendo que demonstrar conhecimentos que vão além da programação. Ele deve saber reaproveitar código, desenvolver um software que seja de fácil manutenção e usar padrões de mercado.

A aplicação eficaz dessas técnicas é extremamente valorizada no mercado de trabalho. Vale muito a pena se especializar, pois um desenvolvedor com esse conhecimento é muito valorizado nas empresas.

VAMOS PRATICAR

1. Uma classe Avião poderia provavelmente existir em um simulador de voo e em uma aplicação de compra de passagens aéreas. No entanto, no primeiro caso, ela guardaria detalhes relacionados ao próprio voo, enquanto, no segundo caso, importar-se-ia somente com as poltronas disponíveis e os locais delas. Esse pilar é um modelo de um objeto ou fenômeno do mundo real limitado a um contexto específico, que representa todos os detalhes relevantes para esse contexto com grande precisão e omite o resto.

O pilar da POO descrito se refere à(ao):

- a) Encapsulamento.
 - b) Herança.
 - c) Polimorfismo.
 - d) Abstração.
 - e) Objeto.
2. Podemos dizer que classe é uma descrição generalizada (por exemplo, um modelo ou esquema) que descreve uma coleção de objetos similares. Por definição, objetos são instâncias de uma classe específica e herdam os atributos e propriedades disponíveis para manipular os atributos.

A respeito das classes, assinale a alternativa correta:

- a) A instância de uma classe é chamada de objeto.
- b) Classe é um dos pilares da POO.
- c) Classe representa a herança em POO.
- d) Classe representa a especialização e POO.
- e) Classe e objeto são sinônimos

VAMOS PRATICAR

3. Durante as décadas de 1970 e 1980, linguagens de programação orientadas a procedimentos, como C e Pascal, eram amplamente usadas para desenvolver sistemas de software orientados a negócios. À medida que os programas executavam funcionalidades de negócios mais complexas e interagiam com outros sistemas, porém, as deficiências da metodologia de programação estrutural começaram a aparecer (Nyakundi, 2024).

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

- I - A programação orientada a procedimentos foi predominante no desenvolvimento de sistemas de software orientados a negócios nas décadas de 1970 e 1980.

PORQUE

- II - Com o aumento da complexidade dos sistemas e a necessidade de interação entre diferentes componentes, as limitações da programação estrutural começaram a se tornar evidentes, destacando a necessidade de novas abordagens.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

DEVMEDIA. Os 4 pilares da Programação Orientada a Objetos. **DevMedia**, 2014. Disponível em: <https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264>. Acesso em: 29 out. 2024.

NYAKUNDI, H. Significado de OOP – o que é Programação Orientada a Objetos? **freeCodeCamp**, 10 jul. 2024. Disponível em: <https://www.freecodecamp.org/portuguese/news/significado-de-oop-o-que-e-programacao-orientada-a-objetos/>. Acesso em: 29 out. 2024.

PAGE-JONES, M. **Fundamentos do desenho orientado a objeto com UML**. São Paulo: Pearson, 2001.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Belo Horizonte: McGraw Hill, 2016.

SHVETS, A. **Mergulho nos padrões de projeto**. [S. l.: s. n.], 2021.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

CONFIRA SUAS RESPOSTAS

1. Alternativa D.

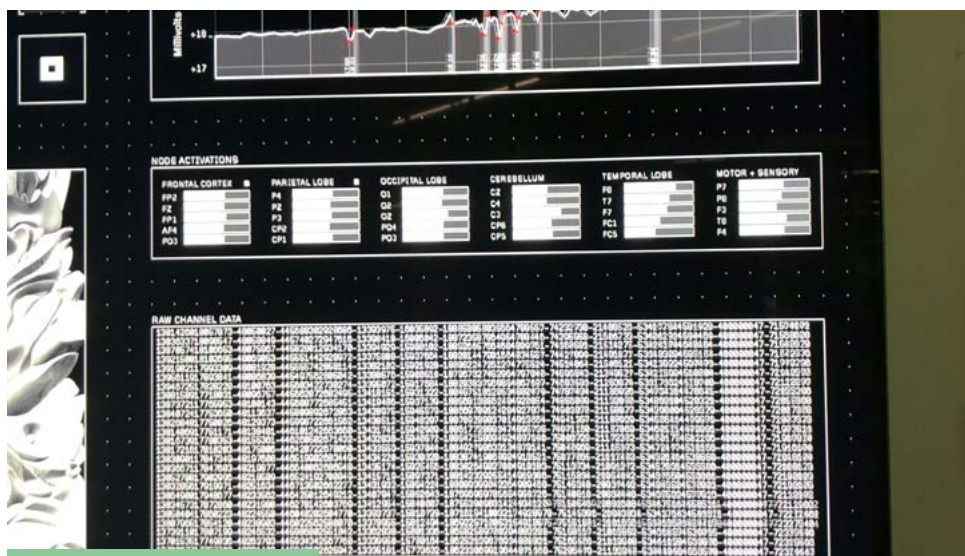
A alternativa A está incorreta, porque o encapsulamento "esconde" o código, e não alterna entre objetos. A alternativa B está incorreta, porque a herança cria um relacionamento com a classe base. A alternativa C está incorreta, porque o polimorfismo chama o método de mesmo nome em objetos diferentes. A alternativa D está correta, porque se refere ao conceito de abstração. A alternativa E está incorreta porque objeto não é um dos pilares de POO.

2. Alternativa A.

A alternativa A está correta, visto que, quando você instancia a classe, ela se torna um objeto em memória. A alternativa B está incorreta, porque a classe não é um dos pilares. As alternativas C e D estão incorretas, porque são conceitos distintos. A alternativa E está incorreta, porque não são sinônimos, embora estejam relacionados.

3. Alternativa A.

A asserção I é verdadeira, dado que eram orientadas a procedimentos, pois traziam simplicidade e a adequação às necessidades da época. Outro ponto importante é que os softwares e os hardwares mostravam certa limitação e a programação orientada a procedimentos tendia a ser mais eficiente em termos de desempenho. A asserção II também é verdadeira, porque se mostraram limitadas em relação a fatores, como manutenção, reaproveitamento de código, falta de integração entre componentes e falta de abstração.



TEMA DE APRENDIZAGEM 5

DIAGRAMAS DE SISTEMA

MINHAS METAS

- Entender a representação gráfica das classes.
- Compreender a importância dos diagramas.
- Desenvolver o diagrama de classes.
- Criar um diagrama de sequência.
- Modelar diagramas de máquina de estados.
- Entender a representação do fluxo do programa.
- Criar um diagrama de atividades.

INICIE SUA JORNADA

Como seria possível garantir o entendimento e a comunicação em um projeto de software complexo e que possui inúmeras classes relacionadas? O diagrama de classes é crucial para o cenário apresentado. Sem o uso dele, a comunicação ficaria seriamente comprometida e o projeto estaria fadado ao fracasso.

Em um projeto de desenvolvimento de software, a **comunicação** é fundamental para o sucesso da solução. Todos os envolvidos no projeto têm leituras diferentes e interpretações diversas. Por isso, faz-se necessário que tenhamos documentos de fácil visualização e entendimento.

Podemos considerar que os diagramas são essenciais para o projeto, desde a estrutura estática até o comportamento dinâmico de um software. Citamos, como exemplos, o diagrama de classes, que mostra as relações, como as peças de um quebra-cabeça, que formam o sistema, e o diagrama de sequência, que captura a interação entre os objetos do sistema.

Imagine, estudante, que você está desenvolvendo um sistema de gestão de tarefas: você precisaria criar um cadastro de tarefas, um cadastro de pessoas e um controle de tarefas. Primeiramente, você precisaria levantar os elementos básicos, como as pessoas que irão utilizar, as tarefas e as interações. Diante disso, diagramas, como os diagramas de classes, poderiam te ajudar, porque mostram a relação entre as classes do sistema. Além disso, são válidos os diagramas de atividades, sequência, entre outros.

O uso dos diagramas vai muito além da simples prática burocrática. Eu te convido a refletir sobre a necessidade de diagramas em um projeto de software e a maneira como eles auxiliam na interação entre todos os envolvidos, na evolução e, consequentemente, no sucesso do projeto.



PLAY NO CONHECIMENTO

Neste podcast, abordamos o tema *Boas Práticas para a Criação de Diagramas UML*. Ouça para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Os diagramas UML têm uma aplicação primordial na criação dos sistemas e no planejamento do que será desenvolvido, evidenciando os componentes de orientação a objetos. O entendimento das bases é primordial para a elaboração de diagramas. Dessa forma, eu te convido a relembrar os principais diagramas UML. O site DevMedia define a UML, descreve e explora os principais tipos de diagramas, como os de casos de uso, classes e sequência, explicando como cada um contribui para a representação clara da estrutura e do comportamento do sistema.

DESENVOLVA SEU POTENCIAL

A **programação orientada a objetos** nos oferece inúmeros benefícios. Logo, traz consigo muitas vantagens, como a representação de objetos do mundo real e as respectivas interações. Com isso, define as ações em sequência, gerando o comportamento do sistema.

Embora essas situações ajudem no desenvolvimento, elas precisam ser planejadas antes da codificação. É essencial considerar as relações e os comportamentos do sistema para elaborar um planejamento detalhado, o que contribuirá significativamente para a comunicação e a realização do trabalho em equipe.

Para isso, utilizamos diagramas UML, que permitem a comunicação eficaz entre os profissionais envolvidos no desenvolvimento do software. Cada membro da equipe pode apresentar e interpretar os diagramas, possibilitando que aqueles com conhecimento adequado completem e ajustem a modelagem do sistema. Esses diagramas detalham o comportamento e a estrutura do sistema, facilitando a modelagem do software a ser desenvolvido.

Nos tópicos a seguir, abordaremos alguns dos principais diagramas UML e definições a respeito, permitindo a devida compreensão.

CLASSE

Uma analogia para a criação de classes seria uma forma de bolo, ou seja, você tem a forma, a qual especifica o formato do seu bolo e as características dele, como al-

tura e largura. A partir dessa forma (que poderíamos fazer analogia a nossa classe), podemos criar vários bolos, que seguem as definições da forma, ou seja, várias instâncias do objeto bolo.

Na programação orientada a objetos, são reproduzidos os objetos do mundo real, com características e ações, porém chamamos essas características de **atributos** ou **propriedades**, ao passo que, às ações, damos o nome de **métodos dos objetos**.

Para criarmos um objeto no projeto, precisamos criar uma estrutura para ele. A essa estrutura, damos o nome de **classe**. Por exemplo, para o objeto pessoa, criamos uma classe pessoa, que, quando for instanciada, passa a ser um objeto. A seguir, temos uma representação gráfica da classe pessoa.

Para criarmos um objeto no projeto, precisamos criar uma estrutura para ele. A essa estrutura, damos o nome de classe.

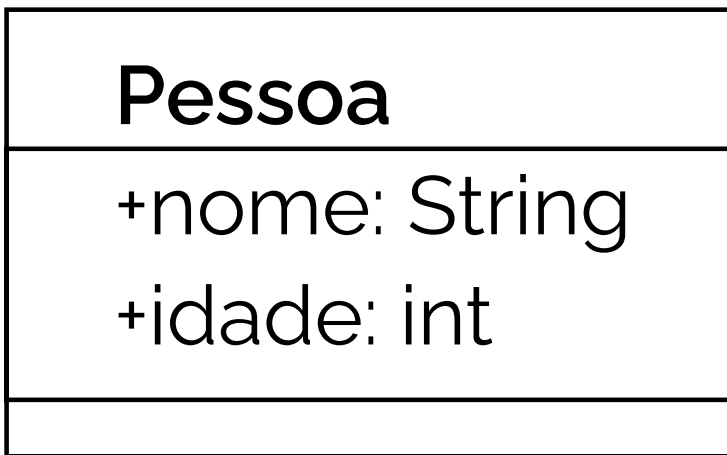


Figura 1 - Representação gráfica da classe pessoa / Fonte: o autor.

Descrição da Imagem: trata-se de um quadrado dividido por duas linhas horizontais. Na primeira parte, encontra-se o nome da classe "Pessoa". Na segunda parte, encontra-se o nome dos métodos, nome: String e idade: int, cada um com o sinal de positivo, o que indica ser público. Fim da descrição.

Código:

```
public class Pessoa
{
    private String nome;
    private int idade;
}
```

A classe é um conceito fundamental na programação orientada a objetos, dado que serve como um molde para a criação de objetos. Ela encapsula os atributos e os métodos que definem o comportamento e as características de determinado tipo de entidade.



INDICAÇÃO DE FILME

Ex Machina.

Um jovem programador de computadores ganha um concurso na empresa onde trabalha para passar uma semana na casa do brilhante e recluso presidente da companhia. Após a chegada, percebe que foi o escolhido para participar de um teste com uma robô com Inteligência Artificial (IA).

O filme aborda a Inteligência Artificial e o Machine Learning, focando no modo como uma IA pode aprender, adaptar-se e ter comportamentos únicos. A ideia de uma entidade autônoma com atributos e métodos pode ser relacionada a objetos em programação.



Por meio da herança, as classes podem compartilhar funcionalidades, promovendo a reutilização de um código e a criação de hierarquias organizacionais. Além disso, a classe permite a abstração, tornando o desenvolvimento mais intuitivo e modular. Em suma, compreender classes é essencial para escrever um código eficiente e organizado.

*compreender
classes é
essencial para
escrever um
código eficiente
e organizado.*



TIPOS DE RELACIONAMENTO

Em um software desenvolvido, as classes não atuam sozinhas. Elas interagem umas com as outras, criando relações de dependência, assim como os objetos interagem no mundo real. Por exemplo, em um laboratório de informática de uma faculdade, há pessoas. Essas pessoas se dividem entre professores e alunos; temos móveis, que podem ser cadeiras e mesas; há, também, computadores, projetores, entre outros objetos.

Os principais tipos de relacionamento serão abordados nos próximos tópicos.

Herança

Trata-se de uma situação em que uma classe herda os atributos e os métodos de outra classe. Isso permite que você, estudante, reutilize o código e crie hierarquia de classes.



A Herança é a habilidade de construir novas classes em cima de classes já existentes. O maior benefício da herança é a reutilização de código. Se você quer criar uma classe que é apenas um pouco diferente de uma já existente, não há necessidade de duplicar o código. Ao invés disso, você estende a classe existente e coloca a funcionalidade adicional dentro de uma subclasse resultante, que herdará todos os campos de métodos da superclasse” (Shvets, 2021, p. 18).

No exemplo citado no início deste tópico, temos a classe pessoa com duas subclasses, que são “Professor” e “Aluno”: essas subclasses herdam atributos de pessoa. No meu código, eu poderia utilizar os atributos da classe pessoa na classe Professor, pois ela estaria herdando esses atributos.

***No meu código,
eu poderia
utilizar os
atributos da
classe pessoa na
classe Professor,***

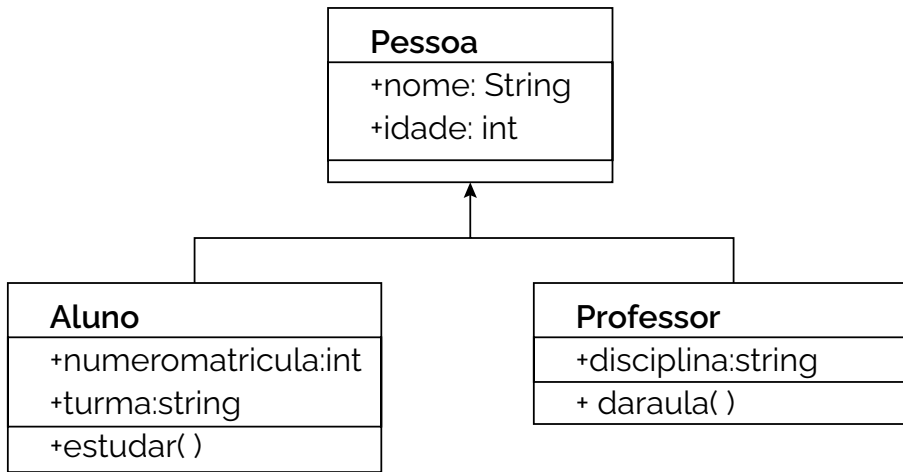
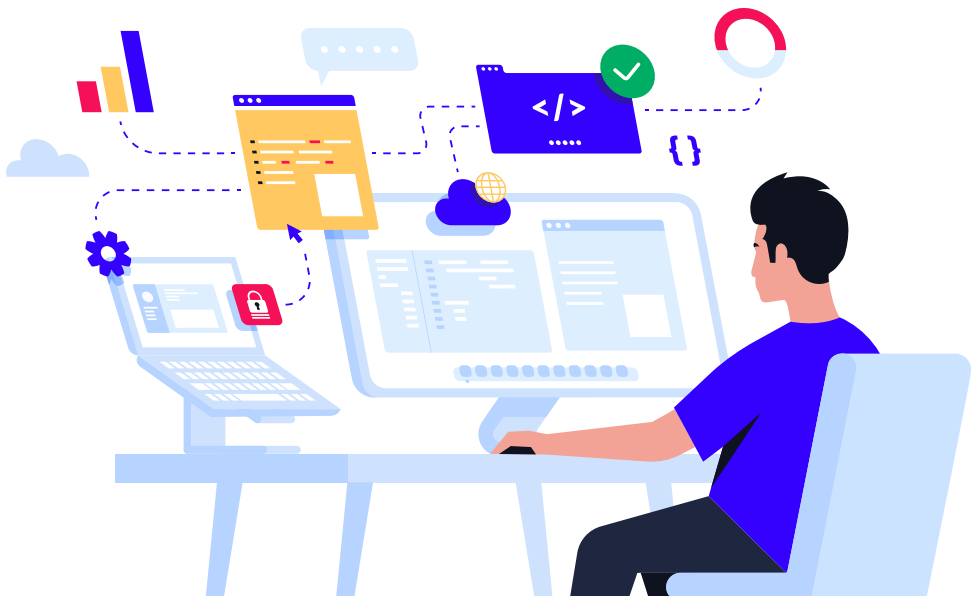


Figura 2 - Representação gráfica das classes aluno e professor herdando a classe pessoa
Fonte: o autor.

Descrição da Imagem: são exibidos quadrados divididos por duas linhas horizontais. Há um quadrado para pessoa, escrito nome:String, idade:int; um para aluno, escrito numeromatrricula:int, turma:string e, abaixo, outra linha escrito estudar(), representando uma ação (método); e um quadrado para professor, escrito disciplina:string e escrito o método daraula(). Fim da descrição.

Associação

Como temos vários objetos relacionados, as classes podem obedecer a uma relação de associação que permite que uma instância de uma classe interaja com a instância de outra.



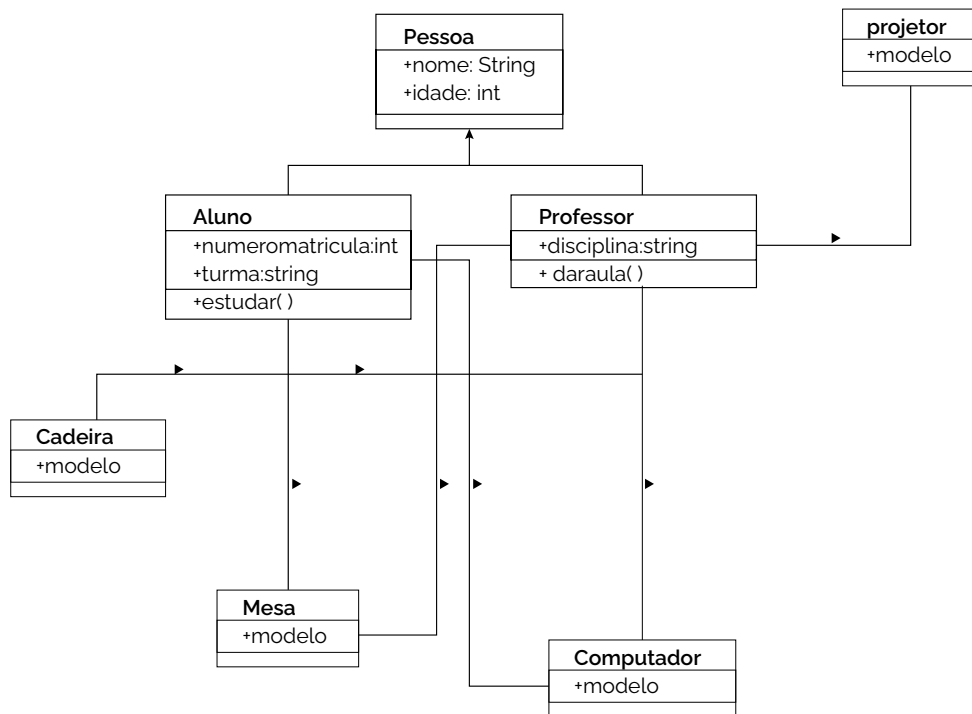


Figura 3 - Representação gráfica das classes interligadas mediante a associação / Fonte: o autor.

Descrição da Imagem: são exibidos quadrados divididos por duas linhas horizontais. Há um quadrado para pessoa, escrito nome:String, idade:int; um para aluno, escrito numeromatrícula:int, turma:string e, abaixo, outra linha escrito estudar(), representando uma ação (método); um quadrado para professor, escrito disciplina:string e o método daraula(); e quadrado para mesa, cadeira, computador e projetor, com os atributos (linhas escritas), modelo, tamanho e modelo, respectivamente, em cada uma delas. Fim da descrição.

Agregação

Temos, também, a relação de agregação, que é um tipo de associação. Ela representa o todo e a parte.

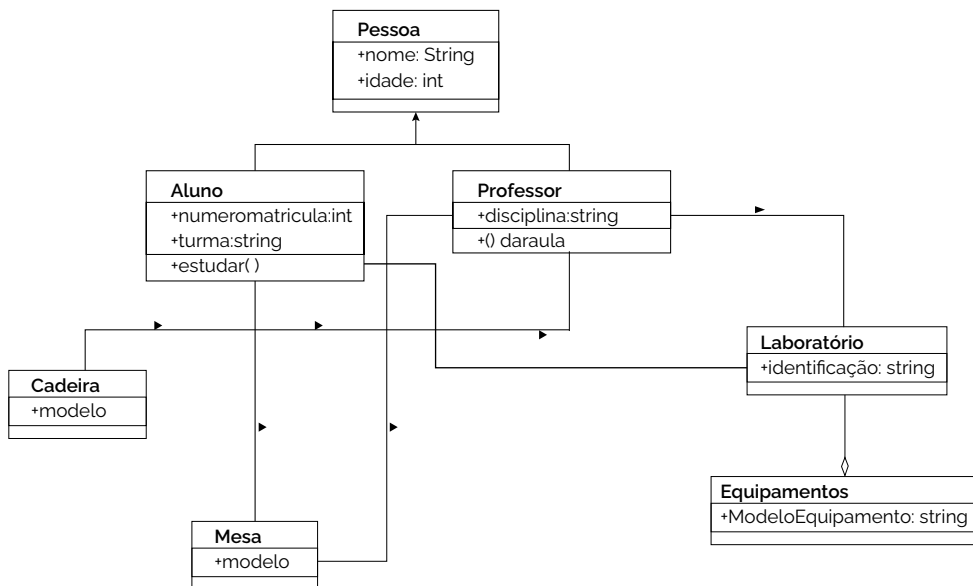


Figura 4 - Representação gráfica das classes interligadas mediante a associação / Fonte: o autor.

Descrição da Imagem: são exibidos quadrados divididos por duas linhas horizontais. Há um quadrado para pessoa, escrito nome:String, idade:int; um para aluno, escrito numeromatrícula:int, turma:string e, abaixo, outra linha escrito estudar(), representando uma ação (método); um quadrado para professor, escrito disciplina:string e o método daraula(); um quadrado para mesa e cadeira, com os atributos (linhas escritas), modelo e tamanho, respectivamente, em cada uma delas; um quadrado chamado laboratório, escrito Identificação:String; e um quadrado Equipamentos, com o texto ModeloEquipamento do tipo String, interligados por uma associação, ou seja, uma linha com um losango na ponta, indicando agregação. Fim da descrição.

Há, também, a relação de **composição**, que é semelhante à agregação. Entretanto, ela tem uma relação mais forte. Na agregação, os objetos podem existir de forma independente, porém, na composição, não podem. Por exemplo, em uma relação carro e motor, o motor é a parte que não pode existir sozinha.

Dependência

Uma classe depende da outra para funcionar corretamente. Por exemplo, se, na classe professor, temos um método chamado "realizar a apresentação" e, para esse, passamos o objeto projetor como parâmetro, temos uma dependência.

DIAGRAMAS

Os diagramas UML são ferramentas visuais fundamentais para a modelagem de sistemas, proporcionando uma representação clara e organizada de conceitos complexos. Eles auxiliam na visualização da estrutura, comportamento e interações dentro de um sistema, facilitando a comunicação entre os membros da equipe e as partes interessadas. Com uma linguagem padronizada, esses diagramas ajudam a eliminar ambiguidades e promovem uma compreensão compartilhada.

DIAGRAMA DE CLASSES

Assim como já estudamos, as classes podem ser relacionadas, formando um diagrama de relacionamento entre essas classes. Esse diagrama de classes auxilia a entender como as classes interagem e como deve ser a codificação dessas classes e relacionamentos, contribuindo para uma comunicação clara e eficaz.

DIAGRAMA DE SEQUÊNCIA

Para ilustrar como os objetos interagem entre si ao longo do tempo, podemos utilizar um diagrama de sequência. Assim como afirmam Pressman e Maxim (2016, p. 877):



Ele mostra a ordem temporal na qual as mensagens são enviadas entre os objetos para executar aquela tarefa. Podemos usar um diagrama de sequência para mostrar as interações em um caso de uso ou em um cenário do sistema de software”.

No diagrama de sequência, cada objeto participante tem uma linha de vida, uma linha vertical que representa o tempo de participação de cada um ao longo da interação.

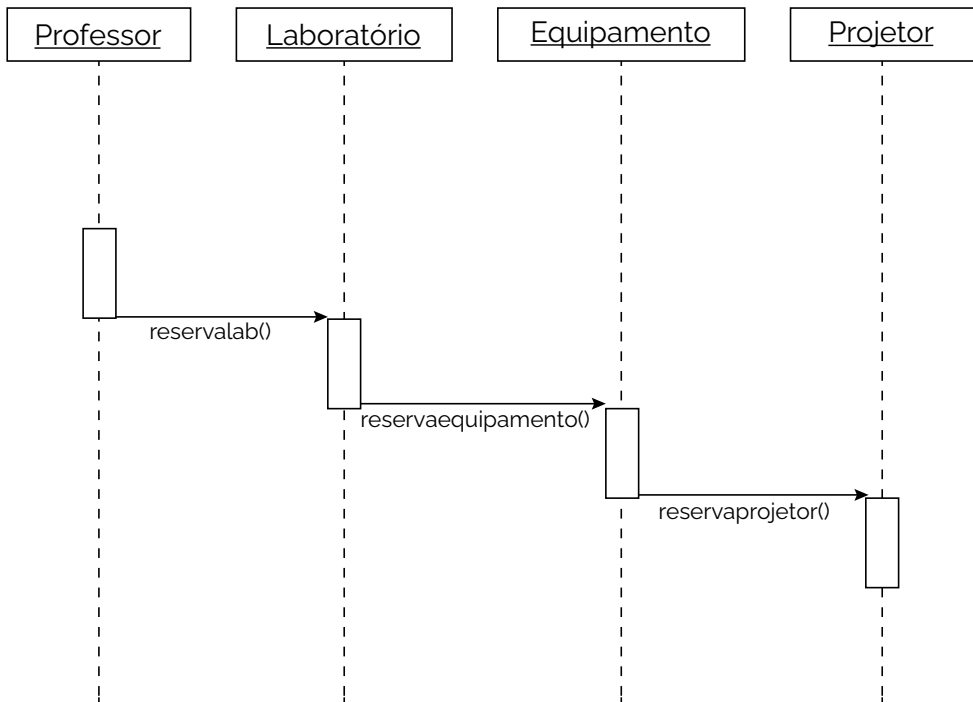


Figura 5 - Representação gráfica das classes interligadas mediante a associação / Fonte: o autor.

Descrição da Imagem: são exibidos quadrados com os textos Professor, Laboratório, Equipamento e Projetor. Há linhas pontilhadas na vertical e pequenos retângulos representando cada método com os textos reservab(), reservaequipamento() e reservaprojetor(), interligados por linha contínua. Fim da descrição.

Nesse exemplo, cada objeto é representado (Professor, Laboratório, Equipamento e Projetor) e cada método desses objetos chama o próximo com uma mensagem representada na linha contínua e com um texto. Em outras palavras, professor reserva o lab, que reserva o equipamento, que reserva o projetor especificamente.

DIAGRAMA DE MÁQUINA DE ESTADOS

O diagrama de máquina de estados facilita a visualização do ciclo de vida de um objeto e o modo como ele reage a eventos em diferentes momentos, tornando-o ideal para representar o comportamento dinâmico e reativo dos sistemas.

Martins (2011, p. 183) explica que:



Cada caixa representa um estado e as linhas representam os eventos que provocam a mudança dos estados e as ações decorrentes, na vertical os estados podem ser opcionalmente agrupados em raias que representam parte do sistema”.

“Diagramas de estados retratam principalmente estados e transições”. Os estados são demarcados “por retângulos com cantos arredondados e rotulados com o nome do estado. As transições são marcadas com setas que fluem de um estado para outro, mostrando como os estados mudam” (Lucidchart, 2024).

No exemplo a seguir, visualizamos a representação de um diagrama de estados para um semáforo, mostrando os estados verde, amarelo e vermelho.

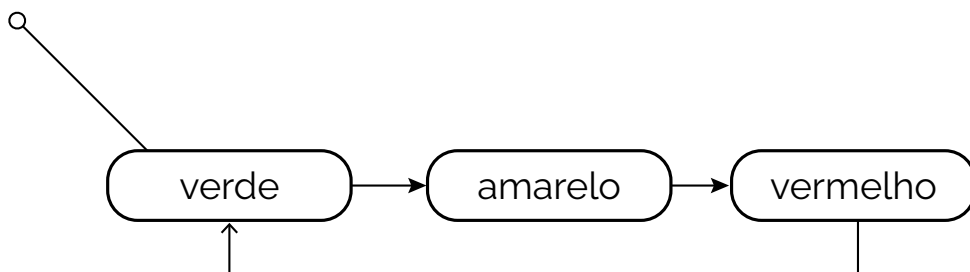


Figura 6 - Diagrama de estados de um semáforo / Fonte: o autor.

Descrição da Imagem: são exibidos três quadrados com cantos arredondados representando os estados verde, amarelo, vermelho. Eles são interligados por linhas contínuas. Fim da descrição.

Cada quadrado representa um dos estados de um semáforo, mostrando a transição entre verde, amarelo e vermelho, retornando ao verde.

DIAGRAMA DE ATIVIDADES

O diagrama de atividades mostra a sequência das atividades que ocorrem em um software. Ele se aproxima muito de um fluxograma, mas é um diagrama em UML mostrando as ações.

O diagrama a seguir exemplifica as atividades de um empréstimo de livros em uma biblioteca.

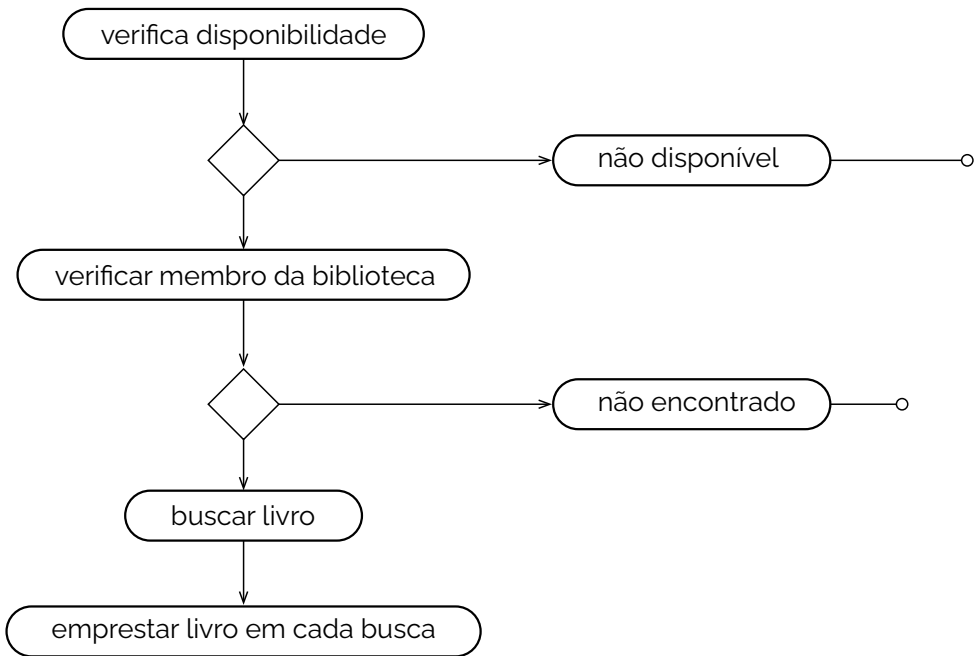


Figura 7 - Diagrama de atividades de uma biblioteca / Fonte: o autor.

Descrição da Imagem: são exibidas quatro atividades representadas por um retângulo com as bordas arredondadas. Há vários textos, como “verificar disponibilidade”, “verificar membro da biblioteca”, “buscar livro” e “emprestar livro”. Temos outro retângulo com o texto “não encontrado”. Fim da descrição.

É possível perceber que os diagramas são valiosos em várias fases do desenvolvimento, desde a análise de requisitos até a documentação final. Ao apresentarem informações de forma gráfica, tornam o planejamento e a implementação mais eficazes. Em resumo, os diagramas UML são essenciais para o sucesso de projetos de software.

NOVOS DESAFIOS

Os diagramas UML (*Unified Modeling Language*) desempenham um papel vital na ligação entre a teoria e a prática no desenvolvimento de software, oferecendo uma base sólida para projetar, documentar e entender sistemas complexos. No mercado de trabalho, a habilidade de criar e interpretar diagramas UML é fundamental para a construção de softwares eficientes e bem estruturados.

Os diagramas UML fornecem uma representação visual do sistema, que pode ser usada para comunicar conceitos complexos de forma clara e concisa. Eles ajudam a mapear a estrutura e o comportamento dos sistemas, permitindo que as equipes de desenvolvimento compreendam melhor os requisitos e o funcionamento interno do software. Isso facilita a colaboração entre desenvolvedores, analistas e outras partes interessadas, promovendo uma compreensão compartilhada do sistema e suas interações.

No ambiente profissional, a aplicação prática dos diagramas UML abrange diversas áreas. Durante a fase de design, os diagramas ajudam a definir e a refinar a arquitetura do sistema, assegurando que todos os componentes estejam corretamente integrados e alinhados com os requisitos.

Durante o desenvolvimento, eles servem como uma referência para implementar e testar o software, garantindo que o produto esteja conforme o planejado. Além disso, os diagramas UML são essenciais para a documentação do sistema, o que é crucial para a manutenção e a evolução contínua do software.

VAMOS PRATICAR

1. Em contraste com os diagramas de classe e de implantação, que mostram a estrutura estática de um componente de software, o diagrama de sequência é utilizado para indicar as comunicações dinâmicas entre objetos durante a execução de uma tarefa.

Considerando a função do diagrama de sequência, marque a alternativa correta:

- a) Mostra como as classes do sistema se relacionam.
 - b) Mostra como os objetos mudam o estado.
 - c) Mostra como os objetos interagem entre si ao longo do tempo.
 - d) Mostra a interação entre os casos de uso.
 - e) Mostra a sequência das atividades.
2. Para modelar classes, incluindo propriedades, operações, relações e associações com outras classes, a UML tem um diagrama de classe. Um diagrama de classe fornece uma visão estática ou estrutural do sistema. Ele não mostra a natureza dinâmica das comunicações entre os objetos das classes no diagrama.

Considerando a temática apresentada, analise as afirmativas a seguir:

- I - Classes são elementos de estrutura e têm atributos e métodos.
- II - Associações entre classes indicam relacionamentos
- III - Agregações e composições são tipos específicos de associações

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

VAMOS PRATICAR

3. Os elementos principais são caixas, ou seja, ícones usados para representar classes e interfaces. Cada caixa é dividida em partes horizontais. A parte superior contém o nome da classe. A seção do meio lista os atributos da classe. Os atributos podem ser valores que a classe calcula a partir de suas variáveis de instância ou valores que a classe pode obter de outros objetos dos quais é composta.

A respeito do diagrama de classes, assinale a alternativa correta:

- a) Mostra a ordem temporal na qual as mensagens são enviadas.
- b) Auxilia a entender como as classes interagem e como deve ser a codificação dessas classes e relacionamentos.
- c) Possui a linha de vida, que é uma linha vertical que parte de cada participante representando o tempo de participação.
- d) Facilita a visualização do ciclo de vida de um objeto.
- e) Cada caixa representa um estado, enquanto as linhas representam os eventos que provocam a mudança dos estados.

REFERÊNCIAS

LUCIDCHART. O que é um diagrama de estados em UML?. **LUCIDCHART**, c2024. Disponível em: <https://www.lucidchart.com/pages/pt/o-que-e-diagrama-de-maquina-de-estados-uml>. Acesso em: 29 out. 2024.

MARTINS, J. C. C. **Gerenciando projetos de desenvolvimento de software com PMI, RUP e UML**. 5. ed. Rio de Janeiro: Brasport, 2011.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Belo Horizonte: McGraw Hill, 2016.

SHVETS, A. **Mergulho nos padrões de projeto**. [S. l.: s. n.], 2021.

CONFIRA SUAS RESPOSTAS

1. Opção C.

A alternativa C está correta, porque se refere ao diagrama de sequência, que mostra a interação ao longo do tempo.

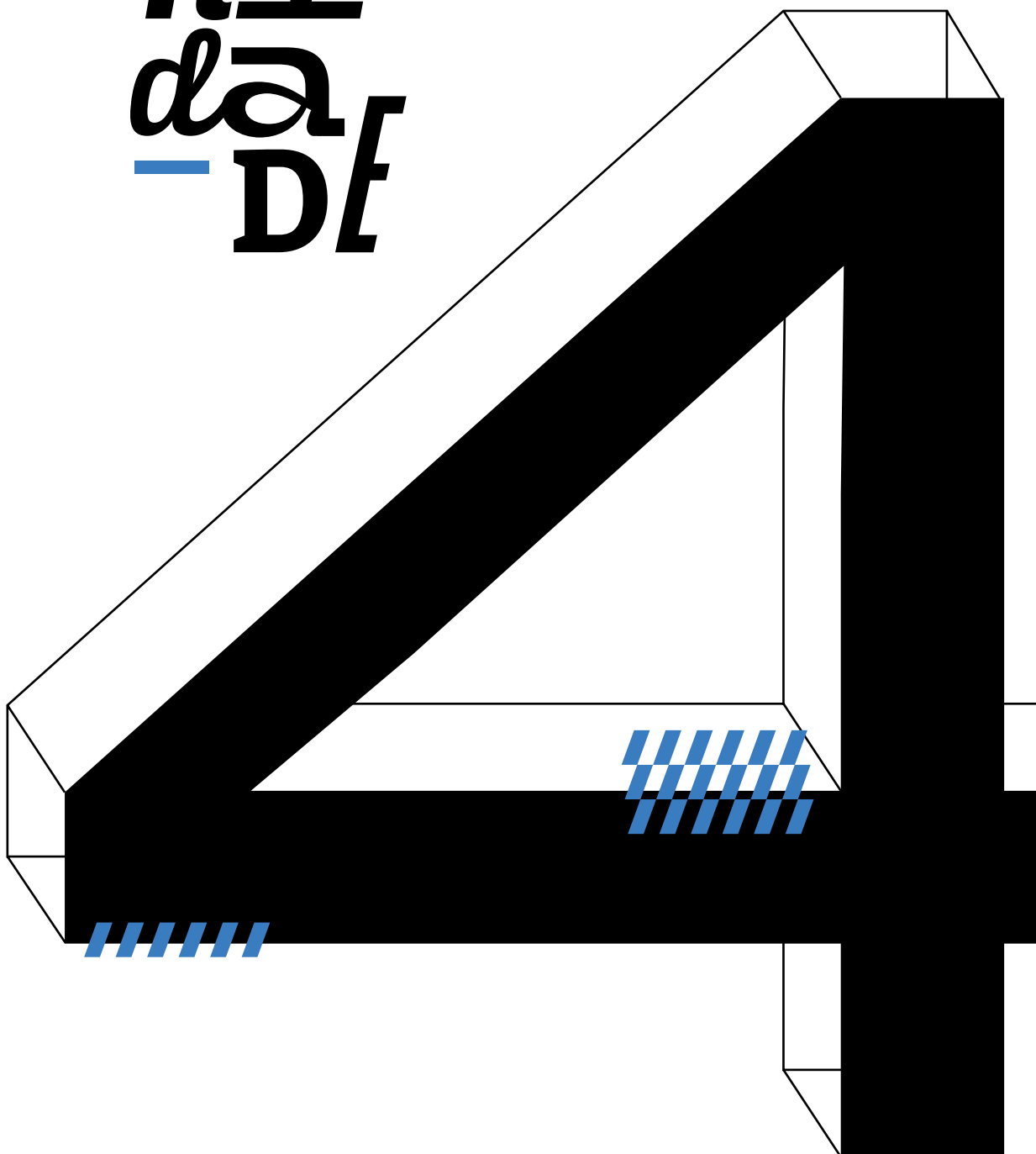
2. Opção E.

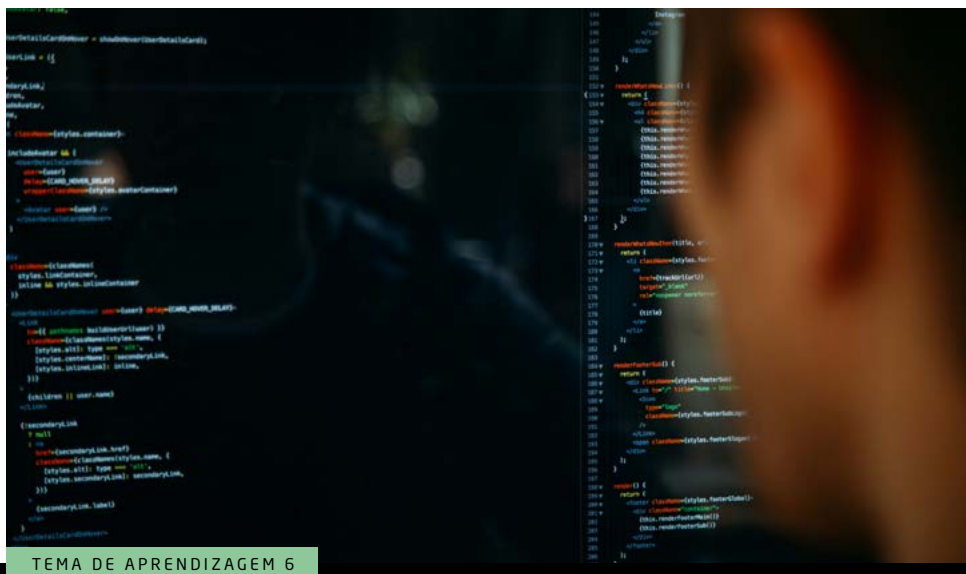
A afirmativa I é verdadeira, porque as classes são representações de elementos no sistema com atributos e métodos. A afirmativa II é verdadeira, porque as associações mostram como as classes se relacionam, definindo ligações entre elas. A afirmativa III é verdadeira, porque a agregação e a composição são variações de associações, com diferenças em dependência.

3. Opção B.

A alternativa A está incorreta, pois se refere ao diagrama de sequência. A alternativa B está correta, pois aborda o relacionamento entre as classes. A alternativa C está incorreta, pois se refere ao diagrama de sequência. As alternativas D e E estão incorretas, pois se referem à máquina de estados.

**u
ni
da
-
DE**





IMPLEMENTAÇÃO DE SOFTWARE

MINHAS METAS

- Compreender o processo de implementação de software, desde o planejamento até a entrega.
- Reconhecer as principais atividades da implementação;
- Identificar as características de uma implementação eficaz.
- Adotar boas práticas de estilo de programação.
- Desenvolver habilidades de depuração para identificar bugs de forma eficiente.
- Implementar processos de revisão de código.
- Aprender o uso adequado de comentários em código.

INICIE SUA JORNADA

Muitas vezes, você, estudante, pode enfrentar problemas em conectar a teoria aprendida na sala de aula com as exigências do mercado de trabalho. A implementação de software é um exemplo disso: apesar de ser amplamente abordada na teoria, na prática, existem inúmeros desafios. Um dos maiores desafios é transformar o conhecimento teórico em uma solução funcional e de qualidade. Durante a implementação, é preciso lidar com situações complexas e garantir que as atividades sejam executadas de forma a minimizar erros, maximizar resultados e, ao mesmo tempo, cumprir prazos e respeitar os orçamentos.

Essa é uma etapa em que o conhecimento técnico ganha significado. Cada boa prática preconizada é fundamental para a construção de sistemas manuteníveis e escaláveis. A adoção dessas práticas torna a equipe mais eficiente e facilita a colaboração entre os membros do time.

A implementação do software oferece a oportunidade de praticar e experimentar esses conceitos, pois muitas habilidades apenas se aprimoram com a prática constante, aumentando o olhar crítico sobre as situações que se apresentam no dia a dia.

É fundamental que você, estudante, reflita sobre todo esse processo, pois o processo de codificação e implementação é muito mais extenso do que se pode pensar, indo além de simplesmente uma execução, tendo um extenso planejamento prévio.



PLAY NO CONHECIMENTO

Neste podcast, abordamos o tema *Documentação vs. Código Limpo: Equilíbrio na Implementação de Software*. Ouça para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal relembrar as atividades básicas de desenvolvimento? É interessante relembrar, pois são atividades comuns a todos os projetos de desenvolvimento.

<https://www.devmedia.com.br/modelagem-de-sistemas-atraves-de-uml-uma-visao-geral/27913>

DESENVOLVA SEU POTENCIAL

A implementação de software é uma etapa fundamental no desenvolvimento, dado que é momento em que as especificações do projeto são transformadas em código funcional. Durante essa fase, os desenvolvedores usam linguagens de programação e ferramentas adequadas para construir as funcionalidades planejadas.

A qualidade do código depende da aplicação de boas práticas, como clareza na estrutura, além de técnicas eficazes de testes e depuração. A cooperação entre as equipes, o uso de sistemas de controle de versão e uma boa documentação são essenciais para garantir um processo organizado. No fim, o software deve estar pronto para ser testado, integrado e entregue aos usuários finais.

A qualidade do código depende da aplicação de boas práticas, como clareza na estrutura, além de técnicas eficazes de testes e depuração.



Nessa etapa, o sistema é codificado a partir da descrição computacional da fase de projeto em uma outra linguagem, onde se torna possível a compilação e geração do código-executável para o desenvolvimento software.

Em um processo de desenvolvimento orientado a objetos, a implementação se dá, definindo as classes de objetos do sistema em questão, fazendo uso de linguagens de programação como, por exemplo: Delphi (Object Pascal), C++, Java, etc. Pode-se também utilizar na implementação ferramentas de software e bibliotecas de classes preexistentes para agilizar a atividade, como também o uso de ferramentas CASE, que dinamizam o processo de desenvolvimento, nas várias atividades, onde inclui-se geração de código-fonte, documentação etc. (DevMedia, 2024).

Ao passar por todas as etapas, o software será mais robusto e, com certeza, atenderá às necessidades do cliente.

IMPLEMENTAÇÃO DE SOFTWARE

O que foi desenhado no sistema será efetivamente construído e codificado. Esse código-fonte será convertido no executável, envolvendo a linguagem de programação do projeto e ferramentas e metodologias adequadas. Isso faz com que o planejamento levantado seja convertido em software rodando.

Essa fase tem, como principal objetivo, garantir que o que está sendo desenvolvido seja o que foi, de fato, planejado. Portanto, os requisitos funcionais e não funcionais que foram estabelecidos previamente devem ser atendidos em sua totalidade.

Para uma implementação bem-sucedida, é crucial que sejam seguidos alguns passos, como objetivos bem definidos. Esses objetivos devem atender às necessidades definidas. O **planejamento** deve ser seguido e, **após o desenvolvimento**, deve-se ter um **treinamento**, uma **comunicação efetiva**, uma boa bateria de **testes** e o **feedback** sobre a aplicação. Todos esses pontos constituem os fatores críticos de sucesso para o software.



Quando o modelo de implementação e o modelo mental do usuário são coincidentes, em geral os usuários sentem-se à vontade com o software e o utilizam de maneira eficaz. Para conseguir tal “amálgama” dos modelos, o modelo de projeto deve ter sido desenvolvido levando em conta as informações contidas no modelo de usuário, e o modelo de implementação deve refletir precisamente as informações sintáticas e semânticas sobre a interface (Pressman; Maxim, 2016, p. 323).

O código deve ser:

- **Funcional:** deve executar todas as funções planejadas previamente.
- **Manutenível:** o software deve ser de fácil manutenção, então, deve ser claro e organizado.
- **Eficiente:** deve utilizar recursos de forma otimizada.

Algumas atividades são fundamentais para que essa fase seja executada com sucesso. São elas:

- **Codificação:** momento em que o código é, de fato, desenvolvido, sempre observando as boas práticas.
- **Testes unitários:** são executados os testes do desenvolvedor, ou seja, chamadas individuais.
- **Integração:** a integração entre os diferentes módulos é fundamental para o funcionamento do sistema.
- **Documentação:** embora, muitas vezes, relegada a um segundo plano, a documentação é fundamental para a construção e manutenção do software.



O processo de testes começa com testes que experimentam o conteúdo e a funcionalidade da interface, pontos imediatamente visíveis para os usuários. Na medida em que o teste é realizado, são experimentados aspectos da arquitetura de projeto e da navegação da WebApp. Por fim, o foco muda para testes que examinam os recursos tecnológicos que nem sempre são aparentes para os usuários – tópicos de infraestrutura e de instalação/implementação da WebApp (Pressman; Maxim, 2016, p. 544).

Os testes são cruciais ao desenvolvimento de software. Com eles, os *bugs* são detectados a tempo de serem corrigidos antes de o sistema ser entregue, garantindo uma maior confiabilidade à solução.



INDICAÇÃO DE FILME

Jobs

Focado na vida de Steve Jobs, o filme explora os desafios da criação de produtos revolucionários no mundo da tecnologia, como o desenvolvimento do iPhone e do Macintosh, além de mostrar como ideias inovadoras são implementadas e lançadas no mercado.

A implementação de software é um processo-chave na criação desses produtos, e o filme mostra várias camadas dessa jornada.





*o filme explora os desafios
da criação de produtos
revolucionários no mundo
da tecnologia, como
o desenvolvimento do
iPhone e do Macintosh*

Embora os testes, a documentação ou os outros passos citados sejam fundamentais para o sucesso do software, a comunicação é um fator crucial em qualquer projeto, podendo ser determinante para o sucesso dele.

METODOLOGIAS DE IMPLEMENTAÇÃO

As metodologias de implementação definem como a implementação do software será conduzida, podendo ser *waterfall* ou *agile*.

No método ***waterfall***, ou cascata, as atividades são faseadas, sendo executadas etapa por etapa, de forma a construir uma figura semelhante a uma cascata.



O estágio de implementação no desenvolvimento de software é o processo de elaborar um sistema executável para ser entregue ao cliente. Às vezes, isso envolve atividades distintas, que são o projeto (design) e a programação do software. No entanto, se uma abordagem ágil for utilizada para o desenvolvimento, o projeto e a implementação são intercalados, sem documentos de projeto (design) formais produzidos durante esse processo. Naturalmente, o software ainda é projetado, mas o projeto está registrado informalmente nas lousas ou nas anotações feitas pelos programadores. O projeto de software é uma descrição da estrutura do software a ser implementado, dos modelos e estruturas de dados utilizados pelo sistema, das interfaces entre os componentes do sistema e, às vezes, do algoritmo utilizado. Os projetistas não chegam a um projeto acabado imediatamente, mas o desenvolvem em estágios. Eles acrescentam detalhes à medida que desenvolvem seu projeto, com revisões constantes para modificar os projetos iniciais (Sommerville, 2018, p. 41).

A abordagem ágil vem ganhando terreno, sobretudo, no desenvolvimento de software, que, muitas vezes, é um ambiente com requisitos instáveis. Desse modo, com as revisões e as entregas constantes, mostra-se uma ótima alternativa ao desenvolvimento.

No método ágil, o projeto passará por sucessivas iterações, com ciclos curtos passando por planejamento, desenvolvimento e implantação em ciclos de algumas semanas. Em outras palavras, torna-se mais flexível para que o cliente avalie com uma frequência maior o software gerado.



A metodologia ágil abrange também a filosofia proposta no manifesto ágil. Ela incentiva a estruturação e as atitudes em equipe que tornam a comunicação mais fácil (entre membros da equipe, entre o pessoal ligado à tecnologia e o pessoal da área comercial, entre os engenheiros de software e seus gerentes). Enfatiza a entrega rápida do software operacional e diminui a importância dos artefatos intermediários (nem sempre um bom negócio); aceita o cliente como parte da equipe de desenvolvimento e trabalha para eliminar a atitude de “nós e eles” que continua a impregnar muitos projetos de software; reconhece que o planejamento em um mundo incerto tem seus limites e que o plano (roteiro) de projeto deve ser flexível (Pressman; Maxim, 2016, p. 68).

Essas iterações podem seguir um intervalo de tempo definido, em geral, duas semanas. Com isso, as entregas são sucessivas e podem gerar tempo para a avaliação de novas funcionalidades.

FERRAMENTAS

Algumas ferramentas contribuem para a eficiência do processo de desenvolvimento de software. Alguns critérios a serem considerados na escolha da ferramenta são:

- **Compatibilidade:** é necessário verificar se é compatível com todo o ambiente de software, se essa ferramenta se integra bem com outros sistemas
- **Produtividade:** ferramentas com vasta documentação e que otimizem o seu tempo de desenvolvimento devem ser consideradas.
- **Funcionalidades:** verificar se a ferramenta atende bem, se tem o que você precisa, depuração, integração com sistemas de versionamento, característica esperada (exemplo: Orientado a Objetos).
- **Suporte:** ferramentas que têm bom suporte para você utilizar quando necessário.

Livre x Pago

As ferramentas livres são boas e vêm ganhando muita visibilidade no mercado. Apresentam vantagens, como:

- **Flexibilidade:** podem ter o código alterado.
- **Comunidade:** as ferramentas *open source* possuem uma grande comunidade que contribuem com atualizações e auxílio.
- **Custo:** com relação ao custo, como são gratuitas, fornecem uma grande vantagem, principalmente para empresas iniciantes.
- Por outro lado, as ferramentas pagas apresentam como vantagens:
- **Melhor suporte:** você tem um suporte técnico para contatar em caso de necessidade.
- **Funcionalidades mais avançadas e integrações:** muitas integrações com outras ferramentas do próprio fabricante (pode ser usado, como exemplo, o Office, que tem muitas integrações com outros sistemas Microsoft).
- **Atualizações:** são constantes e regulares.

Quanto à utilização, as ferramentas livres e pagas podem ser utilizadas em um mesmo ambiente, formando uma mesma solução.

Por exemplo, pode ser usado o VSCode (gratuito) com o GithubEnterprise, que é pago, ou o Android Studio com o github gratuito, e assim por diante.

Como categorias de ferramentas, podemos citar:

IDE (INTEGRATED DEVELOPMENT ENVIRONMENT):

esses ambientes integrados de desenvolvimento aumentam a produtividade, ao facilitar o trabalho com linguagens específicas. Algumas IDEs amplamente conhecidas incluem o Eclipse, voltado principalmente para o desenvolvimento em Java, e o Visual Studio, muito utilizado por desenvolvedores .NET. O VSCode também é bastante popular, oferecendo suporte abrangente para desenvolvimento e depuração.

CONTROLADORES DE VERSÃO:

ferramentas, como o Github, SVN e Team System, auxiliam no trabalho em equipe, gerenciando as diferentes versões do código-fonte. No Github, você pode inserir o código da sua aplicação, controlar versões, facilitar o trabalho em equipe e outras opções de revisão de código.

FERRAMENTAS CI (CONTINUOUS INTEGRATION):

ferramentas nesse segmento auxiliam o *deploy* de forma contínua, fazendo com que o tempo entre o desenvolvimento e os testes seja reduzido. Um grande representante desse segmento é o Bitbucket, produto da Atlassian.

Diante disso, o código é submetido a um controlador de versão que controla e distribui para a equipe a versão correta do software.

Após o desenvolvimento finalizado, ferramentas de Continuous Integration podem ser utilizadas para fazer o *deploy* da aplicação, percorrendo uma parte considerável do ciclo de desenvolvimento de software.

As ferramentas citadas são fundamentais ao desenvolvimento, pois, a partir delas (a IDE, por exemplo), o código será criado

DESAFIOS DA IMPLEMENTAÇÃO DE SOFTWARE

A implementação de software é um ambiente complexo, cercado por vários desafios que são enfrentados no dia a dia. Como exemplo, podemos citar os sistemas legados, que são um dos maiores desafios na manutenção ou integração dos softwares novos.

A complexidade excessiva no desenvolvimento pode gerar grandes problemas, especialmente em termos de manutenção e colaboração com outros desenvolvedores. Por exemplo, é comum encontrar equipes de desenvolvimento com um código que apenas um desenvolvedor entende e consegue manter. Portanto,

buscar a simplicidade é essencial para facilitar o trabalho em equipe e garantir um código mais acessível e sustentável.

Quando falamos de simplicidade, um ponto fundamental é a reutilização de código. A seguir, Schvetz (2021, p. 37) cita níveis de reutilização de códigos:



Eu vejo três níveis de reutilização. No nível mais baixo, você reutiliza classes: classes de bibliotecas, contêineres, talvez “times” de classes como o contêiner/iterator. Os frameworks são o mais alto nível. Eles realmente tentam destilar suas decisões de projeto. Eles identificam abstrações importantes para a solução de um problema, representam eles por classes, e definem relações entre eles. O JUnit é um pequeno framework, por exemplo. Ele é o ‘Olá, mundo’ dos frameworks. Ele tem o Test, TestCase, TestSuite e relações definidas.

Outro ponto que representa um desafio constante é a **performance**, que deve ser sempre perseguida, pois geralmente consiste no gargalo da aplicação.

Boas práticas na implementação

A adoção de **boas práticas** também é imprescindível para o desenvolvimento e a implementação de software, pois elas garantem a criação de sistemas robustos, eficientes e sustentáveis, que podem ajudar a resolver problemas complexos de forma mais eficiente, além de facilitarem a manutenção e a evolução do sistema.

PADRÕES DE PROJETO:

os padrões de projeto são soluções documentadas por desenvolvedores que funcionaram bem em situações específicas, servindo como guia para outros. Existem diversas categorias de padrões: os arquiteturais, que incluem MVC e Microserviços; os criacionais, que focam na criação eficiente de objetos, como o Factory Method e o Builder; os estruturais, que definem como os componentes se organizam, como o padrão Adapter, útil para integrar sistemas legados com novos, quando as interfaces são diferentes; e os comportamentais, que tratam das interações entre objetos, como o padrão Observer, que, ao alterar uma notícia (objeto principal), todos os assinantes (leitores) são notificados.

ARQUITETURA DE SOFTWARE:

quanto à arquitetura de software, podemos citar as arquiteturas monolíticas, que são implantadas em um único ambiente, e a arquitetura de microsserviços, que são chamadas para APIs, formando um sistema único, porém independente. Podemos ter também arquiteturas em camadas, que, em que cada camada do software, responde por uma área do aplicativo. Entre elas, podemos citar a MVC, em que uma camada é a responsável pela visualização (parte do front), o controler, que interpreta a ação vinda da view, e a model, que atualiza os dados.

REVISÃO DE CÓDIGO:

ferramentas, como github, podem ser utilizadas para o versionamento do código, e verificações de versão, por exemplo, comparando uma versão com outra para visualizar o que foi alterado. A revisão de código é essencial para uma boa evolução: logo, promover uma cultura de revisão de código evita bugs e garante a conformidade com boas práticas.

O uso de **testes automatizados** também agiliza essa fase tão importante e busca garantir que o processo de qualidade seja seguido, aumentando a qualidade, resultando em um software mais robusto na entrega.

Estilos de programação e codificação

Existem inúmeros estilos de programação no mercado, como programação procedural, orientado a objetos, funcional etc.

Na codificação, o uso de boas práticas consiste em um importante padrão a ser seguido, auxiliando equipes, pois, ao contrário de cada desenvolvedor usar o próprio modelo de desenvolvimento, ele passa a **seguir um padrão de escrita adotado por todos**. Isso contribui para a manutenção e a comunicação.

O uso de princípios para guiar a simplicidade na reutilização de código, como o *Don't Repeat Yourself* (DRY) ou o *Keep it Simple Stupid* (KISS), direcionam essa busca por simplicidade no código.

Comentários no código são um pouco controversos, pois, ao passo que alguns defendem que auxiliam no entendimento, outros defendem que eles poluem o código, pois o desenvolvedor entende o que o código fará, se ele for bem escrito. Então, o comentário contribui para que o código não seja limpo.



Quando um arquiteto usa a expressão “estilo colonial americano com hall central” para descrever uma casa, a maioria das pessoas familiarizadas com casas dos Estados Unidos será capaz de evocar uma imagem geral da aparência da casa e como provavelmente será a sua planta. O arquiteto usou um estilo arquitetural como um mecanismo descritivo para diferenciar a casa de outros estilos (por exemplo, casa pré-fabricada sobre uma estrutura de madeira em forma de A, rancho montado sobre uma base elevada, Cape Cod). Porém, mais importante ainda, o estilo arquitetural também é um modelo para construção (Pressman; Maxim, 2016, p. 258).

Assim como o modelo usado na arquitetura, podemos aplicar modelos para o desenvolvimento de software. O desenvolvimento de software é amparado por padrões que facilitam a comunicação entre a equipe e auxiliam na padronização do desenvolvimento e na codificação de forma clara e bem estruturada, facilitando a manutenção. Há padrões que podem seguir as categorias estruturais, comportamentais, arquiteturais ou estruturais.

O código a seguir, desenvolvido em Java, exemplifica o algoritmo de um cadastro de livros de uma biblioteca. Observe que, neste exemplo, foram inseridos comentários ao longo do código:

```
import java.util.Scanner; //Importando a biblioteca Scanner

// Definindo uma classe para armazenar informações sobre um livro

class Livro {

    String titulo;

    String autor;

    int ano;

}

public class CadastroDeLivros {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        // Solicitando ao usuário o número de livros a serem cadastrados

        System.out.print("Quantos livros deseja cadastrar? ");
```

```
int n = scanner.nextInt();

scanner.nextLine(); // Consumir a nova linha

    // Criando um array para armazenar os livros

Livro[] livros = new Livro[n];

// Loop para cadastrar cada livro
for (int i = 0; i < n; i++) {

    livros[i] = new Livro();

    System.out.println("Cadastro do livro " + (i + 1) + " :");

    // Solicitando o título do livro

    System.out.print("Título: ");

    livros[i].titulo = scanner.nextLine();

    // Solicitando o autor do livro

    System.out.print("Autor: ");

    livros[i].autor = scanner.nextLine();

    // Solicitando o ano de publicação do livro

    System.out.print("Ano de publicação: ");

    livros[i].ano = scanner.nextInt();

    scanner.nextLine(); // Consumir a nova linha
}

// Exibindo os livros cadastrados

System.out.println("\nLivros cadastrados:");

for (int i = 0; i < n; i++) {

    System.out.println("Livro " + (i + 1) + " :");

    System.out.println("Título: " + livros[i].titulo);

    System.out.println("Autor: " + livros[i].autor);

    System.out.println("Ano de publicação: " + livros[i].ano);

}

scanner.close();

}

}
```

A implementação do software com as respectivas metodologias pode trazer ganhos ou simplicidade, padronização ou performance para os projetos. Esses ganhos trazem benefícios para as empresas, de forma que os desenvolvedores com domínio dessas práticas são necessários no mercado.

NOVOS DESAFIOS

A teoria sobre implementação de software oferece a base necessária para que você entenda os conceitos fundamentais, como boas práticas de programação, controle de versão e técnicas de depuração. Entretanto, é na prática que você se prepara para o mercado de trabalho, ambiente onde esses conhecimentos são aplicados em cenários reais e desafiadores.

No ambiente profissional, os desenvolvedores enfrentam prazos apertados, colaboram em equipes diversas e utilizam metodologias ágeis, como Scrum e Kanban, para gerenciar projetos. Além disso, o setor exige que os profissionais estejam em constante atualização, acompanhando o avanço de novas tecnologias, ferramentas e frameworks.

As oportunidades são amplas, desde o desenvolvimento de sistemas web e mobile até áreas, como automação, Inteligência Artificial e DevOps. Dessa forma, a teoria é fundamental para capacitar profissionais capazes de implementar soluções eficientes, acompanhar as mudanças contínuas do setor de tecnologia e aplicar o conhecimento na prática.

VAMOS PRATICAR

1. A implementação de software representa o processo de transformar um projeto de software em um produto funcional e utilizável. Envolve a tradução das especificações e requisitos do software em código de programação, além da configuração e da integração de componentes de software para criar uma solução completa. A implementação de software é uma etapa crítica no ciclo de vida do desenvolvimento de software, pois é nesse momento que o software começa a ganhar vida e se torna disponível para uso pelos usuários finais (O que [...], 2024a).

A respeito da implementação de software e dos passos necessários, marque a alternativa correta:

- a) A criação de diagramas, como o caso de uso, é crucial para a implementação de software.
 - b) Objetivos bem definidos e planejamento são passos necessários para a implementação.
 - c) Os testes são cruciais para a implementação de software.
 - d) A implementação de software só pode ser feita no modelo *waterfall*.
 - e) O desenvolvimento em pares é fundamental à implementação.
2. Ferramentas para desenvolvedores são tecnologias que tornam o desenvolvimento de software mais rápido e eficiente. O desenvolvimento de software é um processo complexo de converter objetos do mundo real em representações matemáticas e eletrônicas que as máquinas podem entender e manipular. Ferramentas de desenvolvedor atuam como uma interface entre a realidade física e os processos de computação. Elas incluem linguagens de programação, frameworks e plataformas que abstraem diferentes níveis de complexidade (O que [...], 2024b).

Considerando a temática apresentada, analise as afirmativas a seguir:

- I - A IDE é uma ferramenta fundamental para o desenvolvimento e a depuração.
- II - Controladores de versão auxiliam o trabalho em equipe.
- III - Ferramentas CI auxiliam o *deploy*.
- IV - Algumas IDEs conhecidas são Eclipse, Visual Studio, VSCode.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

VAMOS PRATICAR

3. Um modelo de processo fornece um guia específico para o trabalho de engenharia de software. Ele define o fluxo de todas as atividades, ações e tarefas, o grau de iteração, os artefatos e a organização do trabalho a ser feito.

Considerando a temática apresentada, assinale a alternativa correta:

- a) No método *waterfall* ou cascata, as atividades são faseadas, sendo executadas etapa por etapa, de forma a construir uma figura semelhante a uma cascata.
- b) No método cascata, as fases ocorrem sem planejamento prévio.
- c) O projeto ágil é executado fase por fase.
- d) A metodologia ágil incentiva o trabalho individual.
- e) Na metodologia ágil, o cliente não é parte da equipe de desenvolvimento.

REFERÊNCIAS

DEV MEDIA. Atividades básicas ao processo de desenvolvimento de Software. **DevMedia**, c2024. Disponível em: <https://www.devmedia.com.br/atividades-basicas-ao-processo-de-desenvolvimento-de-software/5413>. Acesso em: 30 out. 2024.

O QUE é Implementação de Software? (Tecnologia). **Creapar**, c2024a. Disponível em: <https://creapardesenvolvimento.com.br/glossario/o-que-e-implementacao-de-software-tecnologia/>. Acesso em: 30 out. 2024.

O QUE são ferramentas para desenvolvedores? **AWS**, c2024b. Disponível em: <https://aws.amazon.com/pt/what-is/developer-tools/>. Acesso em: 30 out. 2024.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Belo Horizonte: McGraw Hill, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

SHVETS, A. Mergulho nos padrões de projeto. [S. l.: s. n.], 2021 .

CONFIRA SUAS RESPOSTAS

1. Alternativa B.

A alternativa A está incorreta, porque o caso de uso pode auxiliar, mas a implementação não depende desse artefato. A alternativa B está correta, pois um planejamento e uma boa definição dos objetivos são cruciais. A alternativa C está incorreta, porque os testes ocorrem após a implementação. A alternativa D está incorreta, porque a implementação não depende do modelo adotado. A alternativa E está incorreta, porque o desenvolvimento em pares é uma técnica importante, mas a implementação não depende dela.

2. Alternativa E.

A afirmativa I é verdadeira, porque a IDE é fundamental para o desenvolvimento. A afirmativa II é verdadeira, porque controladores de versão, como Github, auxiliam o trabalho em equipe. A afirmativa III é verdadeira, porque ferramentas CI agilizam o *deploy*. A afirmativa IV é verdadeira, porque são IDEs conhecidas.

3. Alternativa A.

A alternativa A está correta, porque essa é a característica do *waterfall*. A alternativa B está incorreta, pois, no *waterfall*, as fases anteriores são bem planejadas. A alternativa C está incorreta, pois a fase por fase é uma característica do *waterfall*. A alternativa D está incorreta, pois a metodologia ágil incentiva o trabalho em equipe. A alternativa E está incorreta, pois, na metodologia ágil, o cliente pode ser parte da equipe de desenvolvimento.



TEMA DE APRENDIZAGEM 7

GERENCIAMENTO DE SOFTWARE

MINHAS METAS

- Compreender a importância do planejamento no desenvolvimento de software.
- Relacionar o planejamento e a gestão ao sucesso do software.
- Entender a área de qualidade de software.
- Definir os testes de software.
- Explorar a evolução do software.
- Conhecer a configuração de software.
- Refletir a respeito da utilização dos processos no desenvolvimento de software.

INICIE SUA JORNADA

No atual cenário competitivo, o desenvolvimento de software exige um planejamento e um gerenciamento cuidadoso, o que torna essa disciplina essencial para qualquer profissional de Tecnologia da Informação (TI). Um software lançado com muitos *bugs* e falhas não apenas gera prejuízos financeiros, mas também pode prejudicar a imagem da empresa, resultando na perda de confiança por parte dos usuários.

A garantia de qualidade é um processo fundamental na entrega de software. Para assegurar que um produto tenha alta qualidade, é necessário contar com uma área de testes bem definida, um planejamento estruturado e uma gestão de processos eficaz. Esses elementos são essenciais para o sucesso de qualquer projeto de desenvolvimento.

Por exemplo, ao desenvolver um software para o controle de Recursos Humanos, a qualidade final do produto dependerá fortemente dos processos seguidos durante o desenvolvimento. Portanto, é importante refletir sobre a aplicação de práticas de gestão e o respectivo impacto na construção de software, analisando como esses processos influenciam o sucesso do projeto como um todo.



PLAY NO CONHECIMENTO

Nesse podcast, abordaremos o tema *Gestão de Projetos e a Comunicação*. Ouça para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal relembrar as fases de um projeto? É interessante relembrá-las, porque são etapas comuns a todos os projetos vida.

<https://www.youtube.com/watch?v=NSNfzEz1dn8>

DESENVOLVA SEU POTENCIAL

Em um tempo em que temos um mercado cada vez mais competitivo e o diferencial entre as empresas pode se resumir ao menor detalhe, torna-se indesejável que tenhamos um software lançado no mercado apresentando problemas de qualidade, tendo que lançar inúmeras atualizações para corrigir o sistema. Contudo, ao mesmo tempo, não podemos exigir uma perfeição em todos os detalhes e etapas na construção de um software: do contrário, nunca concluiremos o projeto e corremos o risco de trazer prejuízo financeiro à empresa.

INTRODUÇÃO À QUALIDADE DE SOFTWARE

Em todas as áreas do consumo de produtos e/ou serviços, a qualidade é um fator primordial, pois, por meio dela, comparamos uma marca com a outra, optamos por adquirir um produto e não comprar outro, recomendamos uma marca e outra não.

Devemos sempre ter em mente que a qualidade é uma característica um tanto subjetiva. O que representa boa qualidade para alguns pode não ser o mesmo para outros. Por isso, torna-se fundamental estabelecer o que caracteriza o programa em desenvolvimento como um software de qualidade, já que essas expectativas podem variar de acordo com o contexto.

Critérios de qualidade incluem, por exemplo, a capacidade de suportar 10.000 usuários simultâneos sem perda de desempenho, a manutenção da integridade dos dados em um banco de dados relacional com movimentações intensas ou, ainda, uma interface gráfica intuitiva e de fácil acesso. Listar esses pontos de forma objetiva é essencial para saber o que se deseja.

Ter um nível de qualidade na execução dos processos é importante. Ter um nível de aceitação na performance ou em outras características do software é igualmente importante. Todavia, considerar o usuário e as respectivas necessidades é crucial para se atingir a plena qualidade. De acordo com Robert Glass (1998, p. 103, tradução nossa), “a qualidade de um produto é função do quanto ele transforma o mundo para melhor”, ou seja, uma visão que foca no benefício do produto de software.

Para melhor visualizar, Glass (1998) define a seguinte fórmula:

Satisfação do usuário = produto compatível + boa qualidade + entrega dentro do orçamento e do prazo previsto

Assim, podemos entender que a qualidade identificada para cada usuário é o fator a ser perseguido pela equipe de desenvolvimento de software.



PENSANDO JUNTOS

Seja um parâmetro de qualidade. Algumas pessoas não estão acostumadas com um ambiente onde se espera excelência.

(Steve Jobs)

DIMENSÕES DA QUALIDADE

Quando consideramos a aplicação do controle de qualidade em um software, devemos considerar vários aspectos que, embora sejam importantes individualmente, precisam ser considerados em conjunto para compor a percepção de qualidade do software como um todo.

Garvin (1987, p. 101 apud Pressman; Maxim, 2016, p. 415) considera oito dimensões da qualidade. São elas:

1. Qualidade do Desempenho.
2. Qualidade dos Recursos.
3. Confiabilidade.
4. Conformidade.
5. Durabilidade.
6. Facilidade de Manutenção.
7. Estética.
8. Percepção.

As dimensões de Garvin não foram criadas especificamente para o desenvolvimento de software, porém elas podem ser consideradas para a aplicação nessa área, assim como é descrito a seguir:

QUALIDADE DO DESEMPENHO

Devemos verificar se o desempenho aplicado está dentro dos parâmetros solicitados e definidos para o projeto, fornecendo todos os requisitos solicitados.

QUALIDADE DOS RECURSOS

Neste momento, devemos considerar a utilização eficiente dos recursos, como CPU, memória e outros componentes. Com isso, evitam-se desperdícios. Um ponto muito importante a se considerar é a percepção do usuário, isto é, se ele é surpreendido pelos recursos utilizados pela primeira vez.

CONFIABILIDADE

Neste fator, devemos considerar pontos mais técnicos em relação ao software, para garantirmos a condição do software em operar em situações normais ou anormais com questões a respeito da robustez do software, ou seja, verificando se ele é um software confiável a ser utilizado.

CONFORMIDADE

Pontos, como normas a serem seguidas, devem ser considerados. Um exemplo é uma norma ISO a que a empresa esteja submetida ou normas preconizadas pela LGPD.

DURABILIDADE

Capacidade do sistema de ser considerado útil e eficaz ao longo do tempo mediante versões e modificações, mantendo o software estável sem a degradação ou o aumento do número de bugs.

FACILIDADE DE MANUTENÇÃO

Neste ponto, devemos focar na manutenção do software, ou seja, se ele é fácil de se manter ao longo do tempo, considerando se, após o lançamento, esse software terá como aplicar pacotes de manutenção de forma fácil e bem estruturada.

ESTÉTICA

A estética de um software é essencial para atrair o usuário e ser considerado um software de qualidade. Aqui, entram áreas, como *User Experience* (UX) e *User Interface* (UI). Essa qualidade é avaliada mediante testes com o usuário final e feedback contínuo.

PERCEPÇÃO

É a percepção que temos em relação a um software desenvolvido. Por exemplo, se, no passado, um software de uma empresa foi lançado e trouxe problemas, bugs, ou seja, não era um software confiável, a nossa percepção para outro lançamento da mesma empresa pode ser influenciada e permanecemos com a rejeição em nossa mente. Portanto, este é um ponto importante a se considerar em relação à qualidade.

Essas dimensões fornecem um embasamento para tratar da qualidade de software como um todo em diversos momentos do ciclo de desenvolvimento de software, seja no levantamento das informações junto ao usuário, seja na codificação, seja na execução dos testes com ferramentas adequadas.



PENSANDO JUNTOS

As pessoas esquecem quão rápido um trabalho foi realizado — mas elas sempre lembram quão bem ele foi realizado.

(Howard Newton)



TESTE DE SOFTWARE

O teste de software é uma fase crucial no ciclo do desenvolvimento. Nela, detectamos os diversos *bugs* que podem ser identificados antes que esse software seja disponibilizado em produção. No entanto, é válido ressaltar que os testes constituem uma tarefa de detecção, e não de prevenção, ou seja, os *bugs* já estão no código: eles apenas são encontrados e reduzidos. Contudo, não podemos descartar que, embora os testes sejam uma ferramenta de detecção de *bugs*, eles contribuem para a melhoria contínua e a prevenção de novos *bugs*, pois direcionam o desenvolvedor a criar um software mais confiável.

O teste de software é uma fase crucial no ciclo do desenvolvimento. Nela, detectamos os diversos bugs que podem ser identificados antes que esse software seja disponibilizado em produção.

Conceitos básicos

Os testes de software têm alguns elementos que compõem a fundação e são necessários para o entendimento. São eles:

Defeito: também conhecido como *bug*, é uma falha ou erro no código-fonte do software que gera um comportamento errado no código, podendo ser exemplificado como a instrução ou o comando incorreto dentro de um código. É importante destacar que o defeito pode, ou não, manifestar-se na execução do software, não sendo visível para o usuário até ser executado.

Erro: é a manifestação do defeito em um software desenvolvido. Quando o software é executado, o erro pode gerar uma diferença entre o valor obtido e o valor esperado durante a execução. Um erro pode causar um comportamento inesperado, mas pode não ser imediatamente percebido.

Falha: é quando o software se comporta de forma diferente da esperada pelo usuário. A falha pode ter sido ocasionada por diversos erros que estão presentes no código. Quando um erro é encontrado durante a operação do software em ambiente de produção, ele se manifesta como uma falha, enquanto um erro que pode, ou não, manifestar-se durante a execução a falha representa a manifestação visível do erro, ou seja, um erro pode, ou não, resultar em uma falha, e uma falha é sempre um erro.

Casos de teste: “descreve uma condição particular a ser testada e é composto por valores de entrada, restrições para a sua execução e um resultado ou comportamento esperado” (Craig; Jaskiel, 2002 apud Dias Neto, 2007, p. 22).

É importante lembrar, estudante, que o teste serve como uma ferramenta de detecção. Em outras palavras, o software já foi desenvolvido e os *bugs* estão presentes; os testes apenas tornam esses problemas visíveis para que possam ser corrigidos. Portanto, a melhor abordagem é prevenir o surgimento desses erros. Isso pode ser alcançado por meio de uma documentação mais detalhada e inspeções regulares de código.

Com a explicação dos conceitos básicos mencionados, você estará mais preparada para avançar e entender melhor os testes que seguem.

Níveis de teste de software

Desde o início de um projeto, começamos a estruturar o planejamento. Entre as diversas áreas a serem consideradas, o gerenciamento da qualidade desempenha um papel essencial.

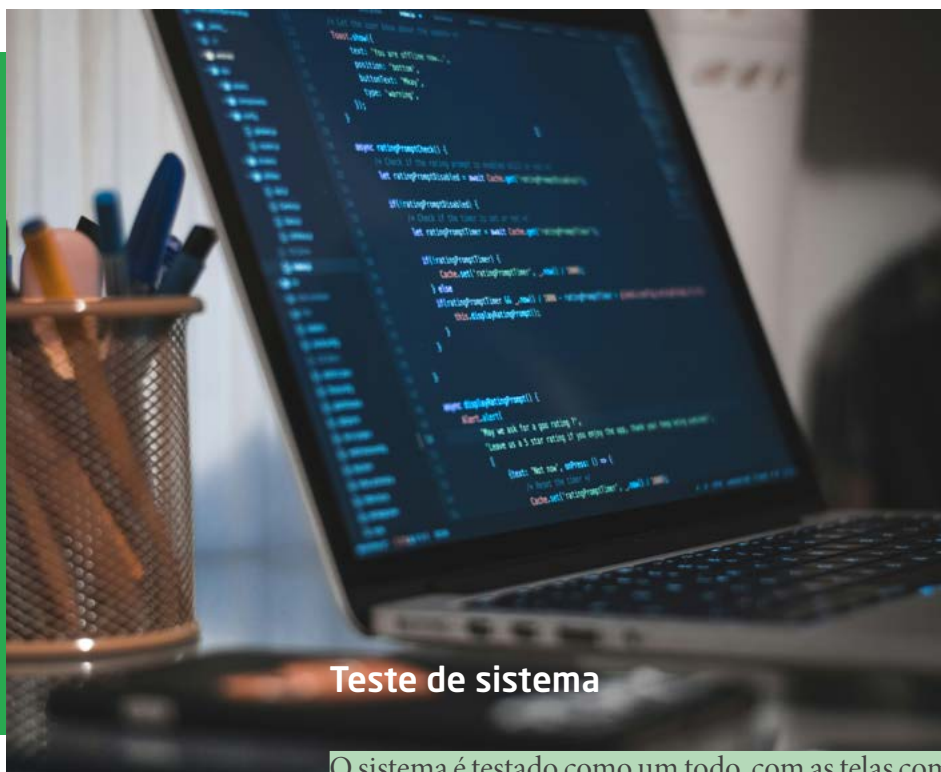
Esse aspecto abrange desde a definição de padrões até a execução de testes, que garantem que o produto atenda aos requisitos e às expectativas estabelecidos. Nos testes de qualidade, existem diferentes tipos, cada um com funções específicas para avaliar aspectos variados do sistema. A seguir, destacamos alguns dos principais tipos de testes aplicados ao longo do ciclo de desenvolvimento para assegurar a entrega de um software confiável e funcional.

Testes unitários (ou testes de unidade)

São os testes do desenvolvedor realizados na menor unidade de programação. Eles podem ser realizados em funções, métodos ou classes, com o objetivo de encontrar chamadas erradas ou comportamentos indesejados. Eles são geralmente automatizados e executados durante o desenvolvimento.

Teste de integração

Enquanto testes unitários são realizados focados em uma unidade de desenvolvimento, os testes integrados são focados na interação entre diferentes módulos e componentes do sistema. Por exemplo, a interação entre sistemas internos e externos mediante APIs (Application Programming Interface) não envolve necessariamente as GUI (*Graphic User Interface*).



Teste de sistema

O sistema é testado como um todo, com as telas completas, relatórios e outros componentes de interface.

Teste de aceitação

É guiado pelas solicitações do usuário nos documentos de especificação, ou seja, nesse

momento, será confrontado o que foi feito com o que foi solicitado pelo cliente. O objetivo é garantir que todas as necessidades do cliente sejam atendidas e que o sistema compre o propósito para o qual foi projetado.

Teste de regressão

Um tipo de teste realizado visando garantir que novas funcionalidades não insiram *bugs* e que o sistema como um todo continue operando corretamente após essas adequações serem inseridas.

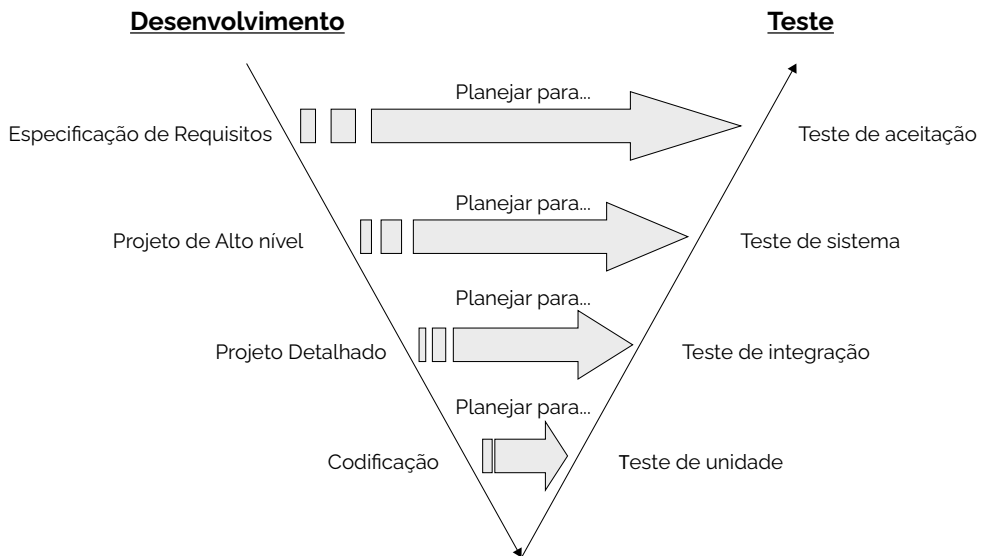


Figura 1 – Modelo V descrevendo o paralelismo entre as atividades de desenvolvimento e o teste de software / Fonte: Dias Neto (2007, p. 56).

Descrição da Imagem: é exibido um modelo em formato de "V". O lado esquerdo do "V" exibe as etapas de desenvolvimento, sendo: "Especificação de Requisitos", "Projeto de Alto Nível", "Projeto Detalhado" e "Codificação". No lado direito do "V", constam as etapas de testes, sendo elas: "Teste de Aceitação", "Teste de Sistema", "Teste de Integração" e "Teste de Unidade". Fim da descrição.

Percebemos que o desenvolvimento ocorre em paralelo com os eventos de testes, o que faz com que o software evolua mediante as próprias fases, porém sendo testado.

EVOLUÇÃO DE SOFTWARE

O software tem uma linha de evolução muito grande, desde as aplicações em cartões perfurados até as aplicações mobile ou que rodam em *cloud*. A seguir, são exibidos os principais marcos no desenvolvimento do software:

DÉCADA DE 1950 - LINGUAGENS DE ALTO NÍVEL

Nesta década, surgiram as primeiras linguagens de alto nível, como Fortran e Cobol. Isso foi um grande avanço, pois permitiu que programadores desenvolvessem em linguagem próxima da linguagem humana. Contudo, a manutenção e o desenvolvimento nessas linguagens eram operações extremamente complexas, podendo ser demoradas. Até a atualidade, encontramos programas desenvolvidos na linguagem COBOL nos bancos.

DÉCADA DE 1960 - SISTEMAS OPERACIONAIS

Sistemas como Unix proporcionaram uma base sólida para a execução de software, sendo um sistema sólido e que oferecia um sistema multitarefa e multiusuário. A manutenção era feita diretamente por técnicos especialistas, o que envolvia pouca automação. O Unix possui diversas variantes rodando atualmente, sendo o Linux o maior representante.

DÉCADA DE 1970 - METODOLOGIAS DE DESENVOLVIMENTO

Surgimento de metodologias, como cascata, que organizaram os processos em fases sequenciais. Essa metodologia, chamada de preditiva, ainda tem vasta utilização na atualidade, sobretudo em projetos estáveis e com escopo bem definido.

DÉCADA DE 1980 - PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)

Popularizou-se na década de 1980 com linguagens, como C++ e Smalltalk, trazendo uma nova forma de pensar o desenvolvimento de software, pois considerava objetos do mundo real como representações no software. Até hoje, as bases de POO são utilizadas amplamente.

DÉCADA DE 1990 - DESENVOLVIMENTO ÁGIL

Scrum e outras metodologias ágeis demonstraram ser muito eficientes para projetos de desenvolvimento de software, sendo flexíveis e apresentando mudanças de maneira mais ágil, adequando-se muito aos projetos dos dias atuais, pois vivemos em um mundo em constante mudança. Projetos com alta necessidade de adaptação requerem a aplicação de metodologias ágeis.

DÉCADA DE 2000 - OPEN SOURCE E COLABORAÇÃO

Movimentos de software livre, como Apache ou Linux, mostraram poder de colaboração mundial. Com a popularização da web e do acesso à internet, o suporte de uma comunidade global ganhou força. Logo, ferramentas de código aberto são primordiais ao desenvolvimento.

DÉCADA DE 2010 - DEVOPS E INTEGRAÇÃO CONTÍNUA

A aproximação das áreas de desenvolvimento e da área de operações se mostrou muito eficiente no cenário de desenvolvimento de software. As ferramentas de integração contínua e entrega contínua melhoraram muito a qualidade no desenvolvimento de software, consistindo em um modelo amplamente utilizado por empresas que buscam agilidade e controle de qualidade.

DÉCADA DE 2020 - INTELIGÊNCIA ARTIFICIAL (IA) E AUTOMAÇÃO

O uso da IA no desenvolvimento vem revolucionando a forma como o software é projetado, permitindo uma maior personalização. Hoje, a IA se integra com várias aplicações, otimizando a performance e auxiliando em nossa produtividade.

Atualmente, as tecnologias de cada década coexistem. Sistemas legados e sistemas novos trocam informações entre si. Por exemplo, é comum termos um sistema em Cobol e um sistema em Java compartilhando os mesmos dados, ou, na área da gestão de projetos, um sistema gerenciado parte em cascata e parte com métodos ágeis dando origem a um método híbrido.



INDICAÇÃO DE FILME

Hackers.

Um adolescente conhecido como Zero Cool é uma lenda entre os *hackers*, pois, com apenas 11 anos, ele inutilizou 1507 computadores em Wall Street, provocando um caos total no mundo das finanças.

Esse filme aborda temas, como desenvolvimento de software, vulnerabilidades e gestão de crises em ambientes tecnológicos, trazendo um foco para a importância da segurança em sistemas.



Pudemos perceber que o desenvolvimento de software evoluiu muito com o passar das décadas, seja trazendo uma nova abordagem tecnológica, seja com novas metodologias.

GERÊNCIA DA CONFIGURAÇÃO DE SOFTWARE

Durante o ciclo de um projeto de desenvolvimento de software, inúmeros softwares são interligados, versões de desenvolvimento são criadas, conexões são administradas, enfim, há vários itens de software, de forma que uma disciplina, para administrar todas essas versões e softwares diferentes, faz-se necessária. É nesse contexto que mencionamos a **gestão de configuração de software**:



Durante a criação de software, mudanças acontecem. E, por isso, precisamos gerenciá-las eficazmente. A gestão de configuração de software (SCM), também chamada de gestão de alterações, é um conjunto de atividades destinadas a gerenciar as alterações, identificando os artefatos que precisam ser alterados, estabelecendo relações entre eles, definindo mecanismos para gerenciar diferentes versões desses artefatos, controlando as alterações impostas e auditando e relatando as alterações feitas (Pressman; Maxim, 2016, p. 623).

Segundo Pressman e Maxim (2016), há quatro fontes fundamentais das alterações solicitadas para o software. São elas:

- Novos negócios: novos negócios que surgem ditam mudanças em requisitos de desenvolvimento de software.
- Novas necessidades: os envolvidos podem ter novas necessidades de mudanças e demandarem alterações.
- Reorganização ou crescimento: ao reorganizar uma empresa, mudanças estruturais no software irão ocorrer.
- Restrições orçamentárias: podem levar à redefinição do produto.



PENSANDO JUNTOS

Não há nada permanente, exceto as mudanças.

(Heráclito, 500 a.C.)

Ao longo do desenvolvimento de um projeto de software, inúmeros artefatos são criados, desde diagramas de planejamento até roteiros de testes.

A gerência de configuração é uma disciplina que é extremamente necessária para controlarmos o ambiente de desenvolvimento de software, pois **o fato de o sistema ser complexo não quer dizer que deva ser caótico.**

Ao longo do desenvolvimento de um projeto de software, inúmeros artefatos são criados, desde diagramas de planejamento até roteiros de testes. Todos esses ativos precisam ser gerenciados de uma forma que seja possível associar um planejamento feito a um código executado e vice-versa, definindo uma rastreabilidade entre os itens.

Cada um desses artefatos pode sofrer alterações. Logo, faz-se necessário o controle dos itens de configuração para que haja um maior controle e não exista inconsistência nos documentos ou demais itens do projeto.

Um exemplo que pode ser considerado é o desenvolvimento de um sistema de Recursos Humanos. Esse sistema é gerenciado por: um gerente de projetos, o qual cria o plano de projeto e outros artefatos;

uma equipe com cinco desenvolvedores, que geram código de programação e diagramas de planejamento; e dois analistas de testes, os quais geram vários testes automatizados. Então imagine, estudante, a quantidade de documentos que serão gerados e precisam ser gerenciados.

Diante dessa necessidade, a empresa responsável contrata um gerente de configuração, que se torna responsável pela política de armazenamento de arquivos e procedimentos relacionados. Essa disciplina é comumente chamada de CM (*Configuration Management*). As principais etapas incluem:

- **Planejamento do gerenciamento da configuração:** plano que define os objetos que serão gerenciados e como (por exemplo, descrevendo ferramentas).
- **Identificação:** identifica todos os itens de configuração que compõem o sistema.
- **Controle da configuração:** gerencia as alterações de cada item de configuração do projeto.
- **Registro de configuração:** etapa em que cada item registrado é rastreado e é elaborado um relatório.
- **Auditoria:** auditorias são realizadas para garantir a conformidade com os requisitos e plano de configuração.
- **Gestão de distribuição:** distribui versões aprovadas para ambientes de testes.

Segundo Pressman e Maxim (2016, p. 626), são elementos de um sistema de configuração:

- Elementos de componente – conjunto de ferramentas acopladas em um sistema de gestão de arquivos (por exemplo, um banco de dados) que possibilita acesso à gestão de cada item de configuração de software.
- Elementos de processo – coleção de procedimentos e tarefas que definem uma abordagem eficaz de gestão de alterações (e atividades relacionadas) para todas as partes envolvidas na gestão, engenharia e uso do software.
- Elementos de construção – conjunto de ferramentas que automatizam a construção do software, assegurando que tenha sido montado o conjunto apropriado de componentes validados (isto é, a versão correta).

- Elementos humanos – conjunto de ferramentas e características de processo (abrangendo outros elementos de CM) usados pela equipe de software para implementar uma SCM eficaz

A aplicação profissional dos conceitos citados envolve inúmeros desafios e afeta diretamente o seu desenvolvimento profissional. No entanto, trará um maior controle e probabilidades de sucesso.

NOVOS DESAFIOS

A conexão entre o conteúdo abordado nas páginas anteriores e a realidade profissional é fundamental para a abordagem em situações práticas de projetos reais nas empresas em que atuamos diariamente.

No ambiente profissional, o trabalho em equipe é resumido, muitas vezes, a deixar várias pessoas trabalhando em conjunto, a fim de se obter um projeto ao final. Todavia, o trabalho em times é muito mais que isso, visto que, por exemplo, envolve trabalhar a motivação, pois, ao entendermos que a produtividade de uma equipe de software está diretamente relacionada com a motivação dos membros dessa equipe, entenderemos que uma equipe é muito maior que pessoas juntas.

Outro ponto muito importante são os *softskills*, ou seja, aqueles conhecimentos que não são caracterizados como técnicos. Podemos citar a comunicação, o trabalho em equipe, as características comportamentais ou outras características que certamente farão com que o cronograma do projeto seja impulsionado e as atividades sejam entregues em um prazo menor.

Atualmente, as empresas têm recursos muito limitados. Projetos bem definidos e com um alto nível de qualidade são essenciais para evitar perdas. Portanto, no contexto prático, garantir a qualidade não se resume a não haver erros, mas a entregar um produto que atenda às expectativas e às necessidades do usuário.

O mercado valoriza muito quem possui essa visão do todo e a importância das tarefas relacionadas ao desenvolvimento, como gestão e QA (*Quality Assurance*), e de ferramentas de automação de testes, como JUnit, Selenium, TestNG. Isso, pois a automação acelera o processo de validação e auxilia nos testes, reduzindo a probabilidade de problemas.

É muito importante se especializar nessas áreas, pois, certamente, o seu currículo será visto com outros olhos.

VAMOS PRATICAR

1. Qualidade de projeto se refere às características que os projetistas especificam para um produto. A qualidade dos materiais, as tolerâncias e as especificações de desempenho são fatores que contribuem para a qualidade de um projeto.

A respeito da qualidade, assinale a alternativa correta:

- a) A qualidade é uma característica de todos os serviços e é objetiva.
 - b) A qualidade é uma característica de todos os serviços e é subjetiva.
 - c) A qualidade é uma característica de apenas alguns produtos e é objetiva.
 - d) A qualidade é uma característica de apenas alguns produtos e é subjetiva.
 - e) A qualidade é uma característica a ser desconsiderada para software.
2. As dimensões de qualidade de Garvin nos dão uma visão "indulgente" da qualidade de software. Muitas (mas não todas) dessas dimensões podem ser consideradas apenas subjetivamente.

Assinale a alternativa que contém somente as dimensões da qualidade de Garvin:

- a) Qualidade do Desempenho, Confiabilidade, Qualidade do Escopo.
- b) Estética, Percepção, Qualidade do Escopo.
- c) Conformidade, Durabilidade, Confiabilidade.
- d) Escopo, Custo, Tempo.
- e) Escopo, Conformidade, Tempo.

VAMOS PRATICAR

3. No desenvolvimento de software, a qualidade de um projeto engloba o grau de atendimento às funções e às características especificadas no modelo de requisitos.

A respeito da qualidade, considere as afirmativas a seguir:

- I - Ter um nível de qualidade na execução dos processos é importante.
- II - Ter um nível de aceitação na performance ou em outras características do software é igualmente importante.
- III - Considerar o usuário e as respectivas necessidades é crucial para se atingir a plena qualidade.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

REFERÊNCIAS

DIAS NETO, A. C. Introdução a teste de software. **Engenharia de Software Magazine**, v. 1, p. 22, 2007.

GLASS, R. Defining Quality Intuitively. **IEEE Software**, n. 107, p. 103-104, 1998.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8 ed. [S. l.]: McGraw Hill, 2016.

CONFIRA SUAS RESPOSTAS

1. Alternativa B.

A qualidade em serviços pode ser tanto objetiva quanto subjetiva, pois envolve percepções e expectativas do cliente, o que torna difícil defini-la de maneira puramente objetiva. No entanto, diante disso, pode-se afirmar que a qualidade em serviços é frequentemente subjetiva, pois está relacionada às expectativas e às percepções dos usuários e clientes. O que pode ser considerado de alta qualidade para um cliente pode não ser para outro, tornando-a mais difícil de padronizar. Além disso, qualidade é uma característica relevante para todos os produtos e serviços, e pode ser avaliada tanto de maneira objetiva (por padrões técnicos) quanto subjetiva (pelas percepções dos usuários). Ainda, a qualidade é fundamental no desenvolvimento de software, pois garante que o produto atenda aos requisitos e expectativas dos usuários, além de ser confiável e eficiente.

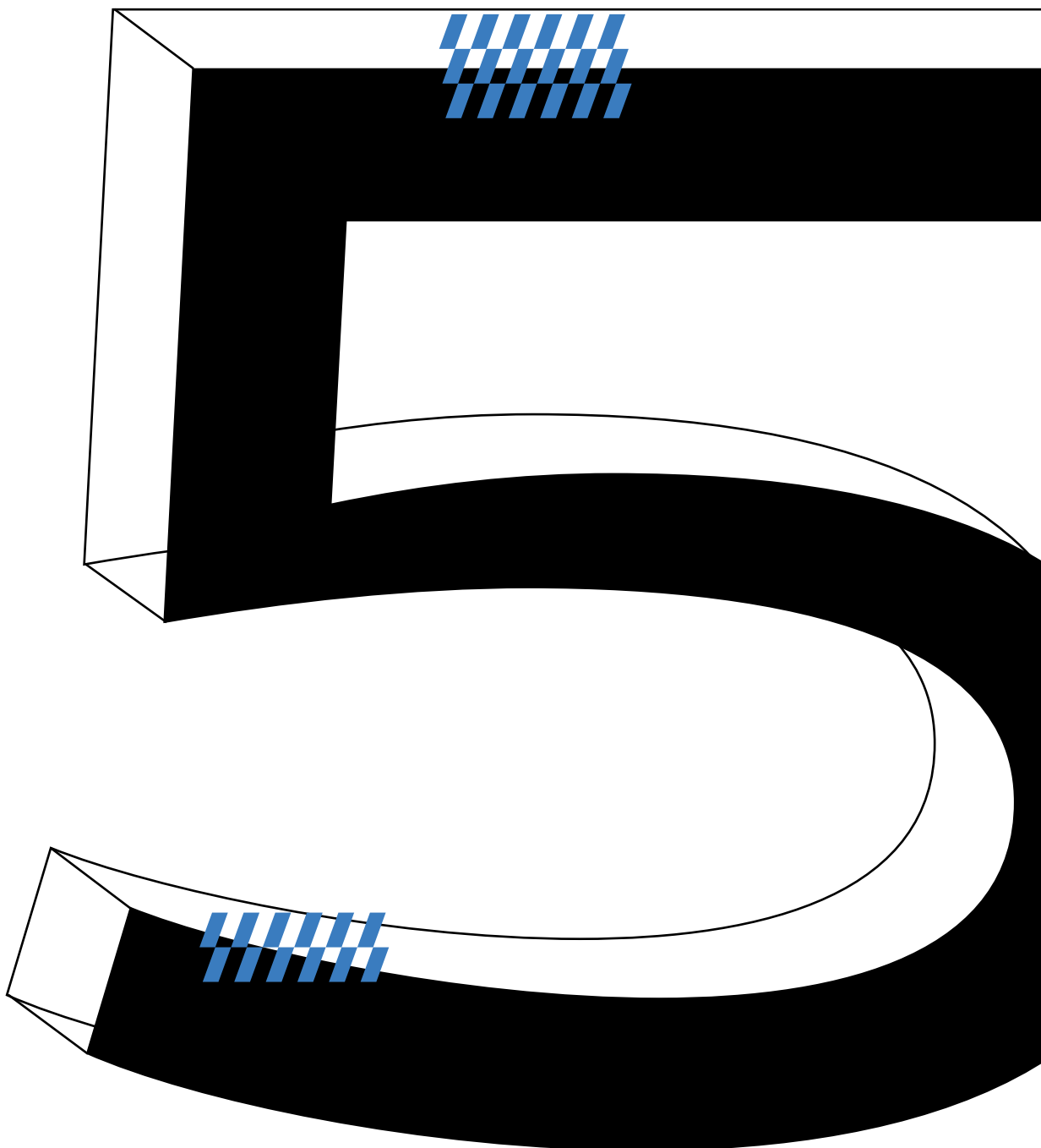
2. Alternativa C.

Desempenho e confiabilidade são dimensões da qualidade de Garvin, mas "Qualidade do Escopo" não é. Escopo é uma dimensão de gerenciamento de projetos, não da qualidade. Estética e percepção são dimensões da qualidade segundo Garvin, mas "Qualidade do Escopo" não pertence a essa classificação. O escopo está relacionado ao gerenciamento de projetos. Estas são dimensões clássicas da qualidade propostas por Garvin: Conformidade, Durabilidade e Confiabilidade. Elas tratam da aderência aos requisitos, vida útil do produto e sua confiabilidade no desempenho. Essas são variáveis do gerenciamento de projetos, e não dimensões da qualidade segundo Garvin. Elas tratam de gerenciamento de projetos, não de atributos de qualidade. Conformidade é uma dimensão da qualidade de Garvin, mas "Escopo" e "Tempo" estão relacionados ao gerenciamento de projetos, não à qualidade do produto.

3. Alternativa E.

A execução dos processos de forma consistente e controlada garante a entrega de um produto ou serviço de alta qualidade. Além dos processos, o produto também precisa atender a requisitos de performance, usabilidade, segurança e outras características que impactam diretamente a experiência do usuário e a eficácia do software em seu propósito. A qualidade de um software não é somente técnica; é também a capacidade de atender às necessidades e às expectativas do usuário. Compreender essas necessidades é fundamental para criar um produto que seja útil, intuitivo e funcional.

 *unidade*





TEMA DE APRENDIZAGEM 8

METODOLOGIAS ÁGEIS

MINHAS METAS

- Compreender as metodologias ágeis.
- Entender os conceitos necessários para a aplicação na prática.
- Estudar o Manifesto Ágil e os respectivos princípios.
- Explorar o Scrum.
- Compreender o *Extreme Programming* – XP.
- Entender a necessidade da colaboração em equipe.
- Estudar as principais diferenças em relação ao método tradicional.

INICIE SUA JORNADA

Atualmente, no mundo, ocorrem mudanças o tempo todo. Um projeto que, hoje, consegue suprir as necessidades da empresa, amanhã, pode não conseguir devido a novas necessidades do negócio, ou seja, o projeto deverá sofrer alterações para se adequar à nova realidade. Isso se torna um problema, caso o projeto não seja organizado e flexível às mudanças.

Metodologias ágeis, como Scrum e XP (*Extreme Programming*), oferecem uma estrutura para a condução dessas mudanças de forma adequada, conduzindo-as de forma ágil, com alto envolvimento do cliente. Para você, estudante, compreender esse cenário não se trata apenas de uma questão acadêmica, mas é necessária para a atuação em ambientes profissionais que exigem flexibilidade e inovação.

Suponha que você está em uma reunião diária de projeto e, ao término de uma sprint de projeto, recebe a notícia do cliente de que as maiores funcionalidades não serão mais utilizadas devido a uma alteração de legislação. Logo, será necessário adequar essas funcionalidades para um novo funcionamento.

Refleta a respeito dessa situação e do modo como essa mudança seria tratada em sua equipe, visto que a modificação alterará os requisitos do projeto. Com isso, o custo e o tempo serão afetados, pois o escopo foi alterado.



PLAY NO CONHECIMENTO

Neste podcast, abordaremos o tema "*A cultura ágil: como criar um ambiente favorável*". Ouça para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal lembrar o que são as metodologias ágeis? É interessante lembrar, pois são etapas e papéis comuns a todos os projetos. O vídeo a seguir demonstra o método ágil.
<https://youtu.be/CED2y4YSVUK>

DESENVOLVA SEU POTENCIAL

As metodologias ágeis ganharam muita notoriedade nos últimos anos, sobretudo, pelo fato de lidarem muito bem com as mudanças. Isso faz com que elas ocorram de forma rápida, considerando as necessidades do cliente, tendo em vista que um software que funcione e seja adequado às necessidades do cliente é muito mais importante do que um plano de projeto no papel.

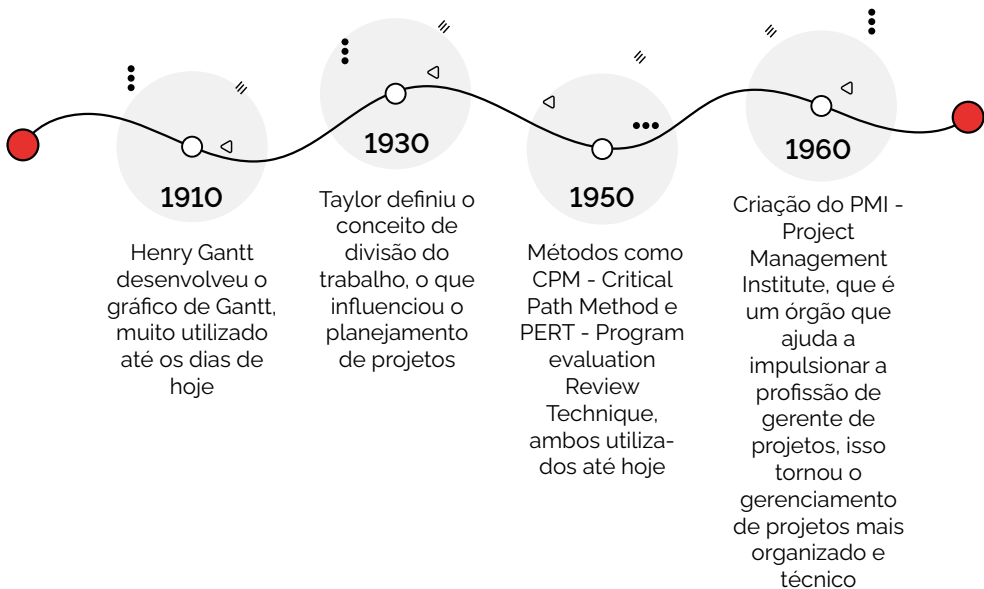
Um projeto desenvolvido com metodologias ágeis pode oferecer uma flexibilidade muito maior. No entanto, a empresa precisa adaptar o próprio *mindset*, pois o desenvolvimento ágil não considera algumas práticas tradicionais enraizadas na cultura corporativa, como cronogramas com prazos fixos. Ao contrário disso, as metodologias ágeis utilizam conceitos de *sprints*, que têm uma duração definida e são realizadas até que as funcionalidades planejadas sejam concluídas. Contudo, o escopo dessas *sprints* pode ser ampliado ou detalhado, o que pode impactar a data de entrega final.

No entanto, a empresa precisa adaptar o próprio mindset, pois o desenvolvimento ágil não considera algumas práticas tradicionais enraizadas na cultura corporativa

O GERENCIAMENTO DE PROJETOS

O gerenciamento de projetos tem origens nos primórdios da humanidade mediante as obras realizadas ao longo do tempo, sejam as pirâmides do Egito, sejam outras obras mundo afora. Isso, pois, de uma forma ou de outra, essas obras ou “projetos” deveriam ser “gerenciados”. Isso poderia ser feito sem métodos, mas tinha certa gestão.

Quando falamos do gerenciamento de projetos moderno, ele se estruturou a partir do século XX, principalmente em indústrias, como engenharia, construção e defesa. A seguir, você pode verificar uma breve linha do tempo:



Nos anos de 1970 e 1980, houve a consolidação profissional, com o surgimento do método *waterfall*, em que uma fase era finalizada para outra ser iniciada. Esse método é muito utilizado até os dias de hoje.

Então, nos anos de 1990, surgiram os métodos ágeis, pois, com o aumento da complexidade dos projetos, essa abordagem era necessária.

INTRODUÇÃO ÀS METODOLOGIAS ÁGEIS

As metodologias ágeis partem do princípio de que as mudanças ocorrerão sempre e os projetos devem se adequar a essas modificações, buscando satisfazer às necessidades dos usuários, tendo-os como centro do desenvolvimento. As metodologias ágeis também usam uma estratégia, que é **trazer o cliente para o lado da equipe do projeto**, mostrando todos os passos, sucessos e dores, para que ele compartilhe os desafios da equipe e a comunicação ocorra de forma mais transparente.

Existe uma metáfora amplamente difundida no ensino de metodologias ágeis que ajuda a ilustrar a questão de trazer o cliente junto à equipe:

Suponha que o seu gerente te pediu para levar uma encomenda ao aeroporto para colocá-la em um avião, pois ela deveria estar no estado vizinho na outra

manhã. Então, você prontamente fala: “Deixa comigo, chefe, vou lá!”. Assim, você entra no carro e vai correndo até o aeroporto. Contudo, já são 17:00 e o pneu fura. Você troca o pneu com rapidez, mas já perdeu 20 minutos. Depois disso, entra na via rápida que irá te levar ao aeroporto, mas houve um acidente na pista e você não sabia. A via está com um engarrafamento de vários quilômetros e você não sai do lugar. Finalmente, você chega à via de acesso ao aeroporto e pensa: “Vai dar tempo”. Já são 18:50 e você não consegue um lugar para estacionar o carro. Então, pensa: “Vou parar na frente da porta e vou rapidinho despachar a encomenda”. Entretanto, você parou em fila dupla e pediram para você tirar o carro dali. Você leva o carro até o estacionamento, localizado na outra quadra. Você o estacionou e retornou para despachar a encomenda. No entanto, chegou às 19:10, e o avião partia às 19:00.



INDICAÇÃO DE FILME

Silicon Valley.

Esta série de comédia da HBO retrata a vida de uma startup de tecnologia e reflete o espírito das metodologias ágeis. As situações de trabalho em equipe, o desenvolvimento de produtos dentro de prazos e os ciclos rápidos se assemelham à realidade do Scrum.



Agora, imagine um cenário em que você fala: “Ok, chefe, eu dirijo, mas vamos juntos”. Você passa pelo mesmíssimo cenário, com todas as dificuldades, lembrando que o seu chefe está junto. Então, vocês finalmente chegam às 19:10 ao balcão para despachar a encomenda, e a atendente avisa que o avião partiu às 19:00. O chefe perceberá que foi feito o possível para chegar no horário. Ele conseguiu ver todas as dificuldades e acompanhou a trajetória. Logo, ele irá compartilhar a sua frustração de não ter chegado na hora e entenderá toda a situação.

Essa metáfora ilustra muito bem a importância de envolver o cliente em todo o processo, demonstrando dois dos pilares das metodologias ágeis, ou seja, a transparência e o compartilhamento com o cliente.

MANIFESTO ÁGIL E OS DOZE PRINCÍPIOS

Os métodos tradicionais de desenvolvimento de software têm raízes nos primórdios da engenharia de software, remontando aos anos de 1970, quando eles eram chamados de “*waterfall*” ou “cascata”. A rigidez dos métodos tradicionais levou a um certo questionamento por parte dos desenvolvedores, que apresentaram descontentamento com a forma de trabalhar utilizando os métodos tradicionais, pois eles estavam um pouco distantes da realidade do mundo moderno do desenvolvimento de software, que tinha necessidades de mudança muito mais imediatas.

Insatisfações, tais como rigidez das metodologias, complexidade em gerir mudanças e dificuldades na colaboração, culminaram na criação do **Manifesto Ágil**, em 2001, criado por um grupo de 17 profissionais de software reunidos em uma estação de esqui em Snowbird, UTAH, nos Estados Unidos. Essa reunião contou com a participação de vários desenvolvedores seniores do movimento *Extreme Programming* (XP).

**EU INDICO**

O Manifesto Ágil é uma leitura interessante para aprofundar os seus conhecimentos. Acesse: <http://agilemanifesto.org/iso/ptbr/manifesto.html>

Os resultados dessa reunião foram o *Manifesto Ágil* e os 12 princípios que se tornaram as bases das metodologias ágeis. Eles focaram principalmente na colaboração, na adaptabilidade e na entrega contínua do valor ao cliente, pois essas eram limitações percebidas pelas metodologias tradicionais.

Os 12 princípios das metodologias ágeis são (Agile Alliance, 2024)

- **Satisfação do cliente:** mediante a entrega contínua e adiantada de software.
- **Mudanças bem-vindas:** mesmo que ocorram tarde, essas mudanças são bem-vindas. Equipes ágeis as usam de maneira estratégica, visando à vantagem competitiva dos clientes.
- **Entregas frequentes:** entregar um software funcionando em uma pequena escala de tempo.
- **Colaboração:** a equipe deve trabalhar diariamente em conjunto por todo o projeto.
- **Motivação das pessoas:** construir o projeto com indivíduos motivados.
- **Comunicação face a face:** preferir a comunicação face a face (presencial).
- **Software funcionando:** é a partir do software funcionando que se visualiza o progresso.
- **Sustentabilidade:** o ritmo deve ser constante com patrocinadores, desenvolvedores e usuários.
- **Excelência técnica e bom design:** a contínua atenção a esses detalhes aumenta a agilidade no projeto.
- **Simplicidade:** é preciso focar nos objetivos do projeto, com ênfase naquilo que realmente é necessário.
- **Auto-organização:** a equipe deve ter autonomia para decidir como realizar o trabalho, distribuir atividades ou resolver problemas.
- **Reflexão e ajuste:** deve-se parar continuamente para avaliar o projeto e aplicar eventuais correções.



O movimento Agile não é anti-metodologia, de fato, muitos de nós querem restaurar a credibilidade da palavra metodologia. Queremos restaurar um equilíbrio. Adotamos a modelagem, mas não para registrar algum diagrama em um repositório corporativo empoeirado. Adotamos a documentação, mas não centenas de páginas de volumes nunca mantidos e raramente usados. Planejamos, mas reconhecemos os limites do planejamento em um ambiente turbulento (Oliveira, 2020).

Kent Beck, um dos signatários do Manifesto Ágil certa vez, estimou em seis semanas o tempo para a realização de uma atividade para dois desenvolvedores da própria equipe. Todavia, o gerente trocou um dos desenvolvedores e o trabalho levou doze semanas. O chefe culpou Kent, levando-o a se sentir culpado por várias semanas, mas Kent percebeu que o “culpado”, na verdade, foi o gerente, ou melhor, a metodologia, que era processual, com a mentalidade de processo fixo.

Então, estudante, esse é um exemplo prático de como a metodologia pode alterar um projeto. Estamos falando de uma única tarefa pequena, agora, imagine o cenário que se formaria com centenas de tarefas em conjunto.

Com relação às metodologias ágeis, atualmente, existem diversas opções no mercado, cada uma com abordagens específicas. Podemos citar, por exemplo, o Scrumban, que combina práticas do Scrum e Kanban. O Kanban, por sua vez, é um quadro visual amplamente utilizado para monitorar projetos e é bastante empregado no Scrum. Neste material, focaremos no Scrum por ser uma das metodologias mais utilizadas no mundo (observe o gráfico a seguir), e na metodologia XP, que é uma das metodologias mais antigas e cujos valores influenciaram a criação do Scrum.

Em relação às metodologias ágeis, atualmente, existem diversas opções no mercado, cada uma com abordagens específicas. Podemos citar, por exemplo, o Scrumban, que combina práticas do SCRUM e Kanban

METODOLOGIAS ÁGEIS: SCRUM

Quando falamos de metodologias ágeis, logo, pensamos na metodologia Scrum, pois, segundo dados do *15th State of Agile Report*, ela é a mais utilizada no mundo.



EU INDICO

O relatório *15th State of Agile Report*, de 2021, pode ser acessado no QR Code a seguir. Ele mostra diversos números interessantes a respeito dos métodos ágeis. O documento está em inglês.

<https://digital.ai/resource-center/analyst-reports/state-of-agile-report>

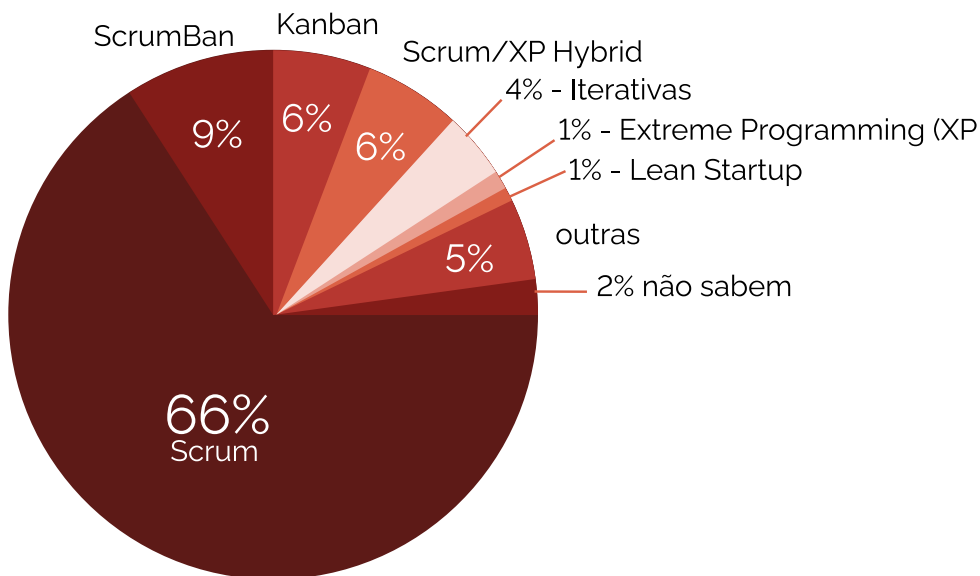


Figura 1 - Metodologias ágeis mais usadas / Fonte: Digital.AI [...] (2021, p. 13).

Descrição da Imagem: trata-se de um gráfico de pizza em tons de vermelho. Ele está segmentado em nove partes da seguinte maneira: 66% usam Scrum, 9% usam Scrumban, 6% usam Kanban, 6% utilizam uma combinação de Scrum/XP Hybrid, 5% optam por uma abordagem Iterative, 4% seguem *Extreme Programming* (XP) e 1% adota Lean Startup. Outros métodos ocupam uma fatia rotulada como Other, sem especificar o percentual. 2% dos respondentes indicam que não sabem ou não especificam o método usado. Há um círculo separado por segmentos e um texto com a porcentagem de uso das metodologias ágeis, que são: 66% - Scrum, 9% - ScrumBan, 6% Kanban, 6% - Scrum/XP Hybrid, 4% - Iterativas, 1% - Extreme Programming (XP), 1% - Lean Startup, 5% outras, 2% não sabem. Fim da descrição.

O Scrum parte do princípio de que temos uma lista de histórias do usuário (atividades). Essas histórias são priorizadas de acordo com o desejo do cliente. Assim, a equipe Scrum escolhe algumas funcionalidades para que possam ser executadas em um intervalo de tempo predeterminado, geralmente, de 2 a 4 semanas. A esse intervalo, dá-se o nome de sprint.

Durante a sprint ocorrem alguns eventos denominados cerimônias Scrum, que ocorrem para o controle do projeto. Esses eventos são: planejamento da sprint reunião diária, retrospectiva da sprint e revisão da sprint

Ao término da sprint um conjunto de funcionalidades é pronto, para que possa ser testado e incorporado ao software que está rodando no ambiente do cliente. Assim ocorre até serem finalizadas todas as funcionalidades, que podem ser correções de *bugs*, histórias do usuário ou outras funcionalidades dos usuários no *backlog* do produto.

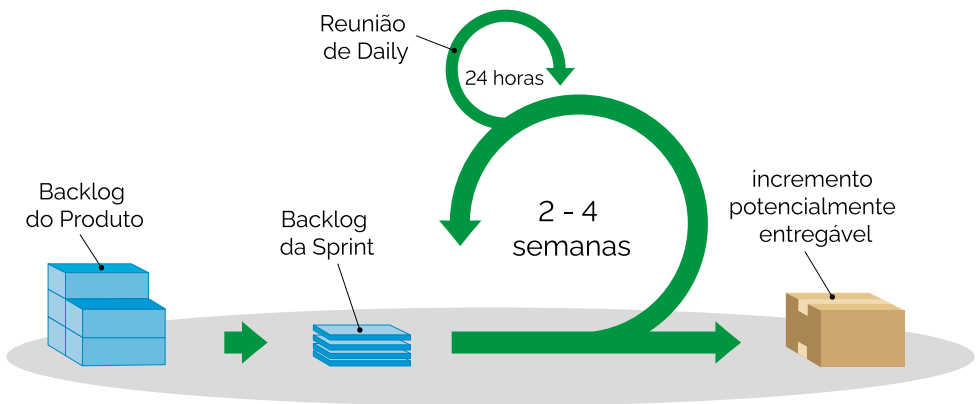


Figura 2 - Representação gráfica do framework Scrum

Fonte: [https://commons.wikimedia.org/wiki/File:Scrum_diagram_\(labelled\).png](https://commons.wikimedia.org/wiki/File:Scrum_diagram_(labelled).png). Acesso em: 29 nov. 2024.

Descrição da Imagem: são exibidos retângulos empilhados representando o backlog do produto. Há uma seta à direita, retângulos empilhados representando o *backlog* da sprint e uma flecha em círculo no sentido anti-horário representando a execução da sprint com o texto "2 a 4 semanas". Acima desse círculo maior, há um círculo menor representando as reuniões diárias do Scrum com o texto "24 horas". À direita, encontra-se um bloco representando um produto potencialmente entregável nessa iteração. Fim da descrição.

Para compreendermos melhor, vamos identificar cada elemento do Scrum.

- **Backlog do produto:** lista priorizada de todas as funcionalidades, melhorias e correções de *bugs*, servindo como “roteiro” para a execução do projeto.
- **Backlog da sprint:** o *backlog* da Sprint é um subconjunto do *backlog* do produto, porém com o que será realizado naquela sprint
- **Reunião diária (*daily*):** reuniões rápidas (em geral, 15 minutos), em que cada um responde o que fez, os eventuais problemas e o que fará.
- **Sprint:** intervalo de tempo (2 a 4 semanas) em que os itens de *backlog* da sprint são executados, visando entregar um incremento funcional.
- **Equipe Scrum:** equipe multidisciplinar e auto-organizada composta por Product Owner, Scrum Master e Time de Desenvolvimento.
- **Product Owner (P.O.):** responsável por priorizar os itens do backlog junto ao cliente. É a voz do cliente junto à equipe, comunica a visão do produto e toma decisões sobre o que deve ser desenvolvido.
- **Scrum Master:** é o facilitador da equipe, removendo obstáculos que possam estar interferindo.
- **Cliente:** qualquer parte interessada no produto em desenvolvimento.
- **Sprint review:** reunião com duração de 1 a 4 horas realizada ao término de cada Sprint. Nela, o trabalho é revisado e o P.O. verifica as funcionalidades a serem entregues e se elas atendem aos critérios de aceitação e à expectativa do cliente.
- **Sprint retrospective:** ocorre após a sprint review. Nela, é discutido o que deu certo e errado na Sprint, para ajustar para a próxima.

Essa é uma descrição do funcionamento do Scrum e dos respectivos papéis. Entretanto, um dos maiores desafios que você poderá observar se relaciona à mudança de *mindset*, principalmente no que diz respeito à cultura da empresa, pois é preciso pensar sempre no usuário final e no fluxo de trabalho Scrum.

Imagine, estudante, iniciar a implementação de metodologias ágeis em uma empresa rigidamente hierárquica e com uma cultura que valoriza prazos bem definidos, com hierarquias claras e processos estabelecidos. Esse é um cenário desafiador, pois, assim como constatamos, o *Manifesto Ágil* prioriza entregas rápidas, ao contrário de processos rígidos, e não exige cronogramas ou orçamentos fechados.

Esses aspectos culturais podem ser obstáculos iniciais, uma vez que os métodos ágeis são flexíveis e adaptáveis a mudanças. Portanto, será necessário um trabalho de adaptação na empresa, especialmente em relação à flexibilização de hierarquias, prazos e custos.

Para aplicar métodos ágeis nesse contexto, recomenda-se a realização de *workshops* para explicar o funcionamento da abordagem ágil, além de reuniões regulares com todos os envolvidos, a fim de mostrar a evolução positiva do projeto. Um plano de comunicação bem estruturado também é essencial para alinhar expectativas e facilitar a transição.

VISUALIZAÇÃO NO QUADRO DE TRABALHO

Todo esse trabalho de execução precisa ter uma visualização adequada, de forma que fique claro tudo o que está sendo planejado e executado.



O time também possui um “quadro de trabalho” onde organiza as atividades, dos itens de backlog da sprint, separando-as em basicamente em quatro estados: **Para fazer**, **Em andamento** (inclui o nome do responsável por executar a tarefa), **Para verificar e Concluído**. Esse “quadro” é muito produtivo, pois basta olhar para ele para realizar a leitura do progresso da sprint inclusive nele pode-se indicar se existe algum impedimento para que as atividades sejam executadas (Pereira; Torreão; Marcal, 2007, p. 64, grifo nosso).



Para você visualizar melhor, estudante, imagine um projeto de desenvolvimento de software de uma loja de produtos de informática. Segundo a metodologia Scrum, estas são as etapas a serem seguidas:

- Cada cadastro, consulta, relatório solicitado pelo cliente faz parte do *backlog* do produto e deve ser priorizado pelo PO junto ao cliente.
- Na reunião de planejamento da sprint a equipe define quais funcionalidades serão definidas nessa sprint e detalha o planejamento para cada funcionalidade.
- Então, a equipe começa realizar a execução da sprint
- A cada dia, é realizada uma reunião *daily*.
- Após o término da sprint uma cerimônia de sprint review é realizada. Com isso, os itens a serem entregues ao cliente são definidos e a entrega é realizada.
- Depois disso, uma deve ser realizada.
- O fluxo é reiniciado e outra sprint é executada até que as atividades descritas no *backlog* do produto finalizem.

O conjunto de execução de todas essas atividades irá trazer uma maior flexibilidade à execução dos projetos, pois trará um acompanhamento mais próximo e sucessivas reuniões e planejamentos.

METODOLOGIAS ÁGEIS: XP

O Extreme Programming - XP é uma metodologia ágil que visa melhorar a qualidade e a capacidade de resposta às mudanças. Ele segue alguns princípios fundamentais. Com base em Soares (2004), são eles:

- **Comunicação:** o XP enfatiza a comunicação, e reuniões diárias (*standup meetings*) são essenciais.
- **Feedback:** o feedback constante é fundamental. O trabalho é revisado e ajustado constantemente.
- **Coragem:** a equipe deve ter coragem para admitir erros ou buscar melhorias no projeto.
- **Simplicidade:** sempre buscar a simplicidade na execução do projeto

- **Respeito:** todos os membros devem prezar pelo respeito e valorizar o trabalho dos colegas.
- Considerando os princípios estabelecidos, algumas práticas foram estabelecidas:
- **Planejamento:** o planejamento em XP consiste em definir o que deve ser feito com base nos requisitos atuais, evitando a dependência de requisitos futuros.
- **Entregas frequentes:** o XP prioriza entregas frequentes de software, atualizando-o conforme surgem novos requisitos. As versões devem ser pequenas e conter os requisitos de maior valor, com entregas ideais a cada mês ou a cada dois meses.
- **Metáfora:** a metáfora em XP refere-se a descrições de software usando linguagem acessível, sem jargões técnicos.
- **Projeto simples:** o XP defende que o software deve ser desenvolvido da forma mais simples possível, atendendo apenas aos requisitos atuais.
- **Testes:** no XP, a validação do software ocorre durante todo o processo de desenvolvimento, com os testes sendo desenvolvidos antes do código.
- **Refatoração:** é o processo de melhorar a estrutura do código sem alterar o comportamento externo, sendo uma prática contínua em XP.
- **Programação em pares:** na programação em pares, dois desenvolvedores trabalham juntos em um único computador, um escrevendo o código e o outro revisando. Essa prática aumenta a qualidade do código, identifica erros e facilita o aprendizado contínuo entre os desenvolvedores.
- **Propriedade coletiva:** no XP, o código pertence a todos os membros da equipe, permitindo que qualquer um faça melhorias, independentemente de quem o escreveu. Essa abordagem promove um ambiente colaborativo onde todos são responsáveis pelo software.
- **Integração contínua:** a integração contínua envolve integrar e construir o software várias vezes ao dia, mantendo todos os programadores atualizados.
- **40 horas de trabalho semanal:** o XP propõe uma carga de trabalho semanal de 40 horas, evitando horas extras constantes. Se for necessário trabalhar mais de 40 horas, isso indica que há problemas que precisam ser resolvidos com um melhor planejamento.

- **Cliente presente:** a presença do cliente é essencial em todo o processo de desenvolvimento, permitindo esclarecimentos de requisitos e evitando mal-entendidos.
- **Código padrão:** o XP promove a padronização na arquitetura do código, permitindo que ele seja compartilhado entre todos os programadores.

Benefícios XP

O XP prioriza a **alta qualidade do software** por meio de práticas, como o desenvolvimento orientado a testes TDD (*Test Driven Development*), que é uma estratégia de desenvolvimento que gira em torno dos testes. Os testes são desenvolvidos, e o desenvolvimento é feito procurando alcançar o que foi definido nos testes.

O XP também busca a melhoria contínua nas ações executadas durante o desenvolvimento. Ele preconiza, assim como o Scrum, uma **alta flexibilidade para mudanças**, uma vez que permite que o time se adapte facilmente a esse cenário, promovendo um alto engajamento do cliente.

O **trabalho em pares** é uma prática incentivada pelo XP, em que um desenvolvedor trabalha revisando o código do outro, ou seja, os dois ficam lado a lado desenvolvendo um **código simples**, reduzindo os riscos e gerando transparência para a motivação e a satisfação da equipe. Esses benefícios tornam o XP uma metodologia eficaz para projetos, em que é importante responder e focar nas mudanças.

Imagine-se desenvolvendo um projeto com seu time e participando de uma revisão por pares. Esse processo tem um impacto significativo na qualidade da entrega. No entanto, algumas empresas podem não perceber os benefícios de dois programadores trabalhando juntos em uma mesma tarefa. Para alguns gestores, a cena de duas pessoas diante de um computador pode transmitir a impressão de que uma delas não está sendo produtiva. Essa percepção, porém, ignora o fato de que o segundo programador está revisando o código ativamente, contribuindo para identificar e prevenir problemas futuros.

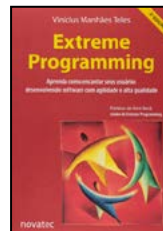


INDICAÇÃO DE LIVRO

Extreme Programming.

Deixo a indicação deste livro sobre *Extreme Programming*, que mostra essa fascinante metodologia.

Extreme Programming (XP) é um processo de desenvolvimento que possibilita a criação de software de alta qualidade de maneira ágil, econômica e flexível. Vem sendo adotado com enorme sucesso na Europa, nos Estados Unidos e, mais recentemente, no Brasil.



As metodologias ágeis acabam por transformar o desenvolvimento de projetos, fazendo com que eles respondam às mudanças com uma flexibilidade muito maior. Com isso, o projeto fica mais aderente às necessidades das pessoas e do mundo, posicionando o profissional de maneira atrativa no mercado de trabalho.

NOVOS DESAFIOS

O uso das metodologias ágeis é cada vez mais aderente ao mercado atual. Como vivemos em um mundo de constante mudança, o uso das metodologias ágeis é uma necessidade.

As empresas necessitam cada vez mais organizar os próprios processos, definindo projetos para execução. Portanto, o profissional com conhecimento das metodologias e com capacidade para conduzir um projeto é cada vez mais requisitado no mercado de trabalho.

Para se preparar cada vez mais para esses desafios, é fundamental o desenvolvimento em algumas áreas, como comunicação, *softskills* e a própria gestão de projetos como um todo.

Seguindo esses pontos para fortalecer os seus conhecimentos teóricos, certamente, sua carreira será impulsionada, de modo que seu currículo fique realmente atrativo aos olhos do mercado.

VAMOS PRATICAR

1. Embora as ideias fundamentais que norteiam o desenvolvimento ágil tenham estado conosco por muitos anos, apenas há menos de duas décadas se consolidaram como um "movimento". Em essência, métodos ágeis se desenvolveram em um esforço para sanar fraquezas reais e perceptíveis da engenharia de software convencional.

A respeito das mudanças em metodologias ágeis, é correto afirmar:

- a) As metodologias ágeis partem do princípio de que as mudanças ocorrerão sempre e os projetos devem se adequar a essas mudanças.
 - b) As metodologias ágeis preconizam que o cliente não se envolva no projeto.
 - c) As metodologias ágeis esperam uma fase finalizar para iniciar outra.
 - d) A metodologia ágil é centrada nos processos.
 - e) A metodologia ágil é avessa a mudanças.
2. A melhor forma de ser ágil é construir somente o que o cliente valoriza, e não mais que isso. O excesso de formalidade pode limitar o progresso do projeto. Por outro lado, o caos total, sem a utilização de processos, pode impedir que se alcancem os objetivos do projeto. O Scrum permite criar produtos mais bem adaptados à realidade do cliente de forma ágil.

A respeito dos métodos ágeis, considere as afirmativas a seguir:

- I - É importante envolver o cliente em todo o projeto.
- II - A metodologia ágil entende que as mudanças sempre ocorrerão e o projeto deve se adequar.
- III - Um projeto desenvolvido usando metodologias ágeis pode trazer uma flexibilidade muito maior.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

VAMOS PRATICAR

3. Um manifesto normalmente é associado a um movimento político emergente: ataca a velha guarda e sugere uma mudança revolucionária (espera-se que para melhor). De certa forma, é exatamente disso que trata o desenvolvimento ágil.

Assinale a alternativa que descreve um dos princípios das metodologias ágeis:

- a) Satisfação do cliente mediante a entrega contínua e adiantada de software.
- b) Entregar o software funcionando somente ao término do projeto
- c) Construir o projeto com um indivíduo desmotivado e dois motivados: eles vão motivá-lo.
- d) Preferir a comunicação por e-mail, pois tudo fica formalizado.
- e) A equipe deve ser gerenciada por um gerente de projetos.

REFERÊNCIAS

ACADEMIA. **As origens da metodologia ágil:** de onde saímos e onde estamos. c2024. Disponível em: https://www.academia.edu/104242490/As_origens_da_metodologia_%C3%A1gil_de_onde_sa%C3%ADmos_e_onde_estamos_Uma_revis%C3%A3o_sistem%C3%A1tica_da_literatura. Acesso em: 20 nov. 2024.

AGILE ALLIANCE. **Princípios por trás do Manifesto Ágil.** c2024. Disponível em: <https://agilemanifesto.org/iso/ptbr/principles.html>. Acesso em: 14 out. 2024.

DOS SANTOS SOARES, M. Metodologias ágeis Extreme Programming e Scrum para o desenvolvimento de software. **Revista Eletrônica de Sistemas de Informação**, v. 3, n. 1, 2004.

DIGITAL.AI. **15th State of Agile Report.** 2021. Disponível em: <https://digital.ai/resource-center/analyst-reports/state-of-agile-report>. Acesso em: 14 out. 2024.

MAESTRO VIRTUALE. **Administração científica:** teoria, história, características, princípios. c2024. Disponível em: <https://maestrovirtuale.com/administracao-cientifica-teoria-historia-caracteristicas-principios/>. Acesso em: 20 nov. 2024.

PEREIRA, P.; TORREÃO, P.; MARÇAL, A. S. **Entendendo Scrum para gerenciar projetos de forma ágil.** Mundo PM, v. 1, n. 14, p. 64-71, 2007.

PROJECT BUILDER. **Conheça a história do gráfico de Gantt.** 2024. Disponível em: <https://www.projectbuilder.com.br/blog/conheca-historia-do-grafico-de-gantt/>. Acesso em: 20 nov. 2024.

QUALYTEAM. **Método do caminho crítico.** 2024. Disponível em: <https://qualyteam.com/pb/blog/metodo-do-caminho-critico/>. Acesso em: 20 nov. 2024.

SCIELO. **As origens da metodologia ágil:** de onde saímos e onde estamos. 2024. Disponível em: <https://www.scielo.br/j/rae/a/Fkq59q8FCfMvqhLPbQpx3pt/>. Acesso em: 20 nov. 2024.

CONFIRA SUAS RESPOSTAS

1. Alternativa A.

O *Manifesto Ágil* e as metodologias ágeis, como Scrum e XP, reconhecem que as mudanças são inevitáveis no decorrer do projeto, e a capacidade de adaptar-se a essas mudanças é um dos pilares fundamentais para o sucesso de projetos ágeis. As metodologias ágeis enfatizam a participação ativa e constante do cliente durante todo o processo de desenvolvimento. O cliente desempenha um papel fundamental em fornecer feedback contínuo e validar o progresso do projeto em cada iteração. As metodologias ágeis utilizam ciclos iterativos e incrementais, em que diferentes fases do desenvolvimento (como design, codificação e teste) ocorrem simultaneamente, sem esperar o término completo de uma fase para iniciar outra. O foco das metodologias ágeis é nas pessoas e nas interações, mais do que nos processos e ferramentas. A metodologia ágil é muito aderente às mudanças, entendendo que as mudanças fazem parte.

2. Alternativa E.

I. Correta. Um dos pilares das metodologias ágeis é a colaboração contínua com o cliente. A participação ativa do cliente é fundamental para garantir que o produto atenda às necessidades e expectativas, permitindo ajustes rápidos conforme o feedback é fornecido durante o desenvolvimento.

II. Correta. As metodologias ágeis reconhecem que as mudanças são inevitáveis e encorajam a adaptação ao longo do projeto. A flexibilidade é um princípio-chave, o que permite que os projetos respondam rapidamente às mudanças nas necessidades do cliente ou no ambiente de negócios.

III. Correta. A flexibilidade é uma das principais vantagens das metodologias ágeis. Ao adotar ciclos curtos de desenvolvimento e promover revisões frequentes, a equipe pode ajustar rapidamente o projeto, garantindo que ele se mantenha alinhado com os objetivos e as expectativas em constante evolução do cliente.

3. Alternativa A.

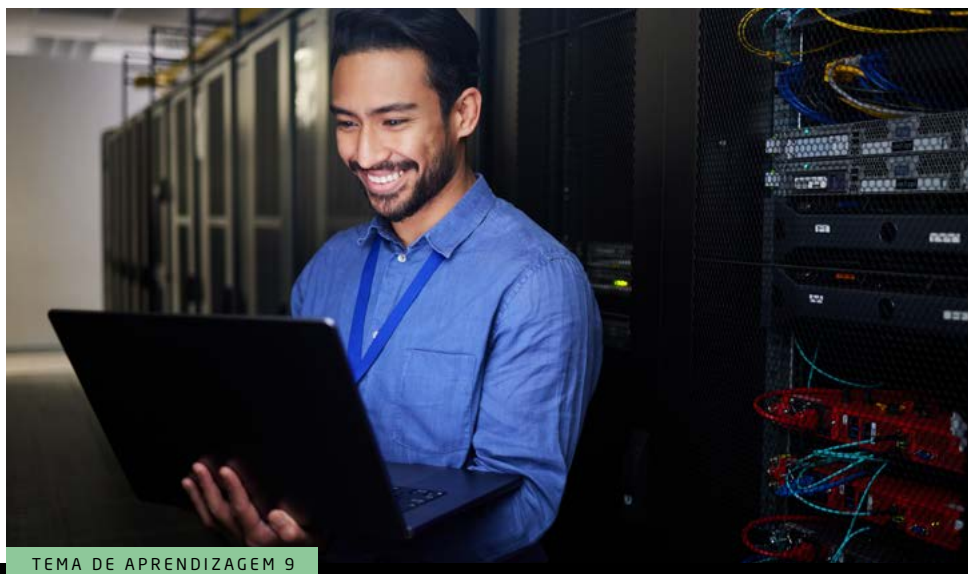
A. Correta. Esse princípio está alinhado com as metodologias ágeis, que priorizam entregas frequentes de software funcional para garantir a satisfação do cliente.

B. Incorreta. As metodologias ágeis incentivam entregas incrementais ao longo do desenvolvimento, e não apenas no final do projeto.

C. Incorreta. Embora a colaboração seja fundamental nas metodologias ágeis, a motivação da equipe não é tratada como algo que "contagia".

D. Incorreta. As metodologias ágeis enfatizam a comunicação direta e face a face como a forma mais eficaz de compartilhar informações.

E. Incorreta. Nas metodologias ágeis, as equipes são geralmente auto-organizadas, com os próprios membros gerenciando o trabalho. O foco está na colaboração e na distribuição de responsabilidades entre a equipe, sem a necessidade de um gerenciamento centralizado.



TEMA DE APRENDIZAGEM 9

ESTUDO DE CASO DE ENGENHARIA DE SOFTWARE

MINHAS METAS

- Identificar um planejamento.
- Planejar um projeto de software.
- Conhecer os riscos inerentes.
- Explorar a importância do trabalho em equipe.
- Entender a importância da comunicação.
- Compreender a relevância dos testes.
- Estudar a fase de implementação do projeto.

INICIE SUA JORNADA

Em ambientes corporativos, o planejamento para o desenvolvimento de um sistema de software é fundamental. Uma equipe de projeto deverá reduzir os custos, os riscos do projeto e utilizar os recursos de forma otimizada, visando entregar o projeto com sucesso e trazendo uma maior competitividade para a empresa.

Você, estudante, está se preparando para enfrentar essa realidade e compor times com esse cenário formado. Suponha que você foi contratado recentemente por uma empresa que atua no ramo de comércio eletrônico. Você e a sua equipe devem lançar um site novo e mais robusto para vender.

O site é de uma loja de departamentos a qual tem uma seção de roupas masculinas e femininas, seção de calçados e seção de acessórios. Essa loja também deve contar com formas de pagamento, tais como cartão de crédito e débito, PIX, boleto e transferência bancária. Contudo, você entrou na empresa agora e o projeto já está em andamento: ele se encontra com o cronograma atrasado, baixa qualidade de desenvolvimento, sem integrações financeiras e equipe desmotivada. Em outras palavras, o projeto está um caos. Diante desse cenário, você consegue imaginar como abordaria a questão?

Infelizmente, um projeto com uma série de problemas é relativamente comum no ambiente empresarial e você deve se preparar com técnicas para encarar esses desafios. O problema referente ao cronograma atrasado pode ser abordado, ao se inserir mais pessoas para trabalhar em horas extras. A baixa qualidade do desenvolvimento pode ser abordada com treinamentos em desenvolvimento para a equipe. Já as integrações financeiras podem ser desenvolvidas pelos desenvolvedores mais sêniores do projeto.

Refleta a respeito dessa abordagem e se esse seria o único caminho possível para se resolver a questão e colocar esse projeto novamente nos trilhos.



PLAY NO CONHECIMENTO

Neste podcast, abordaremos o tema *O desenvolvimento de software e a computação em nuvem*. Ouça-o para saber mais sobre o assunto. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal relembrar o que é um projeto de software? É interessante relembrar, pois são as etapas que ocorrem durante o planejamento do projeto de software e a posterior implementação. <https://youtu.be/Tj92zX-NELE>

DESENVOLVA SEU POTENCIAL

O sucesso de um projeto está intrinsecamente ligado à qualidade da execução de cada um dos artefatos envolvidos, os quais podem ser influenciados por uma série de variáveis. Por isso, as etapas devem ser bem gerenciadas e executadas com excelência, pois isso impacta diretamente o resultado.

Nesse contexto, exploraremos os principais aspectos de um projeto de software, desde o planejamento e análise até a implementação e os testes. Para ilustrar de forma prática, utilizaremos o *case* previamente definido.

Case a ser abordado: site de uma loja de departamentos

Esse site deve ter uma seção de roupas masculinas e femininas, seção de calçados e seção de acessórios. Além disso, o site deve contar com formas de pagamento, como cartão de crédito e débito, PIX, boleto e transferência bancária.

O PLANEJAMENTO E A ANÁLISE

A fase de planejamento em um projeto é uma das mais importantes e tem reflexos diretos nas fases seguintes, como na fase de execução. Embora a análise tenha início na fase de iniciação, ela é detalhada na fase de planejamento, onde se define o escopo.

A análise é uma das principais fases de um projeto de desenvolvimento. Nessa etapa, os requisitos são levantados mediante entrevistas com os principais *stakeholders* e os requisitos funcionais e não funcionais são modelados e documentados em formato de caso de uso ou história do usuário.



Além disso, o site deve contar com formas de pagamento, como cartão de crédito e débito, PIX, boleto e transferência bancária.

O levantamento dos requisitos deve ser feito de uma forma extremamente detalhada, identificando os detalhes de execução e os resultados esperados, pois esse levantamento será parte do escopo do projeto.

De acordo com Sommerville (2018), o processo de Engenharia de Requisitos segue as seguintes fases:

- **Especificação dos requisitos do negócio:** busca-se demonstrar as necessidades do negócio que levarão à execução do projeto.
- **Estudo de viabilidade:** nesse momento, avalia-se se o projeto é tecnicamente e economicamente viável, assim como o impacto para a organização.
- **Elicitação de requisitos de usuário:** nesse momento, são feitos entrevistas, questionários e reuniões para levantamento.
- **Prototipação:** permite a criação de modelos para facilitar a validação dos requisitos. Pode ser um rascunho, *mockup*, versão simplificada, diagramas ou outras formas de representação.
- **Elicitação de requisitos do sistema:** aqui, os requisitos são mais técnicos, momento em que as necessidades do usuário são incorporadas ao sistema.
- **Especificação e modelagem de requisitos de sistema:** nesta etapa, os requisitos são documentados e formalizados.
- **Revisões:** os requisitos são revisados com os diversos *stakeholders* do projeto.

O case da loja de departamentos: planejamento e análise

Assim como é possível observar, estudante, quando elaboramos um projeto de desenvolvimento de software, as técnicas mencionadas são fundamentais. No *case* relativo à loja de departamentos, foram realizadas entrevistas com a área de negócios e, com isso, foi possível compreender os requisitos de negócios e requisitos do usuário.

Em seguida, foram realizados *workshops* sucessivos com as áreas técnicas, a fim de entender os requisitos técnicos e avaliar a respectiva viabilidade, como a comunicação de um sistema legado com o novo sistema existente na empresa.

Em seguida, foram realizados workshops sucessivos com as áreas técnicas, a fim de entender os requisitos técnicos e avaliar a respectiva viabilidade



INDICAÇÃO DE FILME

Startup.com

Kaleil Isaza Tuzman e Tom Herman fizeram faculdade juntos e sonhavam ter um negócio próprio. Em maio de 1999, o sonho se tornou realidade e eles abriram a govworks.com, uma web *company* destinada a facilitar as relações entre as pessoas e os diversos serviços municipais. Esse documentário conta a breve história da govWorks e o modo como os problemas profissionais interferiram na amizade dos dois sócios.

O documentário mostra os desafios técnicos e empresariais enfrentados por uma *startup* durante o boom das empresas ponto-com.



Essas fases seguem um ciclo contínuo, sendo repetidas a cada iteração do projeto, caso a metodologia de gestão adotada seja iterativa, como no caso das abordagens ágeis.

DESIGN

A fase de design (projeto) ocorre como um preparativo à implementação. Nessa fase, o que foi definido na fase de análise do projeto é abordado sob uma ótica voltada à **implementação**. Em outras palavras, na fase de análise, responde-se “o que será elaborado” e, na fase de design, responde-se “como será implementado”.

Nessa fase, são definidos os itens de modelagem do sistema, como os diagramas UML. Um exemplo comum é o diagrama de casos de uso (*use case*), que ilustra o comportamento do sistema em relação a cada funcionalidade definida nos respectivos casos de uso.

A seguir, é possível observar a representação do diagrama de caso de uso da loja de departamentos. Nele, a escolha de produtos e a transação financeira são demonstrados.



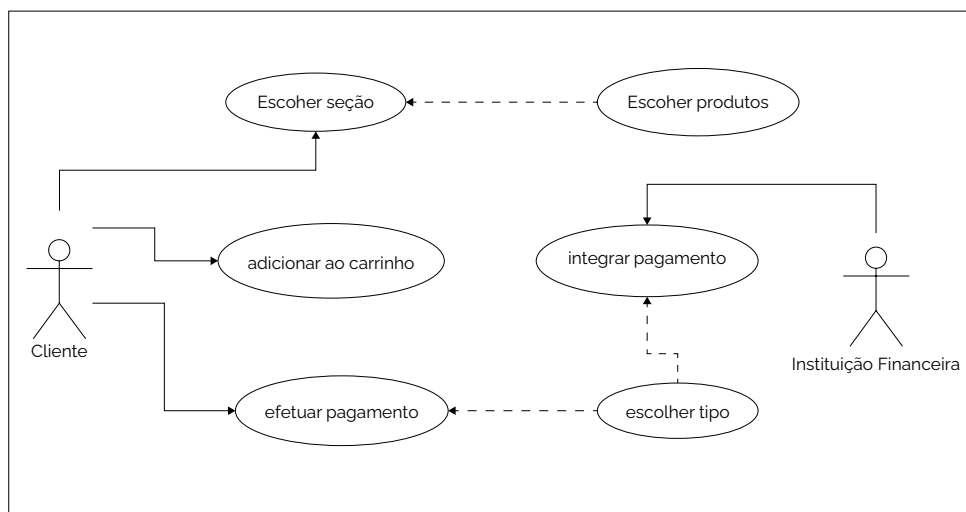


Figura 1 - Representação do diagrama de caso de uso da loja on-line proposta no início do tema
Fonte: o autor.

Descrição da Imagem: é exibido um boneco de palito representando o ator cliente. Há diversas setas saindo desse boneco e indo em direção aos casos de uso, representados por elipses e com os textos: “escolher ação”, “adicionar ao carrinho” e “efetuar pagamento”. As elipses “escolher ação” e “efetuar pagamento” estão ligadas a outras elipses: “escolher produtos” e “escolher tipo”. Do outro lado, encontra-se um boneco de palito representando a instituição financeira. Ele está ligado à elipse “integrar pagamento”. Fim da descrição.

Esse diagrama, estudante, é uma representação simplificada de uma interação entre os atores e o sistema. Em um projeto de software, a modelagem define um passo primordial para o sucesso do projeto, e a representação em diagramas auxilia muito na comunicação do projeto, pois, assim, todos se comunicam da mesma forma.

Segundo Pressman e Maxim (2016, p. 114), a modelagem consiste em:



Criar os modelos para entender melhor o que será construído. Quando a entidade for algo físico (por exemplo, um edifício, um avião, uma máquina), podemos construir um modelo que seja idêntico na forma e no formato, porém em menor escala. Entretanto, quando a entidade a ser construída for software, nosso modelo deve assumir uma forma diferente. Ele deve ser capaz de representar as informações que o software transforma, a arquitetura e as funções que permitem a transformação, as características que os usuários desejam e o comportamento do sistema à medida que a transformação ocorra.

Um projeto pode apresentar pontos em relação à baixa qualidade de desenvolvimento, por não ter uma documentação bem estabelecida e a equipe não se comunicar bem, lacuna que é preenchida pelos diagramas UML e relacionados.

O **case** da loja de departamentos: design

O diagrama de caso de uso representado mostra a área de escolha do produto e a transação financeira do site da loja de departamentos. Esse diagrama é acompanhado por documentos descritivos e outros diagramas UML, os quais ajudam a definir de forma clara o que precisa ser desenvolvido e como essa etapa é uma preparação ao desenvolvimento. Portanto, utilizamos o diagrama de caso de uso e as descrições do caso de uso (documento onde são descritos o fluxo básico da funcionalidade, o fluxo alternativo e o fluxo de exceção). Também foram realizadas entrevistas mais detalhadas com a área técnica.

Nessa fase, também voltamos a nossa atenção para a **arquitetura** do sistema, ao definirmos o modo como uma solução será disposta entre os elementos de hardware.

A seguir, são exibidas algumas questões que podem ser feitas para definir a arquitetura do sistema.

- **Disposição dos componentes:** qual parte do sistema ficará em cada servidor. Exemplos: classes de lógica de negócios em um servidor, acesso a dados em outro e banco de dados em um terceiro.
- **Integração:** devemos responder como as integrações ocorrerão, seja mediante uso de APIs ou fila de mensagens.
- **Definições de hardware e rede:** como o hardware será configurado (balanceamento de rede ou redundância) e conectividade em rede (local ou cloud).

Essas considerações são fundamentais, pois “a arquitetura de software deve modelar a estrutura de um sistema, bem como a maneira pela qual os dados e os componentes procedurais colaboram entre si” (Pressman; Maxim, 2016, p. 254).

Podemos citar como exemplo a arquitetura MVC, em que o software é separado em camadas que podem se dispersar por meio de servidores distintos.

A camada **view** é a responsável pela visualização, ou seja, o que você irá visualizar. Um exemplo são as telas da aplicação.

A camada **controller** é a responsável por representar a lógica de negócios da aplicação, os cálculos e outras operações, comunicando a visualização com a camada **model**, que é a camada responsável pelo acesso aos dados.

Questões a respeito do sistema ajudam a compor a arquitetura como um todo, definindo onde os componentes estarão dispostos e como eles irão interagir.

O case da loja de departamentos: arquitetura do sistema

A arquitetura MVC (model, view e controller) é altamente recomendada para a criação de uma loja on-line, principalmente devido à independência entre camadas de responsabilidades, facilitando a escalabilidade e a manutenção do software.

A arquitetura MVC consiste em banco de dados, model, controller e view, assim como visualizamos na Figura 2.

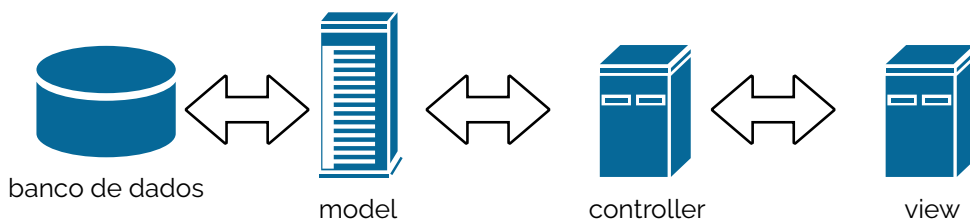


Figura 2 - Arquitetura MVC utilizada no caso abordado

Fonte: o autor.

Descrição da Imagem: são expostos quatro ícones de computadores representando cada um dos servidores. São eles: banco de dados, model, controller e view. Fim da descrição.

CODIFICAÇÃO

A fase de codificação é onde ocorre a **materialização do que foi definido até o momento**. Nela, o código do sistema é desenvolvido na linguagem escolhida e implementado sobre a arquitetura definida.

Nessa fase, os principais objetivos são:



- **Transformação do design em código:** classes, métodos e outras funcionalidades de programação são, de fato, implementadas.
- **Conformidade com o design e os padrões:** padrões a serem estabelecidos são extremamente necessários e o uso acaba por facilitar a comunicação e a manutenção ao longo do tempo.
- **Testes unitários:** os testes são realizados de forma individual, testando as pequenas partes do software. Por exemplo: é chamada uma função de pagamento. Então, os testes irão verificar se o retorno ocorreu como esperado.
- **Documentação:** a documentação é extremamente importante para um software e futuras manutenções, as quais devem ser sempre atualizadas e servir o propósito de auxiliar no entendimento.
- **CI - Continuous Integration (Integração Contínua):** ferramentas de CI permitem integrar constantemente o código ao repositório central, possibilitando a identificação de problemas de integração de forma antecipada.
- **Controle de versão:** esse ponto é fundamental para que o desenvolvimento tenha agilidade e seja realizado de forma organizada.

O case da loja de departamentos: codificação

O planejamento e o design especificado e detalhado, agora, ganham forma na codificação. No caso estudado, foi escolhida a linguagem PHP e banco de dados MySQL, por ser uma linguagem robusta, amplamente utilizada e um banco de dados gratuito, altamente escalável, como repositório de código e controlador de versão e de acessos pela equipe foi escolhido o Github, pois é uma plataforma amplamente utilizada e conhecida entre os desenvolvedores.

A codificação foi conduzida por uma equipe composta por cinco programadores seniores e com ampla experiência na área. Com isso, o desenvolvimento foi realizado sem maiores surpresas.

Todo o controle mencionado na fase de codificação do software trará uma maior estabilidade ao software desenvolvido, fazendo com que a qualidade do desenvolvimento aumente consideravelmente.

TESTES

Os testes representam uma parte importante no desenvolvimento das aplicações e garante que o produto atenda aos requisitos especificados, funcionando corretamente.

Eles são uma ferramenta de detecção dos erros encontrados no software utilizado. O ideal é que todas as fases e os cuidados mencionados sejam realizados para que o número de *bugs* seja reduzido.

Segundo Júlio Viegas (2025), alguns tipos de testes que podemos citar são:

- **Testes de caixa branca:** avalia aspectos internos do componente e como ele se comporta com o fluxo interno.
- **Testes de caixa preta:** verifica se o retorno do componente corresponde ao esperado.
- **Testes de regressão:** verificam se as funcionalidades inseridas não causaram *bugs* no restante do sistema.
- **Testes unitários:** são os testes realizados no desenvolvimento, em funções e componentes unitários.
- **Testes de integração:** verifica como os diferentes componentes do software trabalham integrados.

O *case* da loja de departamentos: testes

Além dos testes unitários realizados pelos desenvolvedores, foram feitos testes por uma equipe especializada em realizar cada etapa e tipo de teste mencionado. Com isso, os *bugs* foram detectados antes do lançamento do site. Além do mais, foram realizados testes de caixa branca, caixa preta e integração. A cada mudança, foram realizados os testes de regressão.

No projeto de loja on-line, uma boa sequência de testes se faz necessária, com foco, sobretudo, em transações financeiras e integração com o banco.

IMPLANTAÇÃO

Após as fases anteriores, o software está preparado para a implantação. Itens mencionados, como a documentação, são extremamente importantes para transferir o software projetado ao setor de manutenção, que será responsável por dar suporte ao software em questão.

A implantação do software é a instalação de fato. A esse processo, chamamos comumente de *deploy* do projeto.

Ferramentas de *deploy*, como integração contínua, também são muito importantes para que a integração seja feita.

Ferramentas, como Jenkins, GitLab CI/CD e CircleCI, podem ser utilizadas para minimizar problemas, como incompatibilidade, falhas de configuração ou outros bugs. Ações, como automação do *deploy*, documentação clara e a realização de testes, também contribuem para reduzir a probabilidade de ocorrência de *bugs* (O que [...], 2024).

O *case* da loja de departamentos: implantação

Para a implantação do caso da loja de departamento, foi utilizado o CircleCI. Assim, o repositório do Github foi utilizado. Esse software permitiu o *deploy* automático, sem a necessidade de ser uma operação manual.

**EM FOCO**

Assista agora ao Vídeo referente ao tema apresentado nessa aula. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Estudante, cada conceito abordado neste tema tem uma aplicação direta no ambiente de trabalho. Embora tenhamos utilizado como exemplo o site de compras on-line, esses princípios podem ser aplicados a qualquer projeto de desenvolvimento de software no mercado. A exemplo disso, se você retornar ao início de nosso tema, certamente, estará mais preparado para resolver os desafios propostos.

Dessa forma, um profissional de desenvolvimento que compreende a importância de práticas, como testes, modelagem e documentação, destaca-se em um mercado de trabalho competitivo.

As empresas buscam profissionais capazes de entregar soluções de qualidade de forma ágil e eficiente, otimizando recursos e minimizando custos. O desenvolvedor que domina esses conceitos e entende as necessidades de um software bem-executado se torna um ativo valioso para qualquer organização.

VAMOS PRATICAR

1. Os requisitos de tecnologia da informação demandados por pessoas, empresas e órgãos governamentais estão mais complexos a cada ano. Atualmente, equipes grandes desenvolvem programas de computador que, antigamente, eram desenvolvidos por um só indivíduo.

Faz parte do levantamento de requisitos na análise:

- a) Especificação dos requisitos do negócio.
 - b) Elaboração de diagrama de arquitetura.
 - c) Elaboração do mapa de testes de integração.
 - d) Especificação dos requisitos de arquitetura.
 - e) Especificação do servidor cloud.
2. Entender os requisitos de um problema está entre as tarefas mais difíceis enfrentadas por um engenheiro de software. Quando você pensa nisso pela primeira vez, entender claramente a Engenharia de Requisitos não parece tão difícil. Afinal, o cliente não sabe o que é necessário? Os usuários não deveriam ter um bom entendimento das características e funções que serão vantajosas? Surpreendentemente, em muitos casos, a resposta a essas perguntas é "não".

Considerando fases do processo de Engenharia de Requisitos, analise as afirmativas a seguir::

- I - Estudo de viabilidade: neste momento, avalia-se se o projeto é tecnicamente e economicamente viável, assim como o impacto para a organização.
- II - Elicitação de requisitos do sistema: os requisitos são mais técnicos, momento em que as necessidades do usuário são incorporadas ao sistema.
- III - Especificação e modelagem de requisitos de sistema: nessa etapa, os requisitos são documentados e formalizados.

É correto o que se afirma em:

- a) I, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III, apenas.
- e) I, II e III.

VAMOS PRATICAR

3. Ele deve ser capaz de representar as informações que o software transforma, a arquitetura e as funções que permitem a transformação, as características que os usuários desejam e o comportamento do sistema à medida que a transformação ocorre.

Na fase de design, definimos:

- a) Itens de modelagem do sistema e arquitetura.
- b) Requisitos iniciais.
- c) Requisitos de negócio.
- d) Necessidades de negócio.
- e) Cronograma do projeto.

REFERÊNCIAS

O QUE é deploy de software? **Programae**, 18 out. 2024. Disponível em: <https://programae.org.br/software/glossario/o-que-e-deploy-de-software/>. Acesso em: 8 jan. 2025.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma abordagem profissional. 8 ed. [S. L.]: McGraw Hill, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

VIEGAS, J. Teste de software: introdução, conceitos básicos e tipos de testes. **One Day Testing**. c2025. Disponível em: <https://blog.onedaytesting.com.br/teste-de-software/>. Acesso em: 8 jan. 2025.

CONFIRA SUAS RESPOSTAS

1. Alternativa A.

A alternativa A está correta, porque a especificação dos requisitos do negócio é uma atividade da fase de levantamento de requisitos, focada em identificar e documentar as necessidades e os objetivos do negócio que o sistema deve atender. A alternativa B está incorreta, porque a elaboração de um diagrama de arquitetura pertence à fase de design, onde a estrutura do sistema é planejada com base nos requisitos levantados. A alternativa C está incorreta, porque o mapa de testes de integração é criado após a definição do design, geralmente, na fase de planejamento e execução dos testes, para assegurar que os componentes do sistema funcionem juntos conforme esperado. A alternativa D está incorreta, porque a especificação dos requisitos de arquitetura é uma atividade da fase de design, onde os requisitos técnicos e estruturais do sistema são detalhados. A alternativa E está incorreta, porque a especificação do servidor cloud envolve decisões de infraestrutura e é uma etapa de planejamento técnico que ocorre após o levantamento dos requisitos. Geralmente, faz parte da fase de design ou implementação.

2. Alternativa E.

I. Correta. O estudo de viabilidade é a etapa inicial, momento em que se verifica se o projeto pode ser realizado dentro das limitações técnicas e financeiras, além de analisar o impacto potencial para a organização.

II. Correta. A fase de elicitação de requisitos inclui o levantamento de necessidades dos usuários, englobando tanto requisitos técnicos quanto funcionais e operacionais.

III. Correta. Na fase de especificação e modelagem, os requisitos levantados são documentados de forma detalhada e organizada, para que todos os envolvidos tenham um entendimento claro e comum.

3. Alternativa A.

A alternativa A está correta, porque, na fase de design, o objetivo é criar uma estrutura detalhada do sistema, que inclui a definição de arquitetura e modelagem dos componentes e interações. Essa fase abrange o desenvolvimento de diagramas e estruturas que servirão de guia para a implementação, garantindo que os requisitos sejam atendidos. A alternativa B está incorreta, porque os requisitos iniciais são identificados nas fases de levantamento e análise, anteriores ao design. A fase de design transforma esses requisitos em uma estrutura técnica do sistema, mas não se concentra em identificar novos requisitos. A alternativa C está incorreta, porque os requisitos de negócio são coletados e analisados nas primeiras fases do projeto, como levantamento e análise, e refletem as necessidades gerais da organização. A alternativa D está incorreta, porque as necessidades de negócio são entendidas e documentadas nas fases iniciais de levantamento e análise de requisitos, não na fase de design.